DARIN'S HANDMADE — построчный разбор всего кода

В этом документе разобран **каждый файл проекта, каждая функция и каждая строка**. Номера строк совпадают с файлами в репозитории: можно открыть файл в VS Code и идти по нему вместе с документом.

Как устроен разбор каждого файла:

- **Что это за файл** — одной-двумя фразами.
- **Кто подключает этот файл** — какие другие файлы делают `require(...)` или `import ... from` на него.
- **Что подключает сам файл** — каждая строка импорта: что берётся и откуда.
- **Каждая функция**: кто её вызывает, что она вызывает сама (с указанием файлов), что получает на вход и что возвращает. Потом код функции и объяснение **каждой строки** по номеру.
- Для больших кусков разметки (JSX — то, что рисуется на экране) объяснение идёт **блоками строк**: «стр. 96–117 — две вкладки Sign In / Create Account». Каждая строка с логикой (условие, обработчик клика, вывод данных) объяснена отдельно.

Обозначения: «стр. 12» — строка 12 этого файла. «стр. 12–20» — строки с 12 по 20. «→» — «вызывает» или «переходит в».

Часть 0. Карта: какой файл с каким связан

0.1 Сервер (папка back/)

```
index.js  (запуск сервера)
 ├ db/connect.js                      ← подключение к MongoDB
 ├ routes/authRoute.js     /api/auth
 │    ├ controllers/AuthController.js
 │    │    ├ models/userModel.js
 │    │    └ utils/email.js
 │    ├ middleware/validate.js
 │    └ middleware/authMiddleware.js
 │         └ models/userModel.js
 ├ routes/productsRoute.js /api/products
 │    ├ controllers/ProductController.js
 │    │    └ models/productModel.js
 │    ├ middleware/authMiddleware.js
 │    ├ middleware/roleMiddleware.js
 │    └ middleware/upload.js
 ├ routes/cartRoute.js     /api/cart
 │    ├ controllers/CartController.js
 │    │    └ models/cartModel.js
 │    └ middleware/authMiddleware.js
 ├ routes/ordersRoute.js   /api/orders
 │    ├ controllers/OrderController.js
 │    │    ├ models/orderModel.js
 │    │    ├ models/productModel.js
 │    │    └ utils/email.js
 │    ├ middleware/authMiddleware.js
 │    └ middleware/roleMiddleware.js
 └ controllers/OrderController.js  (stripeWebhook, /api/orders/webhook)

seed/products.js     ← запускается вручную, сайт его не вызывает
seed/makeManager.js  ← запускается вручную
```

0.2 Сайт (папка front/)

```
index.html  → src/main.jsx
main.jsx    → App.jsx, index.css, BrowserRouter
App.jsx
 ├ api.jsx                          ← axios, адрес сервера
 ├ context/CartContext.jsx          ← корзина для всех страниц (использует api.jsx)
 ├ shared/ui/Header.jsx             ← шапка (использует CartContext)
 ├ shared/ui/Footer.jsx             ← подвал
 └ pages/...                        ← 16 страниц, по одной на адрес
      большинство используют api.jsx (запросы) и/или CartContext (корзина)
```

0.3 Какая страница какой адрес сервера вызывает

Страница (front)	Запрос	Функция на сервере
`App.jsx`	`GET /api/auth/me`	`protect` → `getMe`

`LoginPage`	`POST /api/auth/signin`, `POST /api/auth/signup`	`signin`, `signup`
`ForgotPasswordPage`	`POST /api/auth/forgot-password`	`forgotPassword`
`ResetPasswordPage`	`POST /api/auth/reset-password`	`resetPassword`
`BeadedBagsPage`, `BraceletsPage`, `ManagerProductsPage`	`GET /api/products`	`getAllProducts`
`ProductDetailPage`	`GET /api/products/:id`	`getProductById`
`ManagerProductsPage`	`POST`, `PUT`, `DELETE /api/products`	`createProduct`, `updateProduct`, `deleteProduct`
`CartContext`	`GET`, `POST`, `PATCH`, `DELETE /api/cart`	`getCart`, `addToCart`, `updateQuantity`, `removeFromCart`, `clearCart`
`CheckoutPage`	`POST /api/orders/checkout-session`	`createCheckoutSession`
`OrderSuccessPage`	`GET /api/orders/by-session/:id`	`getOrderBySession`
`ManagerOrdersPage`	`GET /api/orders`, `GET /api/orders/stats`, `PATCH /api/orders/:id/status`	`getAllOrders`, `getStats`, `updateOrderStatus`
Stripe (не браузер)	`POST /api/orders/webhook`	`stripeWebhook`

Часть 1. Сервер: запуск и база данных

1.1 back/package.json

Что это за файл. Паспорт серверного проекта: список библиотек и команд запуска. Его читает команда `npm install` (скачивает библиотеки в `node_modules`) и `npm run ...` (запускает команды).

- **стр. 2–5** `"scripts"` — команды. `npm run dev` запускает `nodemon index.js` (сервер сам перезапускается при сохранении файла, удобно при разработке). `npm start` запускает `node index.js` — так сервер запускается на Render.
- **стр. 6–20** `"dependencies"` — библиотеки, нужные для работы: `bcryptjs` (хеш паролей), `cors` (разрешение запросов с другого адреса), `dotenv` (чтение `.env`), `express` (веб-сервер), `express-mongo-sanitize` и `xss-clean` (установлены, но в коде не используются), `express-rate-limit` (ограничение числа запросов), `helmet` (защитные заголовки), `jsonwebtoken` (токены входа), `mongoose` (работа с MongoDB), `multer` (загрузка файлов), `resend` (отправка писем), `stripe` (оплата). Знак `^` перед версией значит «эта версия или новее в пределах той же главной версии».
- **стр. 21–23** `"devDependencies"` — нужно только при разработке: `nodemon`.

1.2 back/index.js — запуск сервера

Что это за файл. Главный файл сервера. Его запускает команда `node index.js`. Он собирает сервер из частей: подключает защиту, маршруты, раздачу картинок и подключается к базе.

Кто подключает этот файл: никто — это точка входа.

Что подключает сам файл:

```
1  const express = require('express');
2  const cors = require('cors');
3  const path = require('path');
4  const helmet = require('helmet');
5  const rateLimit = require('express-rate-limit');
6  require('dotenv').config();
7
8  const connectDB = require('./db/connect');
9  const productsRoute = require('./routes/productsRoute.js');
10 const authRoute = require('./routes/authRoute.js');
11 const cartRoute = require('./routes/cartRoute.js');
12 const ordersRoute = require('./routes/ordersRoute.js');
13 const { stripeWebhook } = require('./controllers/OrderController');
```

- **стр. 1** — библиотека Express из `node_modules`. `express` — это функция, которая создаёт сервер.
- **стр. 2** — библиотека `cors`. Возвращает функцию-middleware, которая разрешает браузеру обращаться к серверу с другого адреса.
- **стр. 3** — встроенный модуль Node `path`, умеет правильно склеивать пути к папкам на любой системе (Windows, Linux).
- **стр. 4** — библиотека `helmet`, ставит защитные HTTP-заголовки.
- **стр. 5** — библиотека `express-rate-limit`, ограничивает число запросов с одного IP-адреса.

- **стр. 6** — библиотека `dotenv`: `.config()` читает файл `.env` и кладёт каждую строку `ИМЯ=значение` в `process.env.ИМЯ`. На Render файла `.env` нет — там эти значения задаются в настройках сервиса, и `process.env` заполняет сам Render.
- **стр. 8** — наш файл `db/connect.js`, функция подключения к базе.
- **стр. 9–12** — наши четыре роутера (файлы со списками адресов).
- **стр. 13** — из нашего `OrderController.js` берётся одна функция `stripeWebhook` (фигурные скобки — достать из объекта по имени).

```
15 const app = express();
16
17 // The site runs behind the hosting provider's proxy, so express has to trust
18 // it to read the real client IP (the rate limiters count per IP).
19 app.set('trust proxy', 1);
20
21 app.use(cors());
```

- **стр. 15** — вызов `express()` создаёт объект приложения `app`. Дальше все настройки делаются через него.
- **стр. 17–18** — комментарий: сайт работает за прокси хостинга, поэтому нужно доверять прокси, чтобы узнать настоящий IP посетителя.
- **стр. 19** — `app.set('trust proxy', 1)`: верить одному прокси перед сервером. Без этого Express видел бы IP прокси, и ограничитель запросов (стр. 34–58) считал бы всех посетителей одним человеком.
- **стр. 21** — `app.use(...)` подключает middleware ко **всем** запросам. `cors()` добавляет в ответы заголовок `Access-Control-Allow-Origin: *` — браузер разрешает сайту `darinshandmade-front.onrender.com` читать ответы сервера `darinshandmade.onrender.com`.

```
23 // Stripe sends the webhook with a signature over the raw body, so this one
24 // route has to be registered before express.json() parses the body away.
25 app.post('/api/orders/webhook', express.raw({ type: 'application/json' }), stripeWebhook);
26
27 app.use(express.json());
```

- **стр. 23–24** — комментарий: Stripe подписывает сырое тело запроса, поэтому этот адрес объявлен до разбора JSON.
- **стр. 25** — `app.post(адрес, функция1, функция2)`: на запрос `POST /api/orders/webhook` сначала выполнится `express.raw(...)` — он оставляет тело запроса «как есть» (набор байтов, Buffer) и кладёт в `req.body`. Потом `stripeWebhook` из `OrderController.js` проверит подпись по этим байтам.
- **стр. 27** — `express.json()` (встроен в Express): для всех остальных запросов превращает тело из текста JSON в объект JavaScript и кладёт в `req.body`. Без этой строки контроллеры получили бы `req.body === undefined`.

```
29 // Security headers. crossOriginResourcePolicy is off so the React app
30 // on another port can still load the product images from /uploads.
31 app.use(helmet({ crossOriginResourcePolicy: false }));
```

- **стр. 31** — `helmet(...)` возвращает middleware, которое добавляет к ответам защитные заголовки (например, запрет показывать сайт внутри чужой страницы). Настройка

`crossOriginResourcePolicy: false` выключает одно правило, иначе сайт с другого адреса не смог бы показать картинки товаров.

```
33 // General limit: 100 requests per 15 minutes from the same IP.
34 const limiter = rateLimit({
35   windowMs: 15 * 60 * 1000,
36   max: 100,
37   message: 'Too many requests from this IP, please try again later.'
38 });
39 app.use('/api/', limiter);
```

- **стр. 34** — `rateLimit({...})` создаёт middleware-счётчик.
- **стр. 35** — окно счёта: 15 минут в миллисекундах (15 × 60 × 1000).
- **стр. 36** — не больше 100 запросов за окно с одного IP.
- **стр. 37** — текст ответа, когда лимит превышен (код 429).
- **стр. 39** — `app.use('/api/', limiter)`: счётчик работает только для адресов, начинающихся с `/api/`.

```
41 // Stricter limit on the login routes, to block password guessing.
42 const authLimiter = rateLimit({
43   windowMs: 15 * 60 * 1000,
44   max: 10,
45   skipSuccessfulRequests: true,
46   message: 'Too many authentication attempts, please try again later.'
47 });
48 app.use('/api/auth/signin', authLimiter);
49 app.use('/api/auth/signup', authLimiter);
50 app.use('/api/auth/reset-password', authLimiter);
```

- **стр. 42–47** — второй, строгий счётчик: 10 попыток за 15 минут.
- **стр. 45** — `skipSuccessfulRequests: true`: удачные попытки (ответ 2xx) не считаются. Считаются только ошибки — так блокируется подбор пароля, а нормальный пользователь не страдает.
- **стр. 48–50** — счётчик подключён к трём адресам: вход, регистрация, смена пароля по ссылке.

```
52 // Asking for a reset link is limited harder: 5 per hour from the same IP.
53 const resetLimiter = rateLimit({
54   windowMs: 60 * 60 * 1000,
55   max: 5,
56   message: 'Too many password reset requests. Please try again in an hour.'
57 });
58 app.use('/api/auth/forgot-password', resetLimiter);
```

- **стр. 53–57** — третий счётчик: 5 запросов в час.
- **стр. 58** — только для запроса письма «забыли пароль», чтобы через сайт нельзя было засыпать чужую почту письмами.

```
60 // Product images are served as static files.
61 app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
```

- **стр. 61** — `express.static(папка)` отдаёт файлы из папки как есть. `__dirname` — встроенная переменная Node: папка текущего файла (`back`). `path.join(__dirname, 'uploads')` →

`back/uploads`. Запрос `/uploads/bags/pink.JPG` вернёт файл `back/uploads/bags/pink.JPG`. Так браузер получает картинки товаров.

```
63 app.use('/api/products', productsRoute);
64 app.use('/api/auth', authRoute);
65 app.use('/api/cart', cartRoute);
66 app.use('/api/orders', ordersRoute);
```

- **стр. 63–66** — подключение роутеров с префиксами. Например, всё, что начинается с `/api/products`, передаётся в `productsRoute`. Внутри роутера префикс отрезается: запрос `/api/products/123` там видится как `/123`.

```
68 app.get('/', (req, res) => {
69   res.send('<h2>DARINS HANDMADE API</h2>');
70 });
```

- **стр. 68–70** — на запрос главной страницы сервера (`GET /`) отвечает короткой надписью. Удобно проверить в браузере, что сервер живой.

```
72 // One place for errors that were thrown outside a controller, so the client
73 // always gets JSON back (a failed image upload would return HTML otherwise).
74 app.use((err, req, res, next) => {
75   if (err.name === 'MulterError') {
76     const message =
77       err.code === 'LIMIT_FILE_SIZE' ? 'Image is too large (max 8MB)' : 'Image upload failed';
78     return res.status(400).send({ message });
79   }
80   if (err.message === 'Only image files are allowed') {
81     return res.status(400).send({ message: err.message });
82   }
83   console.log(err.message);
84   res.status(500).send({ message: 'Server error' });
85 });
```

Обработчик ошибок. Express узнаёт его по **четырём** параметрам `(err, req, res, next)`. Сюда попадают ошибки, которые произошли в middleware и не были пойманы (чаще всего — при загрузке картинки в `upload.js`).

- **стр. 75** — `err.name === 'MulterError'` — ошибка библиотеки multer.
- **стр. 76–77** — тернарный оператор `условие ? да : нет`: если код ошибки `LIMIT_FILE_SIZE` (файл больше 8 МБ) — одно сообщение, иначе другое.
- **стр. 78** — ответ 400 с сообщением в JSON. `return` останавливает функцию.
- **стр. 80–82** — ошибка, которую бросает наш `fileFilter` в `upload.js` (стр. 32), если прислали не картинку.
- **стр. 83** — любую другую ошибку пишем в консоль сервера (видно в логах Render).
- **стр. 84** — клиенту — общий ответ 500, без подробностей (чтобы не раскрывать внутренности сервера).

```
87 const start = async () => {
88   try {
89     await connectDB(process.env.MONGO_URI);
90     const port = process.env.PORT || 5000;
91     app.listen(port, () =>
92       console.log(`Masad netunim connected. Server is listening on port ${port}...`));
93   } catch (error) {
94     console.log(error);
95   }
96 };
97
98 start();
```

Функция `start` — запуск сервера. Вызывается один раз на стр. 98.

- **стр. 87** — `async` — внутри можно ждать (`await`) долгие операции.
- **стр. 89** — `connectDB` из `db/connect.js` с адресом базы из `.env`. `await` ждёт, пока подключение установится. Если база недоступна — бросается ошибка и выполнение прыгает в `catch` (стр. 93).
- **стр. 90** — порт: из `process.env.PORT` (его задаёт Render), а если нет — 5000 (на своём компьютере).
- **стр. 91** — `app.listen(port, функция)` — начать принимать запросы. Функция вызовется, когда сервер готов.
- **стр. 92** — сообщение в консоль. «Masad netunim» — «база данных» на иврите.
- **стр. 93–95** — если подключиться к базе не удалось — печатаем ошибку. Сервер при этом не запускается: без базы он бесполезен.
- **стр. 98** — вызов `start()` — с этой строки всё и начинается.

1.3 back/db/connect.js — подключение к базе

Что это за файл. Одна функция, которая подключается к MongoDB. **Кто подключает:** `index.js` (стр. 8), `seed/products.js` (стр. 2), `seed/makeManager.js` (стр. 2). **Что подключает:** библиотеку `mongoose`.

```
1 const mongoose = require('mongoose');
2
3 const connectDB = (url) => {
4   return mongoose.connect(url);
5 };
6
7 module.exports = connectDB;
```

- **стр. 1** — библиотека Mongoose — «переводчик» между JavaScript и MongoDB.
- **стр. 3** — функция `connectDB` принимает адрес базы `url` (строка вида `mongodb+srv://логин:пароль@кластер/база`).
- **стр. 4** — `mongoose.connect(url)` открывает соединение и возвращает Promise (обещание результата). Тот, кто вызвал, ждёт его через `await`. После подключения **все модели** (User, Product, Cart, Order) сами используют это соединение.
- **стр. 7** — `module.exports` — отдать функцию наружу, чтобы другие файлы могли получить её через `require`.

Часть 2. Сервер: модели (как данные выглядят в базе)

Общее для всех моделей. `new mongoose.Schema({...})` — описание полей документа. `mongoose.model('Имя', схема)` создаёт **модель** — класс, у которого есть готовые методы Mongoose: `find`, `findOne`, `findById`, `create`, `findByIdAndUpdate`, `findByIdAndDelete`, `updateOne`, `aggregate` и другие. Мы эти методы не пишем — они приходят из библиотеки. Имя модели превращается в имя коллекции в базе: `'User'` → `users`, `'Product'` → `products`, `'Cart'` → `carts`, `'Order'` → `orders`.

Каждый документ в MongoDB автоматически получает поле `_id` — уникальный номер вида `65f1a2b3c4d5e6f7a8b9c0d1`.

2.1 back/models/userModel.js — пользователь

Кто подключает: `AuthController.js` (стр. 4), `authMiddleware.js` (стр. 2), `seed/makeManager.js` (стр. 3).

```
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: true
8      },
9      email: {
10       type: String,
11       required: true,
12       unique: true
13     },
14     password: {
15       type: String,
16       required: true,
17       // select: false keeps the password out of every normal query,
18       // so it is only loaded where we ask for it (in signin).
19       select: false
20     },
21     avatar: {
22       type: String,
23       default: ''
24     },
25     role: {
26       type: String,
27       default: 'user'
28     },
29     // Filled in only while a "forgot password" link is alive. The token is
30     // stored hashed, so even the database never holds the link itself.
31     resetPasswordToken: {
32       type: String,
33       default: null,
34       select: false
35     },
36     resetPasswordExpires: {
37       type: Date,
38       default: null,
39       select: false
40     }
41   },
42   {
43     timestamps: true
44   }
45 );
46
47 const User = mongoose.model('User', userSchema);
48 module.exports = User;
```

- **стр. 1** — подключаем Mongoose.
- **стр. 3** — создаём схему. Первый объект (стр. 4–41) — поля, второй (стр. 42–44) — настройки.
- **стр. 5–8** — `name`: строка, обязательна (`required: true` — без имени Mongoose не сохранит документ и бросит ошибку).
- **стр. 9–13** — `email`: строка, обязательна, `unique: true` — в базе создаётся уникальный индекс: двух пользователей с одной почтой быть не может.

- **стр. 14–20** — `password`: строка, обязательна. В базе здесь лежит **не пароль, а его хеш** (см. `AuthController.js`). **стр. 19** `select: false` — поле не возвращается в обычных запросах. Чтобы получить его, нужно явно попросить `.select('+password')` — это делается только при входе.
- **стр. 21–24** — `avatar`: ссылка на картинку профиля, по умолчанию пустая строка (`default: ''`).
- **стр. 25–28** — `role`: роль. По умолчанию `'user'` (обычный покупатель). Бывает `'manager'` и `'admin'`. Ставится только скриптом `seed/makeManager.js`.
- **стр. 31–35** — `resetPasswordToken`: хеш токена из ссылки «забыли пароль». По умолчанию `null`, скрыт (`select: false`).
- **стр. 36–40** — `resetPasswordExpires`: до какого момента действует ссылка (тип `Date` — дата и время). Скрыт.
- **стр. 43** — `timestamps: true` — Mongoose сам добавит поля `createdAt` (когда создан) и `updatedAt` (когда изменён последний раз).
- **стр. 47** — создаём модель `User` по схеме. Коллекция в базе — `users`.
- **стр. 48** — отдаём модель наружу.

2.2 back/models/productModel.js — товар

Кто подключает: `ProductController.js` (стр. 3), `OrderController.js` (стр. 4), `seed/products.js` (стр. 3).

```
3  const productSchema = new mongoose.Schema(
4    {
5      file_name: { type: String, required: true },      // стр. 5–8
9      model_name: { type: String, required: true },     // стр. 9–12
13     color_palette: { type: [String], default: [] },   // стр. 13–16
17     details: { type: [String], default: [] },         // стр. 17–20
21     style: { type: [String], default: [] },           // стр. 21–24
25     category: { type: String, required: true, enum: ['bags', 'bracelets'] }, // стр. 25–29
30     price: { type: Number },                          // стр. 30–32
35     stock: { type: Number, default: null, min: 0 }    // стр. 35–39
40   },
41   { timestamps: true }
44 );
46 const Product = mongoose.model('Product', productSchema);
47 module.exports = Product;
```

(В файле каждое поле записано в несколько строк; выше они сжаты, номера указаны в комментариях.)

- **стр. 5–8** — `file_name`: путь к картинке на сервере, например `uploads/bags/pink.JPG`. Обязателен.
- **стр. 9–12** — `model_name`: название товара. Обязательно.
- **стр. 13–16** — `color_palette`: `[String]` — **массив строк**, например `['Pink', 'White']`. По умолчанию пустой массив.
- **стр. 17–20** — `details`: массив строк — особенности (например «Pearl handle»). Первый элемент показывается в карточке каталога.
- **стр. 21–24** — `style`: массив строк — стиль («Glamorous»). Первый элемент показывается меткой в карточке.
- **стр. 25–29** — `category`: обязательна, `enum: ['bags', 'bracelets']` — разрешены **только** эти два значения. Любое другое — ошибка при сохранении.

- **стр. 30–32** — `price`: число, не обязательно.
- **стр. 33–34** — комментарий: сколько осталось; `null` — остаток не отслеживается, товар всегда в наличии.
- **стр. 35–39** — `stock`: число, по умолчанию `null`, `min: 0` — отрицательное значение не сохранится.
- **стр. 42–44** — `timestamps: true` — поля `createdAt`, `updatedAt`.
- **стр. 46** — модель `Product`, коллекция `products`.
- **стр. 47** — экспорт.

2.3 back/models/cartModel.js — корзина

Кто подключает: `CartController.js` (стр. 1).

```
3  const cartSchema = new mongoose.Schema(
4    {
5      // Every user has one cart. ref tells mongoose which collection
6      // to pull from when we call populate().
7      user: {
8        type: mongoose.Schema.Types.ObjectId,
9        ref: 'User',
10       required: true
11     },
12     items: [
13       {
14         product: {
15           type: mongoose.Schema.Types.ObjectId,
16           ref: 'Product',
17           required: true
18         },
19         quantity: {
20           type: Number,
21           default: 1
22         }
23       }
24     ]
25   },
26   {
27     timestamps: true
28   }
29 );
30
31 const Cart = mongoose.model('Cart', cartSchema);
32 module.exports = Cart;
```

- **стр. 7–11** — `user`: тип `ObjectId` — то есть `_id` другого документа. `ref: 'User'` — указывает, что это `_id` из модели User. Обязательно: корзина всегда чья-то.
- **стр. 12–24** — `items`: массив позиций. Каждая позиция — объект из двух полей:
  - **стр. 14–18** — `product`: `_id` товара, `ref: 'Product'`. В корзине хранится **только ссылка**, не сам товар. Сам товар подставляется методом `populate` в `CartController.js` (стр. 6).
  - **стр. 19–22** — `quantity`: количество, по умолчанию 1.
- **стр. 27** — `timestamps`.
- **стр. 31** — модель `Cart`, коллекция `carts`.

2.4 back/models/orderModel.js — заказ

Кто подключает: `OrderController.js` (стр. 3).

```
3  const orderItemSchema = new mongoose.Schema({
4    product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', default: null },
5    name: { type: String, required: true },
6    category: { type: String, default: '' },
7    image: { type: String, default: '' },
8    price: { type: Number, required: true },
9    quantity: { type: Number, required: true, min: 1 },
10   custom: {
11     size: String,
12     strap: String,
13     chainColor: String,
14     color: String,
15   },
16 }, { _id: false });
```

Схема одной позиции заказа (стр. 3–16). Это отдельная схема, которая потом вставляется в заказ как массив.

- **стр. 4** — `product`: ссылка на товар каталога. Для сумки из конструктора — `null` (такого товара в базе нет).
- **стр. 5** — `name`: название **на момент покупки**. Копия, а не ссылка: если потом товар переименуют или удалят, заказ останется правильным.
- **стр. 6–7** — категория и картинка, тоже копии.
- **стр. 8** — `price`: цена за штуку на момент покупки. Обязательна.
- **стр. 9** — `quantity`: количество, минимум 1.
- **стр. 10–15** — `custom`: параметры сумки из конструктора — размер, ремешок, цвет цепочки, цвет бусин. Для обычного товара пусто.
- **стр. 16** — `{ _id: false }` — позициям не нужен свой `_id`, они существуют только внутри заказа.

```
18 const orderSchema = new mongoose.Schema({
19   orderNumber: { type: String, required: true, unique: true },
20   user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
21   customer: {
22     name: { type: String, required: true },
23     email: { type: String, required: true },
24     phone: { type: String, default: '' },
25   },
26   shipping: {
27     address: { type: String, default: '' },
28     city: { type: String, default: '' },
29     country: { type: String, default: '' },
30   },
31   deliveryMethod: { type: String, enum: ['delivery', 'pickup'], default: 'delivery' },
32   items: { type: [orderItemSchema], required: true },
33   subtotal: { type: Number, required: true },
34   total: { type: Number, required: true },
35   currency: { type: String, default: 'usd' },
36   status: {
37     type: String,
38     enum: ['pending', 'paid', 'in_production', 'shipped', 'completed', 'cancelled'],
39     default: 'pending',
40   },
41   payment: {
42     provider: { type: String, default: 'stripe' },
43     sessionId: { type: String, default: null },
44     paymentIntentId: { type: String, default: null },
45     paidAt: { type: Date, default: null },
46   },
47   notes: { type: String, default: '' },
48 }, { timestamps: true });
49
50 module.exports = mongoose.model('Order', orderSchema);
```

- **стр. 19** — `orderNumber`: номер вида `DH-20261021-4821`, уникальный.
- **стр. 20** — `user`: кто заказал. `null`, если покупал гость без входа.
- **стр. 21–25** — `customer`: вложенный объект — имя и почта (обязательны), телефон.
- **стр. 26–30** — `shipping`: адрес доставки. Для самовывоза пустой.
- **стр. 31** — `deliveryMethod`: только `delivery` (доставка) или `pickup` (самовывоз).
- **стр. 32** — `items`: массив позиций по схеме `orderItemSchema` (стр. 3–16). Обязателен.
- **стр. 33–34** — `subtotal` (сумма позиций) и `total` (итого; сейчас равны, доставка бесплатная).
- **стр. 35** — валюта, по умолчанию доллары.
- **стр. 36–40** — `status`: один из шести статусов. Новый заказ — `pending` (ждёт оплаты). После оплаты — `paid`, дальше менеджер переводит: `in_production` → `shipped` → `completed`, или `cancelled`.
- **стр. 41–46** — `payment`: данные оплаты: `sessionId` — id страницы оплаты Stripe, `paymentIntentId` — id самого платежа, `paidAt` — когда оплачено.
- **стр. 47** — `notes`: комментарий покупателя.
- **стр. 48** — `timestamps` — `createdAt` используется для сортировки и статистики.
- **стр. 50** — сразу создаём модель `Order` и отдаём наружу (в одну строку, без отдельной переменной).

Часть 3. Сервер: middleware (проверки перед контроллером)

Что такое middleware. Функция с тремя параметрами `(req, res, next)`, которую Express вызывает **до** контроллера. Она либо пропускает запрос дальше вызовом `next()`, либо сама отвечает (`res.status(...).send(...)`) — и тогда контроллер не вызывается.

- `req` (request) — запрос: `req.body` (тело), `req.params` (части адреса вида `:id`), `req.query` (параметры после `?`), `req.headers` (заголовки), `req.file` (загруженный файл), `req.user` (пользователь — его кладёт `protect`).
- `res` (response) — ответ: `res.status(код)`, `res.json(объект)`, `res.send(...)`.
- `next` — функция «передать дальше».

3.1 back/middleware/authMiddleware.js — `protect`, проверка входа

Что это за файл. Проверяет, что пользователь вошёл: в запросе есть правильный токен. **Кто подключает:** `authRoute.js` (стр. 4), `productsRoute.js` (стр. 9), `cartRoute.js` (стр. 3), `ordersRoute.js` (стр. 11). **Что подключает:** `jsonwebtoken` (стр. 1), модель `User` (стр. 2).

Функция `protect` (стр. 6–40)

- **Кто вызывает:** Express, перед каждым защищённым адресом: `GET /api/auth/me`, все `/api/cart`, создание/изменение/удаление товара, все адреса заказов для менеджера.
- **Что вызывает сама:** `jwt.verify` (библиотека `jsonwebtoken`), `User.findById` (модель `userModel.js` → база).
- **Получает:** заголовок `Authorization: Bearer <токен>` — его добавляет функция `authHeader` во фронте (`api.jsx`).
- **Результат:** кладёт пользователя в `req.user` и вызывает `next()`; или отвечает 401.

```
6  const protect = async (req, res, next) => {
7    const authHeader = req.headers.authorization;
8
9    // The header looks like: "Bearer eyJhbGciOi..."
10   if (!authHeader || !authHeader.startsWith('Bearer ')) {
11     return res.status(401).send({ message: 'No token, authorization denied' });
12   }
13
14   try {
15     const token = authHeader.split(' ')[1];
16
17     // verify() throws if the token was changed or has expired.
18     const decoded = jwt.verify(token, process.env.JWT_SECRET);
19
20     // The role is read from the database and not from the token, so a manager
21     // who lost the permission cannot keep using a token that is still valid.
22     const user = await User.findById(decoded.id);
23     if (!user) {
24       return res.status(401).send({ message: 'User no longer exists' });
25     }
26
27     req.user = {
28       id: user._id,
29       name: user.name,
30       email: user.email,
31       avatar: user.avatar,
32       role: user.role
33     };
34
35     next();
36   } catch (error) {
37     console.log(error.message);
38     res.status(401).send({ message: 'Token is invalid or has expired' });
39   }
40 };
```

- **стр. 6** — `async`, потому что внутри будет запрос к базе (`await`).
- **стр. 7** — берём заголовок `Authorization` из запроса. Express хранит имена заголовков маленькими буквами, поэтому `authorization`.
- **стр. 10** — если заголовка нет **или** он не начинается со слова `Bearer` — токена нет.
- **стр. 11** — ответ 401 «нет токена». `return` — дальше не идём.
- **стр. 14** — `try` — если что-то ниже бросит ошибку, перейдём в `catch` (стр. 36).
- **стр. 15** — `split(' ')` режет строку `"Bearer eyJ..."` по пробелу на массив `["Bearer", "eyJ..."]`, `[1]` — второй элемент, сам токен.
- **стр. 18** — `jwt.verify(токен, секрет)` проверяет подпись токена секретом из `.env` и срок жизни. Если токен подделан или истёк — **бросает ошибку** (→ стр. 36). Если всё хорошо — возвращает данные, которые положили в токен при входе: `{ id, name, email, role, iat, exp }`.
- **стр. 22** — ищем пользователя в базе по `id` из токена. Пароль не придёт — у него `select: false`.
- **стр. 23–25** — пользователя удалили, а токен остался — 401.
- **стр. 27–33** — создаём `req.user` — свой объект с нужными полями. Все следующие функции в цепочке (`restrictTo`, контроллер) видят его. Роль берётся **из базы** (стр. 32), а не из токена.

- **стр. 35** — `next()` — всё в порядке, передаём запрос дальше.
- **стр. 36–39** — токен плохой: пишем причину в консоль и отвечаем 401.
- **стр. 42** `module.exports = protect;` — отдаём функцию наружу.

3.2 back/middleware/roleMiddleware.js — `restrictTo`, проверка роли

Кто подключает: `productsRoute.js` (стр. 10), `ordersRoute.js` (стр. 12). **Что подключает:** ничего.

```
1  // Runs after protect. It allows the request only if the role of the user
2  // is one of the roles we listed, for example restrictTo('manager', 'admin').
3  const restrictTo = (...roles) => {
4    return (req, res, next) => {
5      if (!req.user || !roles.includes(req.user.role)) {
6        return res.status(403).send({ message: 'Manager access only' });
7      }
8      next();
9    };
10 };
11
12 module.exports = restrictTo;
```

- **Кто вызывает:** роутеры при запуске сервера: `restrictTo('manager', 'admin')`. Полученный middleware Express вызывает на каждый запрос менеджера.
- **стр. 3** — `restrictTo` — **функция, которая возвращает функцию**. `...roles` (rest-параметр) собирает все переданные аргументы в массив: `restrictTo('manager', 'admin')` → `roles = ['manager', 'admin']`.
- **стр. 4** — возвращаем настоящий middleware `(req, res, next)`. Он «помнит» `roles` из внешней функции — это называется **замыкание**.
- **стр. 5** — если `req.user` нет (забыли поставить `protect` перед) **или** роли пользователя нет в списке (`includes` — есть ли элемент в массиве)…
- **стр. 6** — …ответ 403 «доступ только менеджеру». 403 — «знаем, кто ты, но нельзя» (в отличие от 401 — «не знаем, кто ты»).
- **стр. 8** — роль подходит — пропускаем дальше.
- **стр. 12** — экспорт.

3.3 back/middleware/upload.js — загрузка картинок (multer)

Что это за файл. Настраивает библиотеку multer: куда сохранять картинку, как назвать, что пропускать. **Кто подключает:** `productsRoute.js` (стр. 11). **Что подключает:** `multer` (стр. 1), встроенные `path` (стр. 2) и `fs` (стр. 3, работа с файлами на диске).

```
5  // multer takes the picture out of the form and saves it on the disk.
6  const storage = multer.diskStorage({
7    // Which folder the picture goes to.
8    destination: (req, file, cb) => {
9      const dir = path.join(__dirname, '..', 'uploads');
10     fs.mkdirSync(dir, { recursive: true });
11     cb(null, dir);
12   },
13
14   // Which name it gets. Date.now() is added so two pictures with the
15   // same original name do not overwrite each other.
16   filename: (req, file, cb) => {
17     const ext = path.extname(file.originalname).toLowerCase();
18     const base = path
19       .basename(file.originalname, ext)
20       .replace(/[^a-z0-9]/gi, '-')
21       .toLowerCase()
22       .slice(0, 40);
23     cb(null, `${base}-${Date.now()}${ext}`);
24   }
25 });
```

- **стр. 6** — `multer.diskStorage({...})` — «хранилище на диске». Получает две функции: куда класть и как назвать. Multer сам вызывает их для каждого файла.
- **стр. 8** — функция `destination`. Параметры: `req` — запрос, `file` — данные файла (`originalname`, `mimetype`), `cb` — callback, функция, которую нужно вызвать с ответом.
- **стр. 9** — путь к папке: `__dirname` (это `back/middleware`) + `..` (подняться на уровень) + `uploads` → `back/uploads`.
- **стр. 10** — `fs.mkdirSync` создаёт папку, если её нет. `recursive: true` — не ругаться, если уже есть.
- **стр. 11** — `cb(null, dir)`: первый аргумент — ошибка (`null` — ошибки нет), второй — папка.
- **стр. 16** — функция `filename` — какое имя дать файлу.
- **стр. 17** — `path.extname('Pink Bag.JPG')` → `.JPG`, `.toLowerCase()` → `.jpg`.
- **стр. 18–19** — `path.basename(имя, расширение)` → имя без расширения: `Pink Bag`.
- **стр. 20** — регулярное выражение `/[^a-z0-9]/gi`: всё, что **не** буква и не цифра (`^` внутри скобок — «не»), заменить на `-`. `g` — везде, `i` — без учёта регистра. `Pink Bag` → `Pink-Bag`.
- **стр. 21** — в нижний регистр: `pink-bag`.
- **стр. 22** — обрезать до 40 символов.
- **стр. 23** — итоговое имя: `pink-bag-1718000000000.jpg`. `Date.now()` — текущее время в миллисекундах, поэтому два файла с одинаковым названием не перезапишут друг друга.

```
27 // Only real pictures are accepted, so nobody can upload a script.
28 const fileFilter = (req, file, cb) => {
29   if (/^image\//.test(file.mimetype)) {
30     return cb(null, true);
31   }
32   cb(new Error('Only image files are allowed'), false);
33 };
```

- **стр. 28** — `fileFilter` — функция-фильтр, multer вызывает её до сохранения.

- **стр. 29** — `file.mimetype` — тип файла, например `image/jpeg`. Регулярное выражение `/^image\//` — «начинается с `image/`». `.test(...)` возвращает `true` или `false`.
- **стр. 30** — картинка — `cb(null, true)` — принять.
- **стр. 32** — не картинка — отдаём ошибку. Её поймает обработчик ошибок в `index.js` (стр. 80) и ответит 400.

```
35 const upload = multer({
36   storage,
37   fileFilter,
38   limits: { fileSize: 8 * 1024 * 1024 }
39 });
40
41 module.exports = upload;
```

- **стр. 35–39** — создаём сам объект multer с тремя настройками. `storage` и `fileFilter` — короткая запись `storage: storage`. `limits.fileSize` — максимум 8 МБ (8 × 1024 × 1024 байт). Больше — ошибка `LIMIT_FILE_SIZE`, обрабатывается в `index.js` (стр. 77).
- **стр. 41** — экспорт. В роутере используется как `upload.single('image')` — «в форме одно поле-файл с именем `image`». После этого контроллер получает файл в `req.file`, а текстовые поля формы — в `req.body`.

3.4 back/middleware/validate.js — проверка форм входа и регистрации

Кто подключает: `authRoute.js` (стр. 3). **Что подключает:** ничего.

```
4  const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
5
6  const NAME_MIN = 2;
7  const NAME_MAX = 60;
8  const EMAIL_MAX = 254;
9  const PASSWORD_MIN = 8;
10 const PASSWORD_MAX = 128;
11
12 const isString = (value) => typeof value === 'string';
13
14 const fail = (res, message) => res.status(400).send({ message });
```

- **стр. 4** — регулярное выражение для почты. Читается так: `^` начало, `[^\s@]+` — один или больше символов, которые не пробел и не `@`, потом `@`, снова «не пробел и не @», потом точка `\.`, снова «не пробел и не @», `$` конец. Подходит `anna@mail.com`, не подходит `anna@mail` или `anna@mail.com`.
- **стр. 6–10** — константы-ограничения: имя 2–60 символов, почта до 254 (максимум по стандарту), пароль 8–128.
- **стр. 12** — `isString(value)` — вспомогательная функция: `true`, если значение — строка. Нужна, потому что в JSON можно прислать число или объект вместо строки.
- **стр. 14** — `fail(res, message)` — короткий способ ответить 400 с сообщением. `{ message }` — то же, что `{ message: message }`.

```
16 // Returns the clean email, or null when it is not a valid address.
17 const checkEmail = (email) => {
18   const trimmed = email.trim().toLowerCase();
19   if (trimmed.length > EMAIL_MAX || !EMAIL_RE.test(trimmed)) return null;
20   return trimmed;
21 };
```

- **Функция `checkEmail`** — вызывается из `validateSignup` (стр. 36).
- **стр. 18** — `trim()` убирает пробелы по краям, `toLowerCase()` — в нижний регистр (`Anna@Mail.com` → `anna@mail.com`), чтобы одна почта не регистрировалась дважды разным регистром.
- **стр. 19** — слишком длинная или не подходит под шаблон — `null`.
- **стр. 20** — иначе возвращаем очищенную почту.

```
23 // Validate the signup form
24 const validateSignup = (req, res, next) => {
25   const { name, email, password } = req.body;
26
27   if (!isString(name) || !isString(email) || !isString(password)) {
28     return fail(res, 'Invalid data format');
29   }
30
31   const trimmedName = name.trim();
32   if (trimmedName.length < NAME_MIN || trimmedName.length > NAME_MAX) {
33     return fail(res, `Name must be between ${NAME_MIN} and ${NAME_MAX} characters`);
34   }
35
36   const cleanEmail = checkEmail(email);
37   if (!cleanEmail) {
38     return fail(res, 'Invalid email');
39   }
40
41   if (password.length < PASSWORD_MIN || password.length > PASSWORD_MAX) {
42     return fail(res, `Password must be between ${PASSWORD_MIN} and ${PASSWORD_MAX} characters`);
43   }
44
45   // The controller receives the cleaned values.
46   req.body.name = trimmedName;
47   req.body.email = cleanEmail;
48   next();
49 };
```

Функция `validateSignup` — middleware перед `signup`. Вызывается из `authRoute.js` (стр. 8) на `POST /api/auth/signup`.

- **стр. 25** — деструктуризация: достаём три поля из тела запроса.
- **стр. 27–29** — если хоть одно — не строка, ответ 400.
- **стр. 31** — имя без пробелов по краям.
- **стр. 32–34** — длина имени вне 2–60 — ответ 400. Сообщение собирается шаблонной строкой с подстановкой чисел.
- **стр. 36–39** — проверка почты через `checkEmail`; `null` — ответ 400.
- **стр. 41–43** — длина пароля вне 8–128 — ответ 400. Верхняя граница нужна, потому что bcrypt на очень длинных строках работает медленно.

- **стр. 46–47** — записываем очищенные имя и почту обратно в `req.body` — контроллер получит уже чистые данные.
- **стр. 48** — всё хорошо — дальше, в `signup`.

```
51 // Validate the signin form
52 const validateSignin = (req, res, next) => {
53   const { email, password } = req.body;
54
55   if (!isString(email) || !isString(password)) {
56     return fail(res, 'Invalid data format');
57   }
58   if (!email.trim() || !password) {
59     return fail(res, 'Email and password are required');
60   }
61   if (password.length > PASSWORD_MAX) {
62     return fail(res, 'Invalid data format');
63   }
64
65   req.body.email = email.trim().toLowerCase();
66   next();
67 };
```

Функция `validateSignin` — middleware перед `signin`. Вызывается из `authRoute.js` (стр. 9).

- **стр. 55–57** — почта и пароль должны быть строками.
- **стр. 58–60** — пустая почта (после удаления пробелов) или пустой пароль — 400.
- **стр. 61–63** — слишком длинный пароль — 400.
- **стр. 65** — почту приводим к тому же виду, как при регистрации, иначе `Anna@mail.com` не нашлась бы.
- **стр. 66** — дальше, в `signin`.
- **стр. 69–72** — экспорт обеих функций объектом.

Часть 4. Сервер: отправка писем

4.1 back/utils/email.js — функция `sendMail`

Кто подключает: `AuthController.js` (стр. 5, письмо «сброс пароля»), `OrderController.js` (стр. 5, письма о заказе). **Что подключает:** библиотеку `resend` (стр. 1).

```
1  const { Resend } = require('resend');
...
9  const resend = process.env.RESEND_API_KEY ? new Resend(process.env.RESEND_API_KEY) : null;
...
14 const FROM = process.env.MAIL_FROM || "DARIN'S HANDMADE <onboarding@resend.dev>";
```

- **стр. 1** — из библиотеки `resend` берём класс `Resend`.
- **стр. 3–8** — комментарий: все письма проекта идут через этот файл; Resend выбран, потому что работает через обычный HTTPS-запрос, а не через почтовый сервер (SMTP).
- **стр. 9** — если в `.env` есть ключ `RESEND_API_KEY` — создаём объект отправителя `new Resend(ключ)`, иначе `null`.
- **стр. 11–13** — комментарий: пока свой домен не подтверждён в Resend, можно отправлять только с их тестового адреса.
- **стр. 14** — адрес «От кого»: из `.env` (`MAIL_FROM`), а если нет — тестовый адрес Resend.

```
16 const sendMail = async ({ to, subject, html }) => {
17   // No key on this machine: write the letter to the console instead of
18   // failing, so the site keeps working in development.
19   if (!resend) {
20     console.log(`[email skipped - no RESEND_API_KEY] "${subject}" -> ${to}`);
21     return;
22   }
23
24   // The SDK does not throw on a rejected message; it reports the problem
25   // in the error field, so it has to be checked by hand.
26   const { data, error } = await resend.emails.send({ from: FROM, to, subject, html });
27
28   if (error) {
29     throw new Error(error.message || 'Resend refused the message');
30   }
31
32   return data;
33 };
34
35 module.exports = { sendMail };
```

Функция `sendMail`.

- **Кто вызывает:** `forgotPassword` (AuthController, стр. 114), `sendCustomerReceipt` и `sendManagerReport` (OrderController, стр. 218 и 232).
- **Что вызывает:** `resend.emails.send` — запрос в сервис Resend через интернет.
- **Получает:** объект `{ to, subject, html }` — кому, тема, текст письма в HTML.
- **стр. 16** — параметр сразу разбирается на три переменные (деструктуризация в параметрах).

- **стр. 19–22** — ключа нет: печатаем в консоль, что письмо пропущено, и выходим без ошибки. Так сайт работает и без настроенной почты.
- **стр. 26** — отправляем письмо. Библиотека возвращает объект `{ data, error }`; достаём оба поля.
- **стр. 28–30** — Resend не бросает ошибку сам, а кладёт её в `error`. Поэтому проверяем вручную и бросаем `throw new Error(...)` — тот, кто вызвал, поймает её в своём `try/catch`.
- **стр. 32** — возвращаем данные об отправке (id письма).
- **стр. 35** — экспорт объектом `{ sendMail }`.

Часть 5. Сервер: маршруты (какой адрес какую функцию вызывает)

Общее. `express.Router()` создаёт отдельный «мини-сервер» для группы адресов. Метод `router.get/post/put/patch/delete(адрес, функция1, функция2, ..., контроллер)` — на этот метод и адрес выполнить функции **слева направо**. Всё между адресом и последней функцией — middleware. Префикс (например `/api/auth`) добавляется в `index.js`.

`:id`, `:productId`, `:sessionId` в адресе — **параметры**: часть адреса, которая может быть любой. Контроллер получает её в `req.params.id` и т.п.

5.1 back/routes/authRoute.js — вход и регистрация (`/api/auth`)

```
1  const express = require('express');
2  const { signup, signin, forgotPassword, resetPassword, getMe } =
require('../controllers/AuthController');
3  const { validateSignup, validateSignin } = require('../middleware/validate');
4  const protect = require('../middleware/authMiddleware');
5
6  const router = express.Router();
7
8  router.post('/signup', validateSignup, signup);
9  router.post('/signin', validateSignin, signin);
10
11 // Password reset: the first route emails a link, the second one uses it.
12 router.post('/forgot-password', forgotPassword);
13 router.post('/reset-password', resetPassword);
14
15 // protect runs first and puts the logged in user on req.user.
16 router.get('/me', protect, getMe);
17
18 module.exports = router;
```

- **стр. 1** — Express.
- **стр. 2** — пять функций-контроллеров из `AuthController.js`.
- **стр. 3** — две функции проверки из `validate.js`.
- **стр. 4** — `protect` из `authMiddleware.js`.
- **стр. 6** — создаём роутер.
- **стр. 8** — `POST /api/auth/signup` → `validateSignup` → `signup`. Вызывает: `LoginPage.jsx` (режим регистрации).
- **стр. 9** — `POST /api/auth/signin` → `validateSignin` → `signin`. Вызывает: `LoginPage.jsx` (режим входа).
- **стр. 12** — `POST /api/auth/forgot-password` → `forgotPassword`. Вызывает: `ForgotPasswordPage.jsx`.
- **стр. 13** — `POST /api/auth/reset-password` → `resetPassword`. Вызывает: `ResetPasswordPage.jsx`.
- **стр. 16** — `GET /api/auth/me` → `protect` → `getMe`. Вызывает: `App.jsx` при каждой смене токена.
- **стр. 18** — экспорт роутера; подключается в `index.js` (стр. 64).

5.2 back/routes/productsRoute.js — товары (`/api/products`)

```
1  const express = require('express');
2  const {
3    createProduct,
4    getAllProducts,
5    getProductById,
6    updateProduct,
7    deleteProduct
8  } = require('../controllers/ProductController');
9  const protect = require('../middleware/authMiddleware');
10 const restrictTo = require('../middleware/roleMiddleware');
11 const upload = require('../middleware/upload');
12
13 const router = express.Router();
14
15 // Anyone may read the catalog.
16 router.get('/', getAllProducts);
17 router.get('/:id', getProductById);
18
19 // Adding, editing and deleting a product is for the manager only.
20 // upload.single('image') reads the picture out of the form.
21 router.post('/', protect, restrictTo('manager', 'admin'), upload.single('image'), createProduct);
22 router.put('/:id', protect, restrictTo('manager', 'admin'), upload.single('image'),
updateProduct);
23 router.delete('/:id', protect, restrictTo('manager', 'admin'), deleteProduct);
24
25 module.exports = router;
```

- **стр. 2–8** — пять контроллеров из `ProductController.js`.
- **стр. 9–11** — три middleware: вход, роль, загрузка файла.
- **стр. 16** — `GET /api/products` → `getAllProducts`. Открыто всем. Вызывают: `BeadedBagsPage`, `BraceletsPage`, `ManagerProductsPage`.
- **стр. 17** — `GET /api/products/:id` → `getProductById`. Вызывает: `ProductDetailPage`.
- **стр. 21** — `POST /api/products` (создать товар): `protect` (вошёл?) → `restrictTo` (менеджер или админ?) → `upload.single('image')` (сохранить картинку из поля `image`) → `createProduct`. Вызывает: `ManagerProductsPage` (новый товар).
- **стр. 22** — `PUT /api/products/:id` (изменить): та же цепочка → `updateProduct`. Вызывает: `ManagerProductsPage` (кнопка Edit → Save).
- **стр. 23** — `DELETE /api/products/:id` → `protect` → `restrictTo` → `deleteProduct`. Картинка не нужна, поэтому без `upload`. Вызывает: `ManagerProductsPage` (кнопка Delete).
- **стр. 25** — экспорт; подключается в `index.js` (стр. 63).

5.3 back/routes/cartRoute.js — корзина (`/api/cart`)

```
1  const express = require('express');
2  const { getCart, addToCart, removeFromCart, updateQuantity, clearCart } =
require('../controllers/CartController');
3  const protect = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  // The cart always belongs to the logged in user, so every route is protected.
8  router.get('/', protect, getCart);
9  router.post('/', protect, addToCart);
10 router.patch('/:productId', protect, updateQuantity);
11 router.delete('/:productId', protect, removeFromCart);
12 router.delete('/', protect, clearCart);
13
14 module.exports = router;
```

Все адреса под `protect`: корзина на сервере есть только у вошедшего пользователя. Все вызываются из `CartContext.jsx`.

- **стр. 8** — `GET /api/cart` → `getCart` — получить корзину. Из `fetchCart`.
- **стр. 9** — `POST /api/cart` → `addToCart` — добавить товар (тело: `{ productId }`). Из `addToCart` и при переносе гостевой корзины.
- **стр. 10** — `PATCH /api/cart/:productId` → `updateQuantity` — изменить количество (тело: `{ quantity }`). Из `updateQuantity`.
- **стр. 11** — `DELETE /api/cart/:productId` → `removeFromCart` — убрать одну позицию. Из `removeFromCart`.
- **стр. 12** — `DELETE /api/cart` (без id) → `clearCart` — очистить всё. Из `clearCart`.
- **стр. 14** — экспорт; подключается в `index.js` (стр. 65).

5.4 back/routes/ordersRoute.js — заказы (`/api/orders`)

```
1  const express = require('express');
2  const router = express.Router();
3  const {
4    createCheckoutSession,
5    getOrderBySession,
6    getAllOrders,
7    getOrderById,
8    updateOrderStatus,
9    getStats,
10 } = require('../controllers/OrderController');
11 const protect = require('../middleware/authMiddleware');
12 const restrictTo = require('../middleware/roleMiddleware');
13
14 // Reports and status changes are for the manager only.
15 const manager = [protect, restrictTo('manager', 'admin')];
16
17 // A guest may order too, so starting the payment is open to everyone.
18 router.post('/checkout-session', createCheckoutSession);
19 router.get('/by-session/:sessionId', getOrderBySession);
20
21 router.get('/', manager, getAllOrders);
22 router.get('/stats', manager, getStats);
23 router.patch('/:id/status', manager, updateOrderStatus);
24 router.get('/:id', manager, getOrderById);
25
26 module.exports = router;
```

- **стр. 2** — роутер создан сразу после подключения Express.
- **стр. 3–10** — шесть контроллеров из `OrderController.js`. (Седьмой — `stripeWebhook` — подключён прямо в `index.js`, стр. 25, потому что ему нужен особый разбор тела.)
- **стр. 15** — `manager` — массив из двух middleware. Express понимает массив так же, как перечисление через запятую. Так не нужно повторять `protect, restrictTo(...)` четыре раза.
- **стр. 18** — `POST /api/orders/checkout-session` → `createCheckoutSession`. **Без защиты** — купить может и гость. Вызывает: `CheckoutPage`.
- **стр. 19** — `GET /api/orders/by-session/:sessionId` → `getOrderBySession`. Без защиты. Вызывает: `OrderSuccessPage`.
- **стр. 21** — `GET /api/orders` → `getAllOrders` (только менеджер). Вызывает: `ManagerOrdersPage`.
- **стр. 22** — `GET /api/orders/stats` → `getStats`. Вызывает: `ManagerOrdersPage`. Стоит **выше** `/:id` (стр. 24): Express проверяет адреса сверху вниз, и иначе `/stats` подошёл бы под `/:id` с `id = 'stats'`.
- **стр. 23** — `PATCH /api/orders/:id/status` → `updateOrderStatus`. Вызывает: `ManagerOrdersPage` (выпадающий список статуса).
- **стр. 24** — `GET /api/orders/:id` → `getOrderById`. Сейчас со страниц не вызывается, но доступен менеджеру.
- **стр. 26** — экспорт; подключается в `index.js` (стр. 66).

Часть 6. Сервер: контроллеры (что делает каждый адрес)

Общее для всех контроллеров. Контроллер — функция `async (req, res) => {...}`. Она вызывается последней в цепочке роутера. Почти каждая устроена одинаково:

```
const имя = async (req, res) => {
  try {
    // 1. достать данные из req.body / req.params / req.query / req.user
    // 2. проверить их, при ошибке — return res.status(400 или 404).send({ message })
    // 3. обратиться к базе через модель (await Модель.метод(...))
    // 4. ответить res.status(200 или 201).json({ data: ... })
  } catch (error) {
    console.log(error.message);              // ошибку видно в логах сервера
    res.status(500).send({ message: ... });  // клиент получает 500
  }
};
```

Строки `try {`, `} catch (error) {`, `console.log(error.message);` и `res.status(500)...` есть в каждой функции. Ниже они не расписываются заново каждый раз — смысл всегда такой: «если что-то упало (например, база недоступна) — записать причину в консоль и ответить 500».

6.1 back/controllers/AuthController.js — регистрация, вход, пароль

Кто подключает: `authRoute.js` (стр. 2).

Что подключает:

```
1 const bcrypt = require('bcryptjs');
2 const jwt = require('jsonwebtoken');
3 const crypto = require('crypto');
4 const User = require('../models/userModel.js');
5 const { sendMail } = require('../utils/email');
```

- **стр. 1** — `bcryptjs` — библиотека хеширования паролей. Методы: `bcrypt.hash` (сделать хеш), `bcrypt.compare` (сравнить пароль с хешем).
- **стр. 2** — `jsonwebtoken` — создание токенов: `jwt.sign`.
- **стр. 3** — `crypto` — встроен в Node. Методы: `crypto.randomBytes` (случайные байты), `crypto.createHash` (хеш SHA-256).
- **стр. 4** — модель `User` → коллекция `users`.
- **стр. 5** — `sendMail` из `utils/email.js`.

Константа и вспомогательные функции (стр. 7–38)

```
7  // How long a password reset link stays valid.
8  const RESET_MINUTES = 60;
9
10 // The link we email is the raw token, but only its hash is saved. Someone
11 // who reads the database still cannot build a working link.
12 const hashToken = (raw) => crypto.createHash('sha256').update(raw).digest('hex');
```

- **стр. 8** — ссылка для сброса пароля живёт 60 минут.
- **стр. 12** — **функция `hashToken(raw)`**: делает хеш SHA-256 от строки. `createHash('sha256')` — выбрать алгоритм, `.update(raw)` — передать данные, `.digest('hex')` — получить результат строкой из 64 шестнадцатеричных символов. Вызывается из `forgotPassword` (стр. 106) и `resetPassword` (стр. 153). Хеш нельзя превратить обратно в токен.

```
14 // Blocks asking for a second link within two minutes of the first one.
15 const sentRecently = (user) => {
16   if (!user.resetPasswordExpires) return false;
17   const issuedAt = new Date(user.resetPasswordExpires).getTime() - RESET_MINUTES * 60 * 1000;
18   return Date.now() - issuedAt < 2 * 60 * 1000;
19 };
```

Функция `sentRecently(user)` — была ли ссылка отправлена меньше 2 минут назад. Вызывается из `forgotPassword` (стр. 104).

- **стр. 16** — ссылки ещё не было — `false`.
- **стр. 17** — когда ссылка была выдана: срок окончания минус 60 минут. `getTime()` — дата в миллисекундах.
- **стр. 18** — сейчас минус момент выдачи меньше 2 минут → `true` (новую ссылку не шлём).

```
21 // Builds the JWT. The details inside it are what the client gets back,
22 // and the secret is what proves the token came from our server.
23 const createToken = (user) => {
24   return jwt.sign(
25     { id: user._id, name: user.name, email: user.email, role: user.role },
26     process.env.JWT_SECRET,
27     { expiresIn: '1h' }
28   );
29 };
```

Функция `createToken(user)` — создаёт токен входа. Вызывается из `signup` (стр. 57) и `signin` (стр. 85).

- **стр. 24** — `jwt.sign(данные, секрет, настройки)` возвращает строку-токен из трёх частей через точку: заголовок, данные, подпись.
- **стр. 25** — что кладём внутрь: id, имя, почта, роль. Эти данные может **прочитать** кто угодно, но **изменить** нельзя — пароль сюда не кладём.
- **стр. 26** — секрет из `.env`. Подпись считается с ним; без секрета подделать подпись невозможно.
- **стр. 27** — токен действует 1 час.

```
31 // The user details we are willing to send to the client (never the password).
32 const publicUser = (user) => ({
33   id: user._id,
34   name: user.name,
35   email: user.email,
36   avatar: user.avatar,
37   role: user.role
38 });
```

Функция `publicUser(user)` — выбирает из документа пользователя только безопасные поля. Круглые скобки вокруг `{...}` — стрелочная функция сразу возвращает объект. Вызывается из

`signup` (стр. 58) и `signin` (стр. 86). Пароль сюда не попадает никогда.

Функция `signup` — регистрация (стр. 41–64)

- **Кто вызывает:** `authRoute.js` стр. 8, `POST /api/auth/signup`, после `validateSignup`. С фронта — `LoginPage.jsx`, режим Create Account.
- **Что вызывает:** `User.findOne` и `User.create` (база), `bcrypt.hash`, `createToken`, `publicUser`.
- **Получает:** `req.body` = `{ name, email, password, avatar }`. **Возвращает:** `201 { token, user }` или `400 Email already exists`.

```
41 const signup = async (req, res) => {
42   try {
43     const { name, email, password, avatar } = req.body;
44
45     const exists = await User.findOne({ email });
46
47     if (exists) {
48       return res.status(400).send({ message: 'Email already exists' });
49     }
50
51     // The password is never stored as it was typed, only its hash.
52
53     const hashed = await bcrypt.hash(password, 10);
54
55     const user = await User.create({ name, email, password: hashed, avatar: avatar || '' });
56
57     const token = createToken(user);
58     res.status(201).json({ token, user: publicUser(user) });
59
60   } catch (error) {
61     console.log(error.message);
62     res.status(500).send({ message: error.message });
63   }
64 };
```

- **стр. 43** — достаём поля формы (уже очищенные в `validateSignup`).
- **стр. 45** — `User.findOne({ email })` — ищем пользователя с такой почтой. `{ email }` — короткая запись `{ email: email }`. Результат — документ или `null`.
- **стр. 47–49** — нашёлся — почта занята, ответ 400.
- **стр. 53** — `bcrypt.hash(password, 10)` — хеш пароля. `10` — «стоимость»: 2^{10} = 1024 раунда вычислений, чтобы подбор был медленным. bcrypt сам добавляет случайную **соль**, поэтому одинаковые пароли дают разные хеши. Результат — строка вида `$2b$10$...`, в ней записаны и алгоритм, и стоимость, и соль, и хеш.
- **стр. 55** — `User.create({...})` создаёт и сохраняет документ в базе. В поле `password` кладём **хеш**. `role` не передаём — будет `'user'` по умолчанию, поэтому стать менеджером через регистрацию нельзя. `avatar || ''` — если аватара нет, пустая строка.
- **стр. 57** — создаём токен — пользователь сразу вошёл.
- **стр. 58** — ответ 201 (Created) с токеном и безопасными данными пользователя.

Функция `signin` — вход (стр. 67–91)

- **Кто вызывает:** `authRoute.js` стр. 9, `POST /api/auth/signin`, после `validateSignin`. С фронта — `LoginPage.jsx`, режим Sign In.
- **Что вызывает:** `User.findOne(...).select('+password')` (база), `bcrypt.compare`, `createToken`, `publicUser`.
- **Возвращает:** `200 { token, user }` или `400 Invalid credentials`.

```
67 const signin = async (req, res) => {
68   try {
69     const { email, password } = req.body;
70
71     // select('+password') is needed because the model hides the password.
72     const user = await User.findOne({ email }).select('+password');
73
74     if (!user) {
75       return res.status(400).send({ message: 'Invalid credentials' });
76     }
77
78     // compare() hashes the typed password and checks it against the stored hash.
79     const isMatch = await bcrypt.compare(password, user.password);
80
81     if (!isMatch) {
82       return res.status(400).send({ message: 'Invalid credentials' });
83     }
84
85     const token = createToken(user);
86     res.status(200).json({ token, user: publicUser(user) });
87   } catch (error) {
88     console.log(error.message);
89     res.status(500).send({ message: error.message });
90   }
91 };
```

- **стр. 69** — почта и пароль из формы.
- **стр. 72** — разберём по частям:
  - `User` — модель из `userModel.js` (подключена на стр. 4);
  - `.findOne({ email })` — метод, который модель получила от Mongoose: найти **первый** документ, у которого поле `email` равно введённой почте;
  - `.select('+password')` — у пароля в схеме `select: false`, поэтому обычно он не приходит. Знак `+` значит «добавь это скрытое поле в результат». Пароль нужен только здесь — чтобы сравнить;
  - `await` — ждём ответа базы. Результат — документ или `null`.
- **стр. 74–76** — нет такой почты — 400 «Invalid credentials».
- **стр. 79** — `bcrypt.compare(введённый, хеш_из_базы)`: берёт из сохранённого хеша соль и стоимость, хеширует ими введённый пароль и сравнивает два хеша. Результат `true` или `false`. Пароль при этом **не расшифровывается** — это невозможно.
- **стр. 81–83** — не совпал — 400 с **тем же** сообщением, что и при неверной почте. Так нельзя узнать, зарегистрирована ли почта.
- **стр. 85–86** — всё верно: токен + данные, ответ 200.

Функция `forgotPassword` — отправить ссылку для сброса (стр. 94–141)

- **Кто вызывает:** `authRoute.js` стр. 12, `POST /api/auth/forgot-password`. С фронта — `ForgotPasswordPage.jsx`.
- **Что вызывает:** `User.findOne` (база), `sentRecently`, `crypto.randomBytes`, `hashToken`, `user.save()` (база), `sendMail` (`utils/email.js` → Resend).
- **Возвращает:** всегда `200` с одинаковым сообщением.

```
94  const forgotPassword = async (req, res) => {
95    try {
96      const { email } = req.body;
97      if (!email) {
98        return res.status(400).send({ message: 'Email is required' });
99      }
100
101     const user = await User.findOne({ email: String(email).trim().toLowerCase() })
102       .select('+resetPasswordExpires');
103
104     if (user && !sentRecently(user)) {
105       const rawToken = crypto.randomBytes(32).toString('hex');
106       user.resetPasswordToken = hashToken(rawToken);
107       user.resetPasswordExpires = Date.now() + RESET_MINUTES * 60 * 1000;
108       await user.save();
109
110       const base = req.headers.origin || process.env.CLIENT_URL || 'http://localhost:5173';
111       const resetUrl = `${base}/reset-password?token=${rawToken}`;
112
113       try {
114         await sendMail({
115           to: user.email,
116           subject: "Reset your DARIN'S HANDMADE password",
117           html: `...текст письма с кнопкой-ссылкой resetUrl...`
128         });
129       } catch (mailError) {
130         console.log(mailError.message);
131       }
132     }
133
134     // The same answer is sent whether the email exists or not, so nobody
135     // can use this form to find out who is registered on the site.
136     res.status(200).send({ message: 'If an account exists for that email, a reset link has been
sent.' });
137   } catch (error) {
138     console.log(error.message);
139     res.status(500).send({ message: 'Could not process the request' });
140   }
141 };
```

- **стр. 96–99** — почта обязательна.
- **стр. 101** — ищем пользователя. `String(email)` — на случай, если прислали не строку; `trim().toLowerCase()` — как при регистрации.
- **стр. 102** — добавляем скрытое поле `resetPasswordExpires` — оно нужно для `sentRecently`.
- **стр. 104** — только если пользователь есть **и** ссылку не отправляли последние 2 минуты.
- **стр. 105** — `crypto.randomBytes(32)` — 32 случайных байта, `.toString('hex')` — строкой из 64 символов. Это **сырой токен** — он уйдёт в письме.

- **стр. 106** — в базу пишем **только хеш** токена.
- **стр. 107** — срок: сейчас + 60 минут (в миллисекундах).
- **стр. 108** — `user.save()` — сохранить изменения документа в базе.
- **стр. 110** — адрес сайта: откуда пришёл запрос (`origin`), иначе из `.env`, иначе локальный.
- **стр. 111** — ссылка в письме: `https://сайт/reset-password?token=<сырой токен>`.
- **стр. 113–131** — отправка письма в своём `try/catch`: если почта не ушла, ответ пользователю всё равно не меняется (ошибку видно только в консоли).
- **стр. 117–127** — HTML-текст письма: заголовок, пояснение, кнопка-ссылка `resetUrl`, ссылка текстом на случай, если кнопка не работает.
- **стр. 136** — ответ **всегда одинаковый**, есть такая почта или нет. Иначе через эту форму можно было бы проверять, кто зарегистрирован.

Функция `resetPassword` — новый пароль по ссылке (стр. 144–171)

- **Кто вызывает:** `authRoute.js` стр. 13, `POST /api/auth/reset-password`. С фронта — `ResetPasswordPage.jsx`.
- **Что вызывает:** `hashToken`, `User.findOne` (база), `bcrypt.hash`, `user.save()`.
- **Получает:** `{ token, password }`.

```
144 const resetPassword = async (req, res) => {
145   try {
146     const { token, password } = req.body;
147     if (!token || typeof password !== 'string' || password.length < 8) {
148       return res.status(400).send({ message: 'Invalid token or password too short (min 8)' });
149     }
150
151     // The link is good only if the hash matches and the hour has not passed.
152     const user = await User.findOne({
153       resetPasswordToken: hashToken(token),
154       resetPasswordExpires: { $gt: Date.now() }
155     });
156
157     if (!user) {
158       return res.status(400).send({ message: 'Reset link is invalid or has expired' });
159     }
160
161     user.password = await bcrypt.hash(password, 10);
162     user.resetPasswordToken = null;
163     user.resetPasswordExpires = null;
164     await user.save();
165
166     res.status(200).send({ message: 'Password updated. You can now sign in.' });
167   } catch (error) {
168     console.log(error.message);
169     res.status(500).send({ message: error.message });
170   }
171 };
```

- **стр. 146** — токен из ссылки и новый пароль.
- **стр. 147–149** — нет токена, пароль не строка или короче 8 — 400.
- **стр. 152–155** — ищем пользователя, у которого **одновременно**: хеш токена совпадает (стр. 153) **и** срок больше текущего момента (стр. 154, `$gt` — оператор MongoDB «больше чем»).

- **стр. 157–159** — не нашли — ссылка неверная или просрочена.
- **стр. 161** — новый пароль хешируем и записываем.
- **стр. 162–163** — стираем токен и срок — ссылка становится одноразовой.
- **стр. 164** — сохраняем.
- **стр. 166** — ответ 200.

Функция `getMe` — кто я (стр. 174–177)

```
174 const getMe = async (req, res) => {
175   // protect already checked the token and put the user on req.user.
176   res.status(200).json({ user: req.user });
177 };
```

- **Кто вызывает:** `authRoute.js` стр. 16, `GET /api/auth/me`, после `protect`. С фронта — `App.jsx` (стр. 55–63).
- **стр. 176** — просто возвращает `req.user`, который положил `protect`. Так фронт узнаёт актуальную роль из базы.
- **стр. 179–185** — экспорт пяти функций объектом.

6.2 back/controllers/ProductController.js — товары

Кто подключает: `productsRoute.js` (стр. 2–8). **Что подключает:** `path` (стр. 1), `fs` (стр. 2), модель `Product` (стр. 3).

Вспомогательные функции (стр. 5–29)

```
5  // The form sends color_palette / details / style as one string
6  // ("Pink, White"), so we turn it back into an array before saving.
7  const toArray = (value) => {
8    if (Array.isArray(value)) return value;
9    if (typeof value !== 'string') return [];
10   return value.split(',').map((item) => item.trim()).filter(Boolean);
11 };
```

Функция `toArray(value)` — превращает строку из формы в массив. Вызывается из `createProduct` (стр. 46–48) и `updateProduct` (стр. 99–101).

- **стр. 8** — уже массив — вернуть как есть (`Array.isArray` — встроенная проверка).
- **стр. 9** — не строка (например, поле не прислали) — пустой массив.
- **стр. 10** — `"Pink, White,"` → `split(',')` → `["Pink", " White", ""]` → `map(item => item.trim())` → `["Pink", "White", ""]` → `filter(Boolean)` (оставить только «непустые») → `["Pink", "White"]`.

```
13 // The stock field arrives from the form as text. null means the manager
14 // left it empty, so this product is not stock tracked.
15 const parseStock = (value) => {
16   if (value === undefined || value === null || String(value).trim() === '') return null;
17   const number = Math.floor(Number(value));
18   return Number.isFinite(number) ? Math.max(0, number) : null;
19 };
```

Функция `parseStock(value)` — превращает поле «остаток» в число или `null`. Вызывается из `createProduct` (стр. 49) и `updateProduct` (стр. 102).

- **стр. 16** — поле пустое — `null` (остаток не отслеживается).
- **стр. 17** — `Number("5")` → 5; `Math.floor` — отбросить дробную часть.
- **стр. 18** — `Number.isFinite` — настоящее ли это число (не `NaN` от текста «abc»). Если да — `Math.max(0, number)` (не меньше нуля), иначе `null`.

```
21 // Where the uploaded picture sits, for example "uploads/pink-123.jpg".
22 const buildFilePath = (file) => path.join('uploads', file.filename).replace(/\\/g, '/');
```

Функция `buildFilePath(file)` — путь к загруженной картинке для записи в базу. `file.filename` — имя, которое дал `upload.js`. `path.join` на Windows ставит `\`, поэтому `replace(/\\/g, '/')` меняет все обратные слеши на обычные — этот путь потом станет частью адреса в браузере. Вызывается из `createProduct` (стр. 42, 55) и `updateProduct` (стр. 107, 119).

```
24 // Deletes an old picture from the disk after the product was replaced or removed.
25 const removeImageFile = (fileName) => {
26   if (!fileName) return;
27   const fullPath = path.join(__dirname, '..', fileName);
28   fs.promises.unlink(fullPath).catch(() => {});
29 };
```

Функция `removeImageFile(fileName)` — удаляет файл картинки с диска. Вызывается из `createProduct` (стр. 55), `updateProduct` (стр. 116, 119), `deleteProduct` (стр. 133).

- **стр. 26** — имени нет — ничего не делаем.
- **стр. 27** — полный путь: `back/controllers` + `..` + `uploads/xxx.jpg` → `back/uploads/xxx.jpg`.
- **стр. 28** — `fs.promises.unlink` — удалить файл. `.catch(() => {})` — если файла уже нет, ошибку молча игнорируем.

Функция `createProduct` — создать товар (стр. 32–59)

- **Кто вызывает:** `productsRoute.js` стр. 21, `POST /api/products`, после `protect`, `restrictTo`, `upload.single('image')`. С фронта — `ManagerProductsPage.jsx`.
- **Что вызывает:** `Product.create` (база), `buildFilePath`, `toArray`, `parseStock`, `removeImageFile`.
- **Получает:** `req.file` (картинка от multer), `req.body` (текстовые поля формы).

```
32 const createProduct = async (req, res) => {
33   try {
34     if (!req.file) {
35       return res.status(400).send({ message: 'Product image is required' });
36     }
37     if (!req.body.model_name || !req.body.category) {
38       return res.status(400).send({ message: 'Send required fields: model_name, category' });
39     }
40
41     const product = await Product.create({
42       file_name: buildFilePath(req.file),
43       model_name: req.body.model_name,
44       category: req.body.category,
45       price: req.body.price,
46       color_palette: toArray(req.body.color_palette),
47       details: toArray(req.body.details),
48       style: toArray(req.body.style),
49       stock: parseStock(req.body.stock)
50     });
51
52     res.status(201).json({ data: product });
53   } catch (error) {
54     // The product was not saved, so the uploaded picture is not needed.
55     if (req.file) removeImageFile(buildFilePath(req.file));
56     console.log(error.message);
57     res.status(400).send({ message: error.message });
58   }
59 };
```

- **стр. 34–36** — картинки нет — 400. `req.file` заполняет multer.
- **стр. 37–39** — нет названия или категории — 400.
- **стр. 41–50** — `Product.create` сохраняет товар. Каждое поле: путь к картинке; название; категория (Mongoose проверит `enum`); цена (строка `"120"` из формы — Mongoose сам превратит в число); три списка через `toArray`; остаток через `parseStock`.
- **стр. 52** — 201 и созданный товар.
- **стр. 55** — если сохранить не удалось (например, категория не из списка) — картинка уже лежит на диске, удаляем её, чтобы не копился мусор.
- **стр. 57** — ответ 400 с текстом ошибки Mongoose (обычно это ошибка данных).

Функция `getAllProducts` — все товары (стр. 62–70)

```
62 const getAllProducts = async (req, res) => {
63   try {
64     const products = await Product.find({});
65     res.status(200).json({ count: products.length, data: products });
...
70 };
```

- **Кто вызывает:** `GET /api/products` (`productsRoute.js` стр. 16). Фронт: `BeadedBagsPage`, `BraceletsPage`, `ManagerProductsPage`.
- **стр. 64** — `Product.find({})` — все документы коллекции `products` (пустое условие — без фильтра). Результат — массив.
- **стр. 65** — ответ: количество и массив товаров. Фронт берёт `response.data.data`.

Функция `getProductById` — один товар (стр. 73–84)

```
73 const getProductById = async (req, res) => {
74   try {
75     const product = await Product.findById(req.params.id);
76     if (!product) {
77       return res.status(404).send({ message: 'Product not found' });
78     }
79     res.status(200).json({ data: product });
...
84 };
```

- **Кто вызывает:** `GET /api/products/:id` (стр. 17). Фронт: `ProductDetailPage`.
- **стр. 75** — `req.params.id` — часть адреса после `/api/products/`. `Product.findById` ищет по `_id`.
- **стр. 76–78** — не найден — 404. (Если id вообще не похож на id MongoDB, Mongoose бросит ошибку → `catch` → 500.)
- **стр. 79** — 200 и товар.

Функция `updateProduct` — изменить товар (стр. 87–123)

- **Кто вызывает:** `PUT /api/products/:id` (стр. 22) после `protect`, `restrictTo`, `upload.single('image')`. Фронт: `ManagerProductsPage` — кнопка Save changes.
- **Что вызывает:** `Product.findById`, `Product.findByIdAndUpdate` (база), `toArray`, `parseStock`, `buildFilePath`, `removeImageFile`.

```
87  const updateProduct = async (req, res) => {
88    try {
89      const product = await Product.findById(req.params.id);
90      if (!product) {
91        return res.status(404).send({ message: 'Product not found' });
92      }
93
94      // Only the fields the manager actually filled in are changed.
95      const update = {};
96      if (req.body.model_name !== undefined) update.model_name = req.body.model_name;
97      if (req.body.category !== undefined) update.category = req.body.category;
98      if (req.body.price !== undefined) update.price = req.body.price;
99      if (req.body.color_palette !== undefined) update.color_palette =
toArray(req.body.color_palette);
100     if (req.body.details !== undefined) update.details = toArray(req.body.details);
101     if (req.body.style !== undefined) update.style = toArray(req.body.style);
102     if (req.body.stock !== undefined) update.stock = parseStock(req.body.stock);
103
104     // A new picture was sent, so remember the old one and delete it afterwards.
105     let oldImage = null;
106     if (req.file) {
107       update.file_name = buildFilePath(req.file);
108       oldImage = product.file_name;
109     }
110
111     const result = await Product.findByIdAndUpdate(req.params.id, update, {
112       new: true,
113       runValidators: true
114     });
115
116     if (oldImage) removeImageFile(oldImage);
117     res.status(200).json({ data: result });
118   } catch (error) {
119     if (req.file) removeImageFile(buildFilePath(req.file));
120     console.log(error.message);
121     res.status(400).send({ message: error.message });
122   }
123 };
```

- **стр. 89–92** — сначала находим товар (нужен путь старой картинки). Нет — 404.
- **стр. 95** — пустой объект изменений.
- **стр. 96–102** — каждое поле добавляется в `update`, только если его прислали (`!== undefined`). Так не прислали — не трогаем.
- **стр. 105–109** — если пришла новая картинка: записываем новый путь и запоминаем старый.
- **стр. 111–114** — `findByIdAndUpdate(id, изменения, настройки)` : найти и обновить одним запросом. `new: true` — вернуть **обновлённый** документ (без этого вернулся бы старый). `runValidators: true` — проверить правила схемы (`enum`, `min`) и при обновлении, по умолчанию Mongoose проверяет их только при создании.
- **стр. 116** — обновление прошло — удаляем старую картинку с диска.
- **стр. 117** — 200 и обновлённый товар.
- **стр. 119** — ошибка — удаляем только что загруженную новую картинку.

Функция `deleteProduct` — удалить товар (стр. 126–139)

```
126 const deleteProduct = async (req, res) => {
127   try {
128     const result = await Product.findByIdAndDelete(req.params.id);
129     if (!result) {
130       return res.status(404).send({ message: 'Product not found' });
131     }
132
133     removeImageFile(result.file_name);
134     res.status(200).send({ message: 'Product deleted successfully' });
...
139 };
```

- **Кто вызывает:** `DELETE /api/products/:id` (стр. 23). Фронт: `ManagerProductsPage` — кнопка Delete.
- **стр. 128** — `findByIdAndDelete` — найти и удалить; возвращает удалённый документ (или `null`).
- **стр. 129–131** — не было такого — 404.
- **стр. 133** — удаляем файл его картинки.
- **стр. 134** — 200 с сообщением.
- **стр. 141–147** — экспорт пяти функций.

6.3 back/controllers/CartController.js — корзина вошедшего пользователя

Кто подключает: `cartRoute.js` (стр. 2). **Что подключает:** модель `Cart` (стр. 1).

Во всех функциях `req.user.id` — id пользователя, которого положил `protect`. Поэтому человек работает **только со своей** корзиной: чужой id подставить нельзя.

Функция `loadCart(cartId)` (стр. 5–9) — вспомогательная

```
5 const loadCart = async (cartId) => {
6   const cart = await Cart.findById(cartId).populate('items.product');
7   cart.items = cart.items.filter((item) => item.product);
8   return cart;
9 };
```

- **Кто вызывает:** каждая функция этого файла перед ответом (стр. 21, 53, 76, 106).
- **стр. 6** — находим корзину по `_id` и вызываем `.populate('items.product')`. В базе позиция выглядит так: `{ product: '65f1...', quantity: 2 }` — только id. `populate` по полю `ref: 'Product'` (cartModel.js стр. 16) идёт в коллекцию `products` и **подставляет вместо id весь документ товара**: название, цену, картинку. Это как JOIN в SQL.
- **стр. 7** — если товар удалили из каталога, `populate` вернёт для него `null`. `filter((item) => item.product)` оставляет только позиции с существующим товаром.
- **стр. 8** — возвращаем корзину с полными товарами.

Функция `getCart` (стр. 12–26)

- **Кто вызывает:** `GET /api/cart` (`cartRoute.js` стр. 8). Фронт: `fetchCart` в `CartContext.jsx`.

```
14    let cart = await Cart.findOne({ user: req.user.id });
15
16    // A user who never added anything yet has no cart, so create an empty one.
17    if (!cart) {
18      cart = await Cart.create({ user: req.user.id, items: [] });
19    }
20
21    res.status(200).json({ data: await loadCart(cart._id) });
```

- **стр. 14** — ищем корзину этого пользователя. `let` — потому что ниже переменная может получить новое значение.
- **стр. 17–19** — корзины ещё нет — создаём пустую.
- **стр. 21** — отвечаем корзиной с подставленными товарами.

Функция `addToCart` (стр. 29–58)

- **Кто вызывает:** `POST /api/cart` (стр. 9). Фронт: `addToCart` в `CartContext.jsx` и перенос гостевой корзины при входе.
- **Получает:** `{ productId }`.

```
31    const { productId } = req.body;
32    if (!productId) {
33      return res.status(400).send({ message: 'Send required field: productId' });
34    }
35
36    let cart = await Cart.findOne({ user: req.user.id });
37    if (!cart) {
38      cart = await Cart.create({ user: req.user.id, items: [] });
39    }
40
41    // Already in the cart, so only raise the quantity.
42    const index = cart.items.findIndex(
43      (item) => item.product && item.product.toString() === productId
44    );
45
46    if (index > -1) {
47      cart.items[index].quantity += 1;
48    } else {
49      cart.items.push({ product: productId, quantity: 1 });
50    }
51
52    await cart.save();
53    res.status(200).json({ data: await loadCart(cart._id) });
```

- **стр. 31–34** — без `productId` — 400.
- **стр. 36–39** — находим или создаём корзину.
- **стр. 42–44** — `findIndex` ищет номер позиции с этим товаром (или `-1`, если нет). `item.product.toString()` — в базе id хранится как объект ObjectId, а из запроса пришла строка; чтобы сравнить, превращаем в строку.
- **стр. 46–47** — товар уже есть — количество +1.
- **стр. 48–49** — нет — добавляем новую позицию с количеством 1.
- **стр. 52** — `cart.save()` — сохранить корзину в базе.

- **стр. 53** — ответ с обновлённой корзиной.

Функция `removeFromCart` (стр. 61–81)

- **Кто вызывает:** `DELETE /api/cart/:productId` (стр. 11). Фронт: `removeFromCart` в `CartContext.jsx` (иконка мусорки).

```
63    const { productId } = req.params;
64
65    const cart = await Cart.findOne({ user: req.user.id });
66    if (!cart) {
67      return res.status(404).send({ message: 'Cart not found' });
68    }
69
70    // Keep every item except the one we were asked to remove.
71    cart.items = cart.items.filter(
72      (item) => item.product && item.product.toString() !== productId
73    );
74
75    await cart.save();
76    res.status(200).json({ data: await loadCart(cart._id) });
```

- **стр. 63** — id товара берём из адреса (`req.params`), а не из тела.
- **стр. 65–68** — нет корзины — 404.
- **стр. 71–73** — `filter` оставляет все позиции, **кроме** той, где id совпал (`!==`).
- **стр. 75–76** — сохранить и ответить.

Функция `updateQuantity` (стр. 84–111)

- **Кто вызывает:** `PATCH /api/cart/:productId` (стр. 10). Фронт: `updateQuantity` в `CartContext.jsx` (кнопки + и −).
- **Получает:** id в адресе, `{ quantity }` в теле.

```
86    const { productId } = req.params;
87    const quantity = parseInt(req.body.quantity, 10);
88    if (!quantity || quantity < 1) {
89      return res.status(400).send({ message: 'Quantity must be 1 or more' });
90    }
91
92    const cart = await Cart.findOne({ user: req.user.id });
93    if (!cart) {
94      return res.status(404).send({ message: 'Cart not found' });
95    }
96
97    const item = cart.items.find(
98      (item) => item.product && item.product.toString() === productId
99    );
100   if (!item) {
101     return res.status(404).send({ message: 'Product is not in the cart' });
102   }
103
104   item.quantity = quantity;
105   await cart.save();
106   res.status(200).json({ data: await loadCart(cart._id) });
```

- **стр. 87** — `parseInt(значение, 10)` — превратить в целое число в десятичной системе. `"3"` → 3, `"abc"` → `NaN`.
- **стр. 88–90** — `!quantity` ловит `NaN` и 0; меньше 1 — тоже 400.
- **стр. 92–95** — нет корзины — 404.
- **стр. 97–99** — `find` возвращает саму позицию (не номер, как `findIndex`).
- **стр. 100–102** — товара нет в корзине — 404.
- **стр. 104** — меняем количество прямо в найденной позиции (это объект внутри `cart.items`).
- **стр. 105–106** — сохранить и ответить.

Функция `clearCart` (стр. 114–122)

- **Кто вызывает:** `DELETE /api/cart` (стр. 12). Фронт: `clearCart` в `CartContext.jsx` — кнопка «Clear all items» и страница «Thank You» после оплаты.

```
116     await Cart.findOneAndUpdate({ user: req.user.id }, { items: [] });
117     res.status(200).send({ message: 'Cart cleared' });
```

- **стр. 116** — `findOneAndUpdate(условие, изменения)` — найти корзину пользователя и поставить пустой массив позиций. Один запрос, без загрузки корзины.
- **стр. 117** — 200.
- **стр. 124–130** — экспорт пяти функций.

6.4 back/controllers/OrderController.js — заказы и оплата

Кто подключает: `ordersRoute.js` (стр. 3–10) — шесть функций; `index.js` (стр. 13) — `stripeWebhook`.

Что подключает:

```
1  const mongoose = require('mongoose');
2  const jwt = require('jsonwebtoken');
3  const Order = require('../models/orderModel.js');
4  const Product = require('../models/productModel.js');
5  const { sendMail } = require('../utils/email');
6
7  // Without the key on this machine there is no payment, and checkout answers 501.
8  const stripe = process.env.STRIPE_SECRET_KEY
9    ? require('stripe')(process.env.STRIPE_SECRET_KEY)
10   : null;
```

- **стр. 1** — Mongoose — здесь нужен для проверки `mongoose.Types.ObjectId.isValid` (стр. 47).
- **стр. 2** — `jsonwebtoken` — чтобы достать id покупателя из токена, если он вошёл (стр. 30).
- **стр. 3** — модель `Order` → коллекция `orders`.
- **стр. 4** — модель `Product` — цены и остатки товаров.
- **стр. 5** — `sendMail` — письма о заказе.
- **стр. 8–10** — подключение Stripe. Тернарный оператор: если в `.env` / на Render есть `STRIPE_SECRET_KEY` — `require('stripe')` возвращает функцию, вызываем её с ключом и получаем объект для работы с API Stripe. Если ключа нет — `null`, и оплата отвечает 501.

```
12 // A bag from the Custom Studio is not in the catalog, so its price is
13 // counted here from the chosen size and strap, never taken from the browser.
14 // The same prices are shown on the Custom Studio page.
15 const SIZE_PRICES = { mini: 120, small: 180, medium: 260, large: 360 };
16 const STRAP_PRICES = { golden: 30, silver: 30, mixed: 40, beaded: 45 };
```

- **стр. 15** — цены размеров сумки из конструктора. Объект: ключ — id размера, значение — цена в долларах. `SIZE_PRICES['small']` → 180.
- **стр. 16** — цены ремешков. Те же цифры показаны в `CustomStudioPage.jsx` (стр. 27–48).

Функция `genOrderNumber` (стр. 19–23)

```
18 // Order number like DH-20261021-4821: the date and four random digits.
19 const genOrderNumber = () => {
20   const d = new Date();
21   const ymd = `${d.getFullYear()}${String(d.getMonth() + 1).padStart(2,
'0')}${String(d.getDate()).padStart(2, '0')}`;
22   return `DH-${ymd}-${Math.floor(1000 + Math.random() * 9000)}`;
23 };
```

- **Кто вызывает:** `createCheckoutSession` (стр. 127).
- **стр. 20** — текущая дата и время.
- **стр. 21** — год + месяц + день в виде `20261021`. `getMonth()` считает месяцы с 0, поэтому `+ 1`. `padStart(2, '0')` дописывает ноль слева: `5` → `05`.
- **стр. 22** — `Math.random()` — случайное число от 0 до 1; `1000 + random × 9000` — от 1000 до 9999; `Math.floor` — без дробной части. Результат: `DH-20261021-4821`.

Функция `userIdFromHeader(req)` (стр. 26–36)

```
25 // A guest may order too, so the token is optional here. No token means null.
26 const userIdFromHeader = (req) => {
27   try {
28     const auth = req.headers.authorization;
29     if (auth?.startsWith('Bearer ')) {
30       return jwt.verify(auth.split(' ')[1], process.env.JWT_SECRET).id;
31     }
32   } catch {
33     return null;
34   }
35   return null;
36 };
```

- **Кто вызывает:** `createCheckoutSession` (стр. 128).
- **Зачем:** адрес оформления открыт и гостю, поэтому `protect` здесь не стоит. Но если покупатель вошёл, хочется записать его id в заказ.
- **стр. 28** — заголовок `Authorization`.
- **стр. 29** — `auth?.startsWith(...)` — `?.` защищает от ошибки, если заголовка нет (`undefined`).
- **стр. 30** — проверяем токен (как в `protect`) и берём из него `id`.
- **стр. 32–34** — токен плохой — `null` (заказ будет гостевым, без ошибки).
- **стр. 35** — заголовка нет — `null`.

Функция `buildOrderItems(rawItems)` (стр. 41–97) — главная проверка безопасности

- **Кто вызывает:** `createCheckoutSession` (стр. 115).
- **Что вызывает:** `mongoose.Types.ObjectId.isValid`, `Product.findById` (база), таблицы `SIZE_PRICES` и `STRAP_PRICES`.
- **Получает:** `rawItems` — массив позиций корзины, как его прислал браузер.
- **Возвращает:** `{ items, error }` — позиции, собранные **с нашими ценами**, и текст первой ошибки (или `null`).

```
41 const buildOrderItems = async (rawItems) => {
42   const items = [];
43   let error = null;
44   for (const it of rawItems || []) {
45     const quantity = Math.max(1, parseInt(it.quantity, 10) || 1);
46     let product = null;
47     if (it.id && mongoose.Types.ObjectId.isValid(it.id)) {
48       product = await Product.findById(it.id);
49     }
```

- **стр. 42** — сюда соберём проверенные позиции.
- **стр. 43** — сюда запишем первую найденную ошибку.
- **стр. 44** — цикл по каждой позиции. `rawItems || []` — если ничего не прислали, цикл по пустому массиву.
- **стр. 45** — количество: `parseInt` → целое; если не число — `|| 1`; `Math.max(1, ...)` — не меньше 1.
- **стр. 46** — пока товар не найден.
- **стр. 47** — если у позиции есть `id` и он похож на настоящий id MongoDB (24 шестнадцатеричных символа)…
- **стр. 48** — …ищем товар в базе. У сумки из конструктора id вида `custom-1718...` — проверка на стр. 47 не пройдёт, и `product` останется `null`.

```
50     if (product) {
51       if (typeof product.stock === 'number') {
52         if (product.stock <= 0) {
53           error = error || `"${product.model_name}" is out of stock`;
54         } else if (quantity > product.stock) {
55           error = error || `Only ${product.stock} of "${product.model_name}" left in stock`;
56         }
57       }
58       items.push({
59         product: product._id,
60         name: product.model_name,
61         category: product.category,
62         image: product.file_name,
63         price: product.price || 0,
64         quantity,
65         custom: null,
66       });
```

Ветка «товар из каталога»:

- **стр. 51** — остаток отслеживается (число, а не `null`)?

- **стр. 52–53** — 0 или меньше — ошибка «нет в наличии». `error = error || ...` — записываем, только если ошибки ещё не было (показываем первую).
- **стр. 54–55** — хотят больше, чем есть — ошибка «осталось только N».
- **стр. 58–66** — добавляем позицию. **Все данные берутся из базы** (`product...`), а не из запроса: название, категория, картинка и главное — **цена** (стр. 63). Из запроса взято только количество.

```
67     } else {
68       const config = it.config || it.custom || {};
69       const sizePrice = SIZE_PRICES[config.size];
70       const strapPrice = STRAP_PRICES[config.strap];
71
72       // Not a catalog product and not a known size and strap either.
73       // typeof is checked because a made up size like "constructor" would
74       // find a built in function in the table instead of a price.
75       if (typeof sizePrice !== 'number' || typeof strapPrice !== 'number') {
76         error = error || `"${it.name || 'Item'}" is no longer available`;
77         continue;
78       }
79
80       items.push({
81         product: null,
82         name: String(it.name || 'Custom item').slice(0, 120),
83         category: 'custom',
84         image: String(it.image || ''),
85         price: sizePrice + strapPrice,
86         quantity,
87         custom: {
88           size: config.size,
89           strap: config.strap,
90           chainColor: config.chainColor || null,
91           color: String(config.color || ''),
92         },
93       });
94     }
95   }
96   return { items, error };
97 };
```

Ветка «сумка из конструктора» (или неизвестная позиция):

- **стр. 68** — параметры сумки: `config` (так их кладёт `CustomStudioPage`), запасной вариант — `custom`, иначе пустой объект.
- **стр. 69** — цена размера из нашей таблицы: `SIZE_PRICES['small']` → 180.
- **стр. 70** — цена ремешка.
- **стр. 75** — если хоть одна цена — не число (размер или ремешок неизвестны или подделаны)… Проверка именно `typeof`, потому что у любого объекта JavaScript есть встроенные свойства: `SIZE_PRICES['constructor']` вернёт функцию, а не `undefined`.
- **стр. 76** — …ошибка «товар недоступен».
- **стр. 77** — `continue` — пропустить эту позицию и перейти к следующей.
- **стр. 80–93** — добавляем позицию:
  - **стр. 81** — `product: null` — в каталоге такого товара нет;
  - **стр. 82** — название из запроса, но принудительно строкой и не длиннее 120 символов;

    - **стр. 83** — категория всегда `custom`;
    - **стр. 85** — **цена = размер + ремешок по нашим таблицам**. Цена, которую прислал браузер, не используется вообще;
    - **стр. 87–92** — параметры сумки сохраняются в заказе, чтобы мастер знал, что делать.
- **стр. 96** — возвращаем позиции и ошибку.

Функция `createCheckoutSession` — начать оплату (стр. 100–171)

- **Кто вызывает:** `ordersRoute.js` стр. 18, `POST /api/orders/checkout-session`. С фронта — `CheckoutPage.jsx`, кнопка Proceed to Payment.
- **Что вызывает:** `buildOrderItems`, `genOrderNumber`, `userIdFromHeader`, `Order.create` (база), `stripe.checkout.sessions.create` (Stripe через интернет), `order.save()`.
- **Получает:** `{ items, customer, shipping, deliveryMethod, notes }`.
- **Возвращает:** `{ url }` — адрес страницы оплаты Stripe.

```
100 const createCheckoutSession = async (req, res) => {
101   try {
102     if (!stripe) {
103       return res.status(501).send({ message: 'Payments are not configured' });
104     }
105
106     const { items: rawItems, customer, shipping, deliveryMethod, notes } = req.body;
107
108     if (!customer?.name || !customer?.email) {
109       return res.status(400).send({ message: 'Name and email are required' });
110     }
111     if (!Array.isArray(rawItems) || rawItems.length === 0) {
112       return res.status(400).send({ message: 'Cart is empty' });
113     }
114
115     const { items, error: stockError } = await buildOrderItems(rawItems);
116     if (stockError) {
117       return res.status(400).send({ message: stockError });
118     }
119     const subtotal = items.reduce((sum, it) => sum + it.price * it.quantity, 0);
120     const total = subtotal; // free shipping for now
121
122     if (total <= 0) {
123       return res.status(400).send({ message: 'Order total is invalid' });
124     }
```

- **стр. 102–104** — Stripe не подключён (нет ключа) — 501.
- **стр. 106** — достаём поля. `items: rawItems` — взять поле `items`, но назвать переменную `rawItems` («сырые», непроверенные).
- **стр. 108–110** — без имени или почты — 400. `customer?.name` — не упасть, если `customer` не прислали.
- **стр. 111–113** — `items` не массив или пустой — 400.
- **стр. 115** — собираем позиции с нашими ценами. `error: stockError` — переименовать поле `error` в переменную `stockError`.
- **стр. 116–118** — была ошибка (нет в наличии, неизвестная сумка) — 400 с её текстом.

- **стр. 119** — `reduce` проходит по позициям и копит сумму: начиная с 0, к сумме прибавляется цена × количество.
- **стр. 120** — итого = сумма (доставка бесплатная).
- **стр. 122–124** — итог 0 или меньше — 400.

```
126     const order = await Order.create({
127       orderNumber: genOrderNumber(),
128       user: userIdFromHeader(req),
129       customer: { name: customer.name, email: customer.email, phone: customer.phone || '' },
130       shipping: {
131         address: shipping?.address || '',
132         city: shipping?.city || '',
133         country: shipping?.country || '',
134       },
135       deliveryMethod: deliveryMethod === 'pickup' ? 'pickup' : 'delivery',
136       items,
137       subtotal,
138       total,
139       currency: 'usd',
140       status: 'pending',
141       notes: notes || '',
142     });
```

- **стр. 126** — создаём заказ в базе **до** оплаты.
- **стр. 127** — номер заказа.
- **стр. 128** — id покупателя или `null` для гостя.
- **стр. 129** — контакты; телефон необязателен.
- **стр. 130–134** — адрес; для самовывоза пустые строки.
- **стр. 135** — способ получения: только `pickup` или `delivery`, любое другое значение превращается в `delivery`.
- **стр. 136–138** — проверенные позиции и суммы.
- **стр. 140** — статус `pending` — «ждёт оплаты». Оплаченным его сделает только вебхук.
- **стр. 141** — комментарий покупателя.

```
144     const base = req.headers.origin || process.env.CLIENT_URL || 'http://localhost:5173';
145
146     const session = await stripe.checkout.sessions.create({
147       mode: 'payment',
148       payment_method_types: ['card'],
149       customer_email: customer.email,
150       line_items: items.map((it) => ({
151         price_data: {
152           currency: 'usd',
153           product_data: { name: `${it.name}${it.custom ? ' (Custom)' : ''}` },
154           unit_amount: Math.round(it.price * 100),
155         },
156         quantity: it.quantity,
157       })),
158       metadata: { orderId: order._id.toString() },
159       success_url: `${base}/order-success?session_id={CHECKOUT_SESSION_ID}`,
160       cancel_url: `${base}/checkout`,
161     });
162
163     order.payment.sessionId = session.id;
164     await order.save();
165
166     res.status(200).json({ url: session.url });
```

- **стр. 144** — адрес сайта: откуда пришёл запрос, иначе из настроек, иначе локальный. Нужен, чтобы Stripe вернул покупателя на наш сайт.
- **стр. 146** — запрос в Stripe: создать **сессию оплаты** — страницу, где покупатель введёт карту.
- **стр. 147** — `mode: 'payment'` — разовая оплата (не подписка).
- **стр. 148** — способ — банковская карта.
- **стр. 149** — почта покупателя подставится на странице Stripe.
- **стр. 150–157** — позиции для Stripe: `map` превращает каждую нашу позицию в формат Stripe:
  - **стр. 153** — название; у кастомной сумки дописываем « (Custom)»;
  - **стр. 154** — цена за штуку **в центах**: Stripe принимает суммы в минимальных единицах валюты. `Math.round` убирает ошибки дробей (например, 19.99 × 100 = 1998.9999…);
  - **стр. 156** — количество.
- **стр. 158** — `metadata` — наша пометка: id заказа. Stripe вернёт её в вебхуке, и мы найдём заказ.
- **стр. 159** — куда вернуть после успешной оплаты. `{CHECKOUT_SESSION_ID}` Stripe сам заменит на id сессии.
- **стр. 160** — куда вернуть, если покупатель нажал «назад».
- **стр. 163–164** — записываем id сессии в заказ и сохраняем — по нему страница «спасибо» найдёт заказ.
- **стр. 166** — фронту отдаём только адрес страницы оплаты.
- **стр. 167–170** — любая ошибка — 500 «Could not start checkout».

Функции для писем (стр. 173–233)

```
173 // The helpers below build the HTML of the two order emails.
174 const money = (n) => `$${Number(n).toFixed(2)}`;
```

- **Функция** `money(n)` — формат денег: `50` → `$50.00`. `toFixed(2)` — ровно два знака после точки. Первый `$` — это символ доллара в тексте, `${...}` — вставка значения. Вызывается из `itemsTableRows`, `itemsTable`, `sendManagerReport`.

```
176 const itemsTableRows = (order) =>
177   order.items
178     .map((it) => {
179       const spec = it.custom
180         ? `<br><span style="color:#888;font-size:12px">${[it.custom.size, it.custom.strap,
it.custom.chainColor, it.custom.color]
181             .filter(Boolean)
182             .join(' · ')}</span>`
183         : '';
184       return `<tr>
185         <td ...>${it.name}${spec}</td>
186         <td ...>${it.quantity}</td>
187         <td ...>${money(it.price * it.quantity)}</td>
188       </tr>`;
189     })
190     .join('');
```

- **Функция** `itemsTableRows(order)` — строки HTML-таблицы позиций для письма. Вызывается из `itemsTable` (стр. 204).
- **стр. 178** — `map` — для каждой позиции строим кусок HTML.
- **стр. 179–183** — `spec` — для кастомной сумки серым шрифтом её параметры: массив из четырёх значений → `filter(Boolean)` убирает пустые → `join(' · ')` склеивает через точку: `small · beaded · black`. Для обычного товара — пустая строка.
- **стр. 184–188** — строка таблицы: название (+ параметры), количество, сумма по позиции.
- **стр. 190** — `join('')` склеивает все строки в одну большую строку HTML.

```
192 const deliveryLine = (order) =>
193   order.deliveryMethod === 'pickup'
194     ? 'Self pickup'
195     : `Delivery to ${[order.shipping.address, order.shipping.city,
order.shipping.country].filter(Boolean).join(', ')}`;
```

- **Функция** `deliveryLine(order)` — строка о способе получения: «Self pickup» или «Delivery to адрес, город, страна» (пустые части убираются). Вызывается из обоих писем.

```
197 const itemsTable = (order) => `
198   <table ...>
199     <thead><tr> ...заголовки Item / Qty / Price... </tr></thead>
204     <tbody>${itemsTableRows(order)}</tbody>
205   </table>
206   <p ...><strong>Total: ${money(order.total)}</strong></p>`;
```

- **Функция** `itemsTable(order)` — вся таблица: заголовки (стр. 199–203), строки из `itemsTableRows` (стр. 204), итог (стр. 206). Вызывается из обоих писем.

```
208 const sendCustomerReceipt = async (order) => {
209   const html = `
210     <div ...>
211       <h2 ...>DARIN'S HANDMADE</h2>
212       <p>Hi ${order.customer.name}, thank you for your order!</p>
213       <p>Order <strong>${order.orderNumber}</strong> is confirmed and paid.</p>
214       ${itemsTable(order)}
215       <p ...>${deliveryLine(order)}</p>
216       <p ...>Handmade to order — we'll be in touch about production & delivery.</p>
217     </div>`;
218   await sendMail({ to: order.customer.email, subject: `Your DARIN'S HANDMADE order
${order.orderNumber}`, html });
219 };
```

- **Функция `sendCustomerReceipt(order)`** — письмо-чек покупателю. Вызывается из `stripeWebhook` (стр. 276).
- **стр. 209–217** — HTML письма: приветствие по имени, номер заказа, таблица позиций, способ получения.
- **стр. 218** — `sendMail` (utils/email.js) на почту покупателя.

```
221 const sendManagerReport = async (order) => {
222   const to = process.env.MANAGER_EMAIL;
223   if (!to) return;
224   const html = `
225     <div ...>
226       <h2>New order — ${order.orderNumber}</h2>
227       <p><strong>Customer:</strong> ${order.customer.name} · ${order.customer.email} ·
${order.customer.phone || '—'}</p>
228       <p><strong>Fulfilment:</strong> ${deliveryLine(order)}</p>
229       ${itemsTable(order)}
230       ${order.notes ? `<p><strong>Notes:</strong> ${order.notes}</p>` : ''}
231     </div>`;
232   await sendMail({ to, subject: `New order ${order.orderNumber} — ${money(order.total)}`, html
});
233 };
```

- **Функция `sendManagerReport(order)`** — письмо менеджеру о новом заказе. Вызывается из `stripeWebhook` (стр. 277).
- **стр. 222–223** — почта менеджера из настроек; если не задана — письмо не отправляем.
- **стр. 226–230** — номер, контакты покупателя (телефон или «—»), способ получения, таблица, комментарий (только если есть).
- **стр. 232** — отправка, в теме — номер и сумма.

Функция `stripeWebhook` — Stripe сообщает об оплате (стр. 237–288)

- **Кто вызывает: сервер Stripe** (не браузер!) после успешной оплаты: `POST /api/orders/webhook`. Подключена в `index.js` стр. 25, после `express.raw`.
- **Что вызывает:** `stripe.webhooks.constructEvent` (проверка подписи), `Order.findById`, `order.save()`, `Product.updateOne` (база), `sendCustomerReceipt`, `sendManagerReport`.

```
237 const stripeWebhook = async (req, res) => {
238   if (!stripe || !process.env.STRIPE_WEBHOOK_SECRET) {
239     return res.status(501).send('Stripe webhook not configured');
240   }
241
242   // The signature proves the request really came from Stripe.
243   let event;
244   try {
245     event = stripe.webhooks.constructEvent(
246       req.body,
247       req.headers['stripe-signature'],
248       process.env.STRIPE_WEBHOOK_SECRET
249     );
250   } catch (error) {
251     return res.status(400).send(`Webhook Error: ${error.message}`);
252   }
```

- **стр. 238–240** — нет ключа или секрета вебхука — 501.
- **стр. 243** — переменная для события.
- **стр. 245–249** — `constructEvent(тело, подпись, секрет)` — проверяет, что запрос действительно от Stripe: Stripe подписал сырое тело (`req.body` здесь — байты, благодаря `express.raw`) секретом `whsec_...`; подпись пришла в заголовке `stripe-signature`. Совпало — возвращает объект события. Не совпало — бросает ошибку.
- **стр. 250–252** — подпись неверная (кто-то пытается подделать оплату) — 400.

```
254   if (event.type === 'checkout.session.completed') {
255     const session = event.data.object;
256     try {
257       const order = await Order.findById(session.metadata?.orderId);
258       // Stripe may send the same event twice, so only a pending order is handled.
259       if (order && order.status === 'pending') {
260         order.status = 'paid';
261         order.payment.paymentIntentId = session.payment_intent;
262         order.payment.paidAt = new Date();
263         await order.save();
```

- **стр. 254** — нас интересует только событие «сессия оплаты завершена».
- **стр. 255** — сама сессия оплаты (в ней наш `metadata`).
- **стр. 257** — находим заказ по `orderId`, который мы положили в `metadata` (стр. 158).
- **стр. 259** — только если заказ есть и ещё `pending`. Если Stripe пришлёт то же событие повторно, заказ уже `paid`, и второй раз мы ничего не сделаем.
- **стр. 260** — статус «оплачен».
- **стр. 261** — id платежа в Stripe.
- **стр. 262** — время оплаты.
- **стр. 263** — сохраняем.

```
265         // The stock check is part of the query, so it never goes below zero.
266         for (const it of order.items) {
267           if (it.product) {
268             await Product.updateOne(
269               { _id: it.product, stock: { $gte: it.quantity } },
270               { $inc: { stock: -it.quantity } }
271             );
272           }
273         }
```

- **стр. 266** — по каждой позиции заказа.
- **стр. 267** — только товары каталога (у кастомной сумки `product: null`).
- **стр. 268–271** — `updateOne(условие, изменение)`: найти товар, у которого `_id` совпадает **и** остаток `$gte` (больше или равен) количеству, и уменьшить остаток (`$inc` с минусом). Проверка и уменьшение — **одна операция** в базе, поэтому остаток не уйдёт в минус, даже если два человека платят одновременно. Если остаток `null` (не отслеживается), условие `$gte` не выполнится, и ничего не изменится — так и нужно.

```
275         try {
276           await sendCustomerReceipt(order);
277           await sendManagerReport(order);
278         } catch (mailErr) {
279           console.error('Order email error:', mailErr);
280         }
281       }
282     } catch (error) {
283       console.log(error.message);
284     }
285   }
286
287   res.json({ received: true });
288 };
```

- **стр. 275–280** — письма в отдельном `try/catch`: если письмо не ушло, оплата всё равно учтена.
- **стр. 282–284** — ошибка базы — пишем в консоль.
- **стр. 287** — ответ Stripe «получено» (код 200). Если не ответить 200, Stripe будет повторять запрос.

Константы статусов (стр. 290–291)

```
290 const ALLOWED_STATUS = ['pending', 'paid', 'in_production', 'shipped', 'completed', 'cancelled'];
291 const PAID_STATUSES = ['paid', 'in_production', 'shipped', 'completed'];
```

- **стр. 290** — все разрешённые статусы. Используется в `updateOrderStatus` (стр. 377).
- **стр. 291** — статусы, которые считаются «деньги получены». Используется в `getStats` (стр. 297).

Функция `getStats` — статистика продаж (стр. 294–341)

- **Кто вызывает:** `GET /api/orders/stats` (`ordersRoute.js` стр. 22). Фронт: `ManagerOrdersPage` (`loadStats`), с параметром `?days=7` или `?days=30` или без него.
- **Что вызывает:** `Order.aggregate` (база) три раза.

```
296     const days = Number(req.query.days);
297     const match = { status: { $in: PAID_STATUSES } };
298     if (days > 0) {
299       match.createdAt = { $gte: new Date(Date.now() - days * 24 * 60 * 60 * 1000) };
300     }
```

- **стр. 296** — `req.query.days` — параметр после `?` в адресе. `Number(...)` — в число (если параметра нет — `NaN`).
- **стр. 297** — условие отбора: статус входит (`$in`) в список оплаченных.
- **стр. 298–300** — если указан период — добавляем условие «создан не раньше, чем N дней назад». `days × 24 × 60 × 60 × 1000` — дни в миллисекундах.

```
302     const [totals] = await Order.aggregate([
303       { $match: match },
304       { $group: { _id: null, revenue: { $sum: '$total' }, count: { $sum: 1 } } },
305     ]);
```

- **`aggregate([этапы])`** — обработка данных внутри базы по этапам, как по конвейеру.
- **стр. 303** — `$match` — оставить только заказы под условие.
- **стр. 304** — `$group` с `_id: null` — сложить **все** оставшиеся заказы в одну группу: `revenue` — сумма полей `total` (`'$total'` — значение поля), `count` — количество (по 1 за каждый).
- **стр. 302** — `aggregate` всегда возвращает массив; `[totals]` берёт первый элемент. Если заказов нет — `undefined`.

```
307     const byStatus = await Order.aggregate([
308       { $group: { _id: '$status', count: { $sum: 1 } } },
309     ]);
```

- **стр. 307–309** — сколько заказов в каждом статусе: группировка по полю `status`. Результат вида `[{ _id: 'paid', count: 3 }, ...]`.

```
311     const topProducts = await Order.aggregate([
312       { $match: match },
313       { $unwind: '$items' },
314       {
315         $group: {
316           _id: '$items.name',
317           qty: { $sum: '$items.quantity' },
318           revenue: { $sum: { $multiply: ['$items.price', '$items.quantity'] } },
319         },
320       },
321       { $sort: { qty: -1 } },
322       { $limit: 5 },
323     ]);
```

- **стр. 312** — те же оплаченные заказы.
- **стр. 313** — `$unwind` «разворачивает» массив: заказ с тремя позициями превращается в три документа, по одному на позицию.
- **стр. 315–319** — группируем по названию товара: сколько штук продано и на какую сумму (`$multiply` — цена × количество).

- **стр. 321** — сортировка по количеству, `-1` — по убыванию.
- **стр. 322** — первые пять.

```
325     const revenue = totals?.revenue || 0;
326     const orders = totals?.count || 0;
327
328     res.status(200).json({
329       data: {
330         revenue,
331         orders,
332         avgOrder: orders ? revenue / orders : 0,
333         byStatus,
334         topProducts,
335       }
336     });
```

- **стр. 325–326** — если заказов не было (`totals` — `undefined`), ставим 0.
- **стр. 332** — средний чек: выручка / количество; если заказов 0 — 0 (на ноль не делим).
- **стр. 328–336** — ответ одним объектом.

Функция `getAllOrders` — список заказов (стр. 344–359)

- **Кто вызывает:** `GET /api/orders` (стр. 21). Фронт: `ManagerOrdersPage` (`load`), с параметрами `?status=paid&search=anna`.

```
346     const { status, search } = req.query;
347     const query = {};
348     if (status && status !== 'all') query.status = status;
349     if (search) {
350       const rx = new RegExp(String(search).trim(), 'i');
351       query.$or = [{ orderNumber: rx }, { 'customer.name': rx }, { 'customer.email': rx }];
352     }
353     const orders = await Order.find(query).sort({ createdAt: -1 }).limit(200);
354     res.status(200).json({ data: orders });
```

- **стр. 346** — параметры из адреса.
- **стр. 347** — пустое условие поиска.
- **стр. 348** — выбран статус (и это не «все») — добавляем фильтр по статусу.
- **стр. 350** — `new RegExp(текст, 'i')` — регулярное выражение «содержит этот текст», `i` — без учёта регистра.
- **стр. 351** — `$or` — подходит, если текст найден **хотя бы в одном** поле: номер заказа, имя или почта. `'customer.name'` — поле внутри вложенного объекта.
- **стр. 353** — `find(условие)` — найти заказы; `.sort({ createdAt: -1 })` — новые сверху; `.limit(200)` — не больше 200.
- **стр. 354** — ответ.

Функция `getOrderById` (стр. 362–371)

```
364     const order = await Order.findById(req.params.id);
365     if (!order) return res.status(404).send({ message: 'Order not found' });
366     res.status(200).json({ data: order });
```

- **Кто вызывает:** `GET /api/orders/:id` (стр. 24). Со страниц сейчас не вызывается.
- **стр. 364–366** — найти по id; нет — 404; есть — 200.

Функция `updateOrderStatus` (стр. 374–387)

- **Кто вызывает:** `PATCH /api/orders/:id/status` (стр. 23). Фронт: `ManagerOrdersPage` (`changeStatus`).

```
376     const { status } = req.body;
377     if (!ALLOWED_STATUS.includes(status)) {
378       return res.status(400).send({ message: 'Invalid status' });
379     }
380     const order = await Order.findByIdAndUpdate(req.params.id, { status }, { new: true });
381     if (!order) return res.status(404).send({ message: 'Order not found' });
382     res.status(200).json({ data: order });
```

- **стр. 376** — новый статус из тела.
- **стр. 377–379** — не из списка — 400.
- **стр. 380** — найти заказ и поставить статус; `new: true` — вернуть обновлённый.
- **стр. 381** — нет такого — 404.
- **стр. 382** — 200 и обновлённый заказ (фронт заменит его в списке).

Функция `getOrderBySession` (стр. 390–406)

- **Кто вызывает:** `GET /api/orders/by-session/:sessionId` (стр. 19). Фронт: `OrderSuccessPage` — Stripe вернул покупателя с `?session_id=...`.

```
392     const order = await Order.findOne({ 'payment.sessionId': req.params.sessionId });
393     if (!order) return res.status(404).send({ message: 'Order not found' });
394     res.status(200).json({
395       data: {
396         orderNumber: order.orderNumber,
397         status: order.status,
398         total: order.total,
399         email: order.customer.email,
400       }
401     });
```

- **стр. 392** — ищем заказ, у которого `payment.sessionId` равен id из адреса (его записали на стр. 163).
- **стр. 394–401** — отдаём **не весь заказ**, а только номер, статус, сумму и почту: адрес открыт без входа, лишнего показывать не нужно.
- **стр. 408–416** — экспорт семи функций.

Часть 7. Сервер: служебные скрипты (seed)

Эти файлы сайт **не вызывает**. Их запускают вручную из терминала в папке `back`.

7.1 back/seed/products.js — заполнить каталог

Запуск: `node seed/products.js`.

```
1  const dotenv = require('dotenv');
2  const connectDB = require('../db/connect');
3  const Product = require('../models/productModel.js');
4  const products = require('./productsData.json');
5
6  dotenv.config();
7
8  const start = async () => {
9    try {
10     await connectDB(process.env.MONGO_URI);
11     await Product.deleteMany();
12     await Product.insertMany(products);
13     console.log('Seed done');
14     process.exit(0);
15   } catch (error) {
16     console.log(error.message);
17     process.exit(1);
18   }
19 };
20
21 start();
```

- **стр. 1** — `dotenv` (чтобы прочитать `MONGO_URI`).
- **стр. 2** — функция подключения к базе.
- **стр. 3** — модель товара.
- **стр. 4** — `require` умеет сразу читать JSON-файл: получаем массив товаров из `productsData.json`.
- **стр. 6** — читаем `.env`.
- **стр. 10** — подключаемся.
- **стр. 11** — `deleteMany()` без условия — **удалить все товары**.
- **стр. 12** — `insertMany(массив)` — вставить все товары из файла.
- **стр. 14** — `process.exit(0)` — завершить программу (0 — успешно). Без этого Node ждал бы, пока закроется соединение с базой.
- **стр. 17** — при ошибке — выход с кодом 1 (ошибка).
- **стр. 21** — запуск.

back/seed/productsData.json — массив объектов-товаров: `file_name`, `model_name`, `color_palette`, `details`, `style`, `category`, `price`. Поля `stock` там нет, поэтому после заполнения остаток не отслеживается.

7.2 back/seed/makeManager.js — выдать роль

Запуск: `node seed/makeManager.js почта@example.com manager` (или `admin`).

```
7  const start = async () => {
8    const email = process.argv[2];
9    const role = process.argv[3] || 'manager';
10
11   if (!email) {
12     console.error('Provide an email:  node seed/makeManager.js user@example.com');
13     process.exit(1);
14   }
15
16   try {
17     await connectDB(process.env.MONGO_URI);
18     const user = await User.findOneAndUpdate({ email }, { role }, { new: true });
19     if (!user) {
20       console.error(`User with email "${email}" not found. Register with this email first.`);
21       process.exit(1);
22     }
23     console.log(`Done: ${user.email} now has role="${user.role}"`);
24     process.exit(0);
25   } catch (error) {
26     console.log(error.message);
27     process.exit(1);
28   }
29 };
30
31 start();
```

- **стр. 1–5** — `dotenv`, `connectDB`, модель `User`, чтение `.env`.
- **стр. 8** — `process.argv` — массив слов команды: `[0]` путь к node, `[1]` путь к скрипту, `[2]` — почта.
- **стр. 9** — `[3]` — роль; если не указана — `manager`.
- **стр. 11–14** — почту не указали — подсказка и выход с ошибкой.
- **стр. 18** — найти пользователя по почте и поставить роль; вернуть обновлённого.
- **стр. 19–22** — нет такого пользователя — сначала нужно зарегистрироваться на сайте.
- **стр. 23–24** — готово.

Почему роль выдаётся скриптом, а не на сайте: так на сервере нет ни одного адреса, через который можно стать менеджером.

Часть 8. Сайт: запуск, настройки, связь с сервером

Что нужно знать про React, чтобы читать дальше:

- **Компонент** — функция, которая возвращает разметку (JSX). Пример: `const HomePage = () => { return (<div>...</div>); };`. Используется как тег: `<HomePage />`.
- **JSX** — HTML-подобная запись внутри JavaScript. В фигурных скобках `{...}` — любое JS-выражение: `<h1>{product.model_name}</h1>`. Вместо `class` пишется `className`. Браузер JSX не понимает — его переводит в обычный JS сборщик Vite.
- **Пропсы (props)** — данные, которые родитель передаёт компоненту как атрибуты: `<ManagerProductsPage token={token} />` → внутри `({ token }) => ...`.
- **`useState(начальное)`** — хранит значение в компоненте. Возвращает пару `[значение, функция_изменения]`. Когда вызывают функцию изменения, React **перерисовывает** компонент с новым значением.
- **`useEffect(функция, [зависимости])`** — выполнить код **после** отрисовки. `[]` — один раз при открытии страницы. `[id]` — каждый раз, когда меняется `id`. Если функция внутри возвращает другую функцию, та вызывается при уходе со страницы (очистка).
- **Условная отрисовка:** `{условие && <div/>}` — показать, только если условие истинно. `{условие ? <A/> : <B/>}` — одно или другое.
- **Списки:** `{массив.map((x) => <div key={x.id}>...</div>)}` — нарисовать элемент для каждого. `key` — уникальный ключ, по нему React понимает, какой элемент изменился.
- **`className="..."`** со словами вроде `bg-black text-white py-4 md:px-8 text-[18px]` — это **Tailwind**: каждое слово — один стиль. `bg-black` — чёрный фон, `py-4` — отступ сверху и снизу, `md:` — только на экранах от среднего размера, `[18px]` — точное значение. Классы стилей ниже подробно не расписываются — они отвечают только за внешний вид.

8.1 front/index.html — единственная HTML-страница

```
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7      <title>Darin's Handmade</title>
8    </head>
9    <body>
10     <div id="root"></div>
11     <script type="module" src="/src/main.jsx"></script>
12   </body>
13 </html>
```

- **стр. 1–2** — HTML-документ, язык английский.
- **стр. 4** — кодировка UTF-8.
- **стр. 5** — иконка вкладки.

- **стр. 6** — подстройка под ширину экрана телефона.
- **стр. 7** — заголовок вкладки.
- **стр. 10** — пустой `div` с `id="root"` — сюда React нарисует весь сайт.
- **стр. 11** — подключение `src/main.jsx` — с него начинается приложение. `type="module"` — современный JavaScript с `import`.

Сайт — **одностраничное приложение (SPA)**: браузер загружает один `index.html`, а все «страницы» рисует React, меняя содержимое `#root` без перезагрузки.

8.2 front/package.json — библиотеки сайта

- **стр. 6–11** `"scripts"`: `npm run dev` — `vite` (сайт в режиме разработки на `localhost:5173`); `npm run build` — `vite build` (собрать готовые файлы в папку `dist` для публикации; так делает Render); `npm run lint` — проверка кода ESLint; `npm run preview` — посмотреть собранную версию.
- **стр. 12–22** `"dependencies"`: `react`, `react-dom` — сама библиотека и вывод в браузер; `react-router-dom` — страницы и переходы; `axios` — запросы к серверу; `tailwindcss` и `@tailwindcss/vite` — стили; `framer-motion` — анимации; `lucide-react` — иконки; `react-image-magnifier-lens` — установлена, но в коде не используется.
- **стр. 23–33** `"devDependencies"`: инструменты разработки — `vite`, `@vitejs/plugin-react`, `eslint` и его плагины, типы для редактора.

8.3 front/vite.config.js — настройка сборщика

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3 import tailwindcss from '@tailwindcss/vite'
4
5 export default defineConfig({
6   plugins: [react(), tailwindcss()],
7 })
```

- **стр. 1** — `defineConfig` — помощник Vite для файла настроек (подсказки в редакторе).
- **стр. 2** — плагин React: учит Vite понимать JSX.
- **стр. 3** — плагин Tailwind: превращает классы вроде `bg-black` в CSS.
- **стр. 6** — подключаем оба плагина.

8.4 front/eslint.config.js — правила проверки кода

- **стр. 1–5** — подключение наборов правил: базовые правила JS, браузерные глобальные переменные, правила хуков React, правила быстрой перезагрузки Vite.
- **стр. 8** — не проверять папку `dist` (собранные файлы).
- **стр. 10** — проверять все `.js` и `.jsx`.
- **стр. 11–15** — какие наборы правил включить.
- **стр. 17** — разрешить браузерные переменные (`window`, `localStorage`).
- **стр. 18** — разрешить JSX.

На работу сайта не влияет — только подсказывает ошибки в редакторе и в `npm run lint`.

8.5 front/public/_redirects

```
/*    /index.html   200
```

Правило для хостинга: на **любой** адрес (`/*`) отдавать `index.html` с кодом 200. Нужно, потому что адреса вроде `/products/123` существуют только внутри React: без этого правила при обновлении страницы хостинг искал бы файл `products/123` и отвечал 404.

8.6 front/src/index.css — общие стили

```
@import url("https://fonts.googleapis.com/css2?family=Dorsa&display=swap");
@import "tailwindcss";

* {
  box-sizing: border-box;
}

html,
body,
#root {
  margin: 0;
  padding: 0;
  min-height: 100%;
}

body {
  background: #f3f3f1;
}
```

- **стр. 1** — шрифт Dorsa (высокие узкие буквы заголовков) с Google Fonts.
- **стр. 2** — подключение Tailwind.
- **стр. 4–6** — для всех элементов: ширина включает рамку и отступы (так проще считать размеры).
- **стр. 8–14** — убрать стандартные отступы браузера, растянуть на всю высоту.
- **стр. 16–18** — светло-серый фон страницы.

front/src/shared/ui/Header.css — файл со стилями `.header`, `.header__nav`, `.header__link`. **Нигде не подключается** — остался от старой версии шапки, сейчас шапка оформлена классами Tailwind.

8.7 front/src/main.jsx — точка входа React

Кто подключает: `index.html` (стр. 11).

```
1  import React from 'react';
2  import { createRoot } from 'react-dom/client';
3  import { BrowserRouter } from 'react-router-dom';
4  import App from './App';
5  import './index.css';
6
7  // BrowserRouter has to wrap the whole app, so that every page and the
8  // header can use Link and useNavigate.
9  createRoot(document.getElementById('root')).render(
10   <BrowserRouter>
11     <App />
12   </BrowserRouter>
13 );
```

- **стр. 1** — React (нужен для JSX).
- **стр. 2** — `createRoot` из `react-dom/client` — подключить React к HTML-элементу.
- **стр. 3** — `BrowserRouter` из `react-router-dom` — включает работу с адресами страниц.
- **стр. 4** — главный компонент `App` из `App.jsx`.
- **стр. 5** — подключение стилей (Vite вставит их на страницу).
- **стр. 9** — `document.getElementById('root')` находит `<div id="root">` из `index.html`; `createRoot(...)` делает его корнем React; `.render(...)` — нарисовать туда.
- **стр. 10–12** — `App` обёрнут в `BrowserRouter`. Без этой обёртки `Link`, `NavLink`, `useNavigate`, `useParams` не работали бы ни в одном компоненте.

8.8 front/src/api.jsx — связь с сервером

Что это за файл. Единственное место, где записан адрес сервера, плюс две вспомогательные функции. **Кто подключает:** `App.jsx`, `CartContext.jsx`, и страницы `LoginPage`, `ForgotPasswordPage`, `ResetPasswordPage`, `BeadedBagsPage`, `BraceletsPage`, `ProductDetailPage`, `CheckoutPage`, `OrderSuccessPage`, `ManagerProductsPage`, `ManagerOrdersPage`. **Что подключает:** `axios` (стр. 1).

```
1  import axios from 'axios';
2
3  // One place that knows the address of the server. On the development machine
4  // it is the local server; anywhere else it is the deployed one.
5  const isLocalhost =
6    typeof window !== 'undefined' &&
7    /^(localhost|127\.0\.0\.1)$/.test(window.location.hostname);
8
9  export const API_BASE_URL = isLocalhost
10   ? 'http://localhost:5000'
11   : import.meta.env.VITE_API_URL || 'https://darinshandmade.onrender.com';
12
13 // Same value under the shorter name used by the order and account pages.
14 export const API = API_BASE_URL;
```

- **стр. 1** — библиотека axios — отправка HTTP-запросов.
- **стр. 5–7** — `isLocalhost` — открыт ли сайт на своём компьютере. `typeof window !== 'undefined'` — мы в браузере. `window.location.hostname` — имя сайта в адресной строке. Регулярное выражение проверяет, что это `localhost` или `127.0.0.1`.

- **стр. 9–11** — адрес сервера: на своём компьютере — `http://localhost:5000`; иначе — переменная `VITE_API_URL` (Vite подставляет её при сборке через `import.meta.env`), а если её нет — адрес на Render.
- **стр. 14** — тот же адрес под коротким именем `API`. Сейчас в коде не используется.

```
16 const api = axios.create({
17   baseURL: API_BASE_URL,
18 });
```

- **стр. 16–18** — `axios.create` — настроенная копия axios. Теперь `api.get('/api/products')` сам добавит адрес сервера в начало. Методы: `api.get(адрес, настройки)`, `api.post(адрес, тело, настройки)`, `api.put`, `api.patch`, `api.delete(адрес, настройки)`. Каждый возвращает Promise: `.then(response => ...)` — успех (код 2xx), `.catch(error => ...)` — ошибка (сеть или код 4xx/5xx). Ответ сервера лежит в `response.data`; сообщение ошибки — в `error.response.data.message`.

```
20 // Builds the full address of a picture that the server keeps in /uploads,
21 // for example: imageUrl('uploads/pink.jpg') -> http://localhost:5000/uploads/pink.jpg
22 export const imageUrl = (fileName) => {
23   if (!fileName) return '';
24   return `${API_BASE_URL}/${String(fileName).replace(/^\//, '')}`;
25 };
```

Функция `imageUrl(fileName)` — полный адрес картинки товара.

- **Кто вызывает:** `CartContext` (стр. 46, 121), `BeadedBagsPage`, `BraceletsPage`, `ProductDetailPage`, `ManagerProductsPage`.
- **стр. 23** — имени нет — пустая строка.
- **стр. 24** — адрес сервера + `/` + путь. `replace(/^\//, '')` — убрать слеш в начале пути, если он есть, чтобы не получилось `//`. Картинку по этому адресу отдаёт `express.static` на сервере.

```
27 // The token is kept in localStorage after login and has to be sent with
28 // every protected request, in the header the server expects.
29 export const authHeader = (token) => {
30   return { headers: { Authorization: `Bearer ${token}` } };
31 };
32
33 export default api;
```

Функция `authHeader(token)` — настройки запроса с заголовком входа.

- **Кто вызывает:** `App.jsx`, `CartContext.jsx`, `CheckoutPage`, `ManagerProductsPage`, `ManagerOrdersPage`.
- **стр. 30** — возвращает `{ headers: { Authorization: 'Bearer <токен>' } }`. Передаётся последним аргументом в `api.get/post/...`. На сервере этот заголовок читает `protect` (стр. 7).
- **стр. 33** — `export default api` — главный экспорт файла. Импортируется как `import api, { imageUrl, authHeader } from '../api'` — `api` без скобок, остальное в фигурных скобках.

8.9 front/src/App.jsx — главный компонент, вход и маршруты

Кто подключает: `main.jsx` (стр. 4).

Что подключает:

```
1  import React, { useState, useEffect } from "react";
2  import { Routes, Route, Navigate } from "react-router-dom";
3  import api, { authHeader } from "./api";
4  import { CartProvider } from "./context/CartContext";
5
6  import Header from "./shared/ui/Header";
7  import Footer from "./shared/ui/Footer";
8  import HomePage from "./pages/home/HomePage";
...
20 import ManagerProductsPage from "./pages/admin/ManagerProductsPage";
21 import ManagerOrdersPage from "./pages/admin/ManagerOrdersPage";
22 import PrivacyPolicyPage from "./pages/legal/PrivacyPolicyPage";
23 import TermsOfServicePage from "./pages/legal/TermsOfServicePage";
```

- **стр. 1** — React и два хука.
- **стр. 2** — из роутера: `Routes` (выбор страницы по адресу), `Route` (одно правило «адрес → страница»), `Navigate` (переход).
- **стр. 3** — `api` и `authHeader` из `api.jsx`.
- **стр. 4** — `CartProvider` из `CartContext.jsx` — «раздатчик» корзины.
- **стр. 6–7** — шапка и подвал.
- **стр. 8–23** — все 16 страниц (по одной строке импорта на страницу).

```
25 // Reads the saved user out of localStorage. It is wrapped in try/catch
26 // because localStorage holds text, and broken text would crash JSON.parse.
27 const readUser = () => {
28   try {
29     return JSON.parse(localStorage.getItem("user")) || null;
30   } catch {
31     return null;
32   }
33 };
```

Функция `readUser()` — достать сохранённого пользователя из браузера. Вызывается на стр. 37 и 42.

- **стр. 29** — `localStorage` — хранилище браузера (встроенное, не библиотека). Хранит только строки и не очищается при перезагрузке. `getItem("user")` — строка, `JSON.parse` — превращает её обратно в объект. Если ничего нет — `null`.
- **стр. 30–32** — если строка испорчена, `JSON.parse` бросит ошибку — возвращаем `null`, сайт не падает.

Компонент `App` (стр. 35–103)

```
35 const App = () => {
36   const [token, setToken] = useState(localStorage.getItem("token"));
37   const [user, setUser] = useState(readUser);
```

- **стр. 36** — `token` — токен входа. Начальное значение — из `localStorage` (если раньше входили). `setToken` — функция изменения: её получают `LoginPage` (после входа) и `Header` (при выходе).
- **стр. 37** — `user` — данные пользователя. `useState(readUser)` — передана **сама функция**, а не её результат: React вызовет её один раз при первой отрисовке.

```
39   useEffect(() => {
40     const sync = () => {
41       setToken(localStorage.getItem("token"));
42       setUser(readUser());
43     };
44     window.addEventListener("storage", sync);
45     return () => window.removeEventListener("storage", sync);
46   }, []);
```

Эффект 1 — синхронизация вкладок (один раз при запуске, `[]`).

- **стр. 40–43** — функция `sync` перечитывает токен и пользователя из `localStorage`.
- **стр. 44** — событие `storage` браузер присылает, когда `localStorage` изменили **в другой вкладке** этого же сайта. Например, вышли в другой вкладке — эта тоже узнает.
- **стр. 45** — функция очистки: отписаться от события, когда `App` исчезает.

```
48   useEffect(() => {
49     if (!token) {
50       setUser(null);
51       return;
52     }
53     // Ask the server who this user is, so the role used by the /admin guard
54     // comes from the server and not from an old copy in localStorage.
55     api
56       .get("/api/auth/me", authHeader(token))
57       .then((response) => {
58         setUser(response.data.user);
59         localStorage.setItem("user", JSON.stringify(response.data.user));
60       })
61       .catch((error) => {
62         console.log(error);
63       });
64   }, [token]);
```

Эффект 2 — узнать у сервера, кто вошёл (при запуске и каждый раз, когда меняется `token`).

- **стр. 49–52** — токена нет (не вошли или вышли) — пользователя нет.
- **стр. 55–56** — запрос `GET /api/auth/me` с токеном. На сервере: `authRoute.js` стр. 16 → `protect` → `getMe`.
- **стр. 58** — сервер вернул пользователя с **ролью из базы** — сохраняем в состоянии.
- **стр. 59** — и в `localStorage` (`JSON.stringify` — объект в строку).
- **стр. 61–63** — ошибка (например, токен истёк) — пишем в консоль.

Зачем это: роль в `localStorage` можно подделать в инструментах разработчика. Здесь она каждый раз берётся с сервера. (А даже если подделать — сервер всё равно не пустит: у него своя проверка `protect` + `restrictTo`.)

```
66   const isManager = user?.role === "manager" || user?.role === "admin";
```

- **стр. 66** — `true`, если роль менеджер или админ. `user?.role` — не упасть, если `user` = `null`.

```
68   return (
69     <CartProvider token={token}>
70       <Header token={token} setToken={setToken} setUser={setUser} user={user} />
71
72       <div>
73         <Routes>
74           <Route path="/" element={<HomePage />} />
75           <Route path="/about" element={<AboutPage />} />
76           <Route path="/beaded-bags" element={<BeadedBagsPage />} />
77           <Route path="/bracelets" element={<BraceletsPage />} />
78
79           <Route path="/custom-studio" element={<CustomStudioPage />} />
80           <Route path="/login" element={<LoginPage setToken={setToken} setUser={setUser} />} />
81           <Route path="/forgot-password" element={<ForgotPasswordPage />} />
82           <Route path="/reset-password" element={<ResetPasswordPage />} />
83           <Route path="/cart" element={<CartPage />} />
84           <Route path="/checkout" element={<CheckoutPage />} />
85           <Route path="/order-success" element={<OrderSuccessPage />} />
86           <Route path="/products/:id" element={<ProductDetailPage />} />
87           <Route path="/privacy-policy" element={<PrivacyPolicyPage />} />
88           <Route path="/terms-of-service" element={<TermsOfServicePage />} />
89           <Route
90             path="/admin"
91             element={isManager ? <ManagerProductsPage token={token} /> : <Navigate to="/login"
replace />}
92           />
93           <Route
94             path="/admin/orders"
95             element={isManager ? <ManagerOrdersPage token={token} /> : <Navigate to="/login"
replace />}
96           />
97         </Routes>
98       </div>
99
100      <Footer />
101    </CartProvider>
102  );
103 };
```

- **стр. 69** — всё приложение внутри `CartProvider`; ему передаём `token` — корзина знает, вошёл ли человек. Поэтому корзина доступна на любой странице.
- **стр. 70** — шапка: ей нужны токен (показать Login или Logout), функции `setToken` / `setUser` (для выхода) и `user` (показать ссылку MANAGE).
- **стр. 73** — `Routes` смотрит на текущий адрес и рисует **один** подходящий `Route`.
- **стр. 74–88** — правила «адрес → страница». `path` — адрес, `element` — что нарисовать.
- **стр. 80** — странице входа передаём `setToken` и `setUser`, чтобы после входа обновить `App`.
- **стр. 86** — `/products/:id` — `:id` — параметр: любой текст на этом месте, страница получит его через `useParams`.
- **стр. 89–92** — `/admin`: если `isManager` — страница товаров менеджера (ей передаём `token` для запросов); иначе `<Navigate to="/login" replace />` — сразу перейти на вход. `replace` — заменить запись в истории, чтобы кнопка «Назад» не возвращала на закрытую страницу.
- **стр. 93–96** — `/admin/orders` — то же для заказов.
- **стр. 100** — подвал. Шапка и подвал вне `Routes`, поэтому видны на всех страницах.

- **стр. 105** — `export default App`.

8.10 front/src/context/CartContext.jsx — корзина для всего сайта

Что это за файл. Хранит корзину и функции работы с ней. Любой компонент получает их через `useCart()`, без передачи пропсами через всю цепочку. **Кто подключает:** `App.jsx` (`CartProvider`), `Header`, `BeadedBagsPage`, `BraceletsPage`, `ProductDetailPage`, `CustomStudioPage`, `CartPage`, `CheckoutPage`, `OrderSuccessPage` (`useCart`). **Что подключает:** из `react` — `createContext`, `useContext`, `useState`, `useEffect`, `useRef` (стр. 1); из `api.jsx` — `api`, `imageUrl`, `authHeader` (стр. 2).

Две ветки во всех функциях:

- **гость** (нет `token`) — корзина в `localStorage` под ключом `guestCart`;
- **вошедший** — товары каталога на сервере (адреса `/api/cart`), сумки из конструктора — в `localStorage` под ключом `customItems` (у них нет записи в базе, сервер их хранить не может).

Одна позиция корзины на фронте выглядит так: `{ id, name, category, image, price, quantity }`, у кастомной сумки ещё `config`.

```
4  // A context lets every page read the cart without passing it down as a prop.
5  const CartContext = createContext();
```

- **стр. 5** — `createContext()` создаёт «канал», по которому данные корзины раздаются всем компонентам.

```
7  // A visitor who is not signed in still gets a cart; it lives in this browser.
8  const GUEST_KEY = 'guestCart';
9
10 const loadGuestCart = () => {
11   try {
12     return JSON.parse(localStorage.getItem(GUEST_KEY)) || [];
13   } catch {
14     return [];
15   }
16 };
17
18 const saveGuestCart = (items) => {
19   localStorage.setItem(GUEST_KEY, JSON.stringify(items));
20 };
```

- **стр. 8** — имя ключа в `localStorage` для гостевой корзины.
- **стр. 10–16** — **`loadGuestCart()`** — прочитать гостевую корзину: строка → массив; нет или испорчена — пустой массив.
- **стр. 18–20** — **`saveGuestCart(items)`** — сохранить массив строкой.

```
22 // A bag from the Custom Studio is not a product from the catalog, so the
23 // server cart cannot hold it. For a signed in user it lives in this browser.
24 const CUSTOM_KEY = 'customItems';
25
26 const loadCustomItems = () => { ...то же, что loadGuestCart, но для CUSTOM_KEY... };   // стр. 26–
32
34 const saveCustomItems = (items) => { localStorage.setItem(CUSTOM_KEY, JSON.stringify(items)); };
// стр. 34–36
37
38 // Custom bags get an id like "custom-1718000000000".
39 const isCustom = (id) => String(id).startsWith('custom-');
```

- **стр. 24** — ключ для кастомных сумок вошедшего пользователя.
- **стр. 26–32** — **`loadCustomItems()`** — прочитать их (так же, как `loadGuestCart`).
- **стр. 34–36** — **`saveCustomItems(items)`** — сохранить.
- **стр. 39** — **`isCustom(id)`** — сумка ли это из конструктора: её id начинается с `custom-` (так его создаёт `CustomStudioPage`, стр. 187). `String(id)` — на случай, если id не строка.

```
41 // Turns one row of the server cart into the simple shape the pages display.
42 const toCartItem = (item) => ({
43   id: item.product._id,
44   name: item.product.model_name,
45   category: item.product.category,
46   image: imageUrl(item.product.file_name),
47   price: item.product.price,
48   quantity: item.quantity
49 });
```

- **`toCartItem(item)`** — превращает позицию серверной корзины (после `populate` в ней целый товар `item.product`) в простой объект для страниц. Вызывается из `fetchCart` (стр. 67).
- **стр. 46** — полный адрес картинки через `imageUrl`.

Компонент `CartProvider` (стр. 51–249)

```
51 const CartProvider = ({ children, token }) => {
52   const [items, setItems] = useState([]);
53
54   // Remembers the previous token, so we can tell a fresh login from a redraw.
55   const prevToken = useRef(null);
```

- **стр. 51** — получает `children` (всё, что вложено внутрь `<CartProvider>` в `App.jsx`, — то есть весь сайт) и `token`.
- **стр. 52** — `items` — позиции корзины, которые видят все страницы.
- **стр. 55** — `useRef` — хранит значение между отрисовками, но его изменение **не** вызывает перерисовку. Здесь — предыдущий токен, чтобы понять, что человек **только что** вошёл.

Функция `fetchCart()` (стр. 58–72) — загрузить корзину

- **Кто вызывает:** эффект (стр. 80, 87, 105), `addToCart` (стр. 135), `removeFromCart` (стр. 160), `updateQuantity` при ошибке (стр. 202), `clearCart` (стр. 218).

```
58   const fetchCart = () => {
59     if (!token) {
60       setItems(loadGuestCart());
61       return;
62     }
63
64     api
65       .get('/api/cart', authHeader(token))
66       .then((response) => {
67         setItems([...response.data.data.items.map(toCartItem), ...loadCustomItems()]);
68       })
69       .catch((error) => {
70         console.log(error);
71       });
72   };
```

- **стр. 59–62** — гость — берём корзину из `localStorage`.
- **стр. 64–65** — вошедший — `GET /api/cart` → на сервере `getCart`.
- **стр. 67** — `response.data.data.items` — позиции серверной корзины (первое `data` — от axios, второе — поле, которое отправил сервер). `map(toCartItem)` — в простой вид. Затем `...loadCustomItems()` — добавляем сумки из конструктора. `...` (spread) — «высыпать» элементы массива в новый массив.

Эффект: при входе и выходе (стр. 75–107)

```
75   useEffect(() => {
76     const justLoggedIn = token && !prevToken.current;
77     prevToken.current = token;
78
79     if (!justLoggedIn) {
80       fetchCart();
81       return;
82     }
83
84     // The visitor filled a cart before logging in, so move it to the server.
85     const guestItems = loadGuestCart();
86     if (guestItems.length === 0) {
87       fetchCart();
88       return;
89     }
90
91     // Custom bags cannot go to the server, so they move to their own list.
92     const customItems = guestItems.filter((item) => isCustom(item.id));
93     saveCustomItems([...loadCustomItems(), ...customItems]);
94
95     // One request per unit, because the server adds one item at a time.
96     const products = guestItems.filter((item) => !isCustom(item.id));
97     const requests = products.flatMap((item) =>
98       Array.from({ length: item.quantity }, () =>
99         api.post('/api/cart', { productId: item.id }, authHeader(token))
100      )
101    );
102
103    Promise.allSettled(requests).then(() => {
104      localStorage.removeItem(GUEST_KEY);
105      fetchCart();
106    });
107  }, [token]);
```

- **стр. 107** — `[token]` — эффект срабатывает при первой отрисовке и каждый раз, когда меняется токен (вход, выход).
- **стр. 76** — «только что вошёл»: токен есть сейчас, а раньше не было.
- **стр. 77** — запоминаем текущий токен на следующий раз.
- **стр. 79–82** — не вход (первая загрузка, выход) — просто загружаем корзину.
- **стр. 85–89** — гостевая корзина пустая — переносить нечего, загружаем серверную.
- **стр. 92–93** — кастомные сумки гостя переносим в `customItems` (к тем, что уже там есть).
- **стр. 96** — обычные товары гостя.
- **стр. 97–101** — для каждого товара делаем столько запросов `POST /api/cart`, сколько штук (сервер добавляет по одной). `Array.from({ length: 3 }, () => запрос)` — массив из трёх запросов. `flatMap` склеивает массивы всех товаров в один.
- **стр. 103** — `Promise.allSettled` ждёт **все** запросы, даже если какие-то упали.
- **стр. 104** — гостевую корзину стираем.
- **стр. 105** — загружаем итоговую корзину с сервера.

Функция `addToCart(product)` (стр. 109–141) — добавить товар каталога

- **Кто вызывает:** кнопки Add to Cart в `BeadedBagsPage`, `BraceletsPage`, `ProductDetailPage`.

- **Получает:** товар в том виде, как его вернул сервер (`_id`, `model_name`, …).

```
109  const addToCart = (product) => {
110    if (!token) {
111      const guest = loadGuestCart();
112      const existing = guest.find((item) => item.id === product._id);
113
114      if (existing) {
115        existing.quantity += 1;
116      } else {
117        guest.push({
118          id: product._id,
119          name: product.model_name,
120          category: product.category,
121          image: imageUrl(product.file_name),
122          price: product.price,
123          quantity: 1
124        });
125      }
126
127      saveGuestCart(guest);
128      setItems([...guest]);
129      return;
130    }
131
132    api
133      .post('/api/cart', { productId: product._id }, authHeader(token))
134      .then(() => {
135        fetchCart();
136      })
137      .catch((error) => {
138        console.log(error);
139        alert(error.response?.data?.message || 'Failed to add to cart');
140      });
141  };
```

- **стр. 110–130** — **гость**:
  - **стр. 111** — читаем корзину;
  - **стр. 112** — ищем этот товар (`find` — первый подходящий или `undefined`);
  - **стр. 114–115** — есть — +1;
  - **стр. 116–125** — нет — добавляем позицию в простом виде;
  - **стр. 127** — сохраняем в браузере;
  - **стр. 128** — `setItems([...guest])` — **новый** массив (копия). React перерисовывает, только если получил новый массив, а не тот же изменённый.
- **стр. 132–136** — **вошедший**: `POST /api/cart` → на сервере `addToCart`; после успеха перезагружаем корзину.
- **стр. 139** — ошибка — всплывающее окно с сообщением сервера.

Функция `removeFromCart(productId)` (стр. 143–165) — убрать позицию

- **Кто вызывает:** `CartPage` — иконка мусорки.

```
143  const removeFromCart = (productId) => {
144    if (!token) {
145      const guest = loadGuestCart().filter((item) => item.id !== productId);
146      saveGuestCart(guest);
147      setItems(guest);
148      return;
149    }
150
151    if (isCustom(productId)) {
152      saveCustomItems(loadCustomItems().filter((item) => item.id !== productId));
153      setItems(items.filter((item) => item.id !== productId));
154      return;
155    }
156
157    api
158      .delete(`/api/cart/${productId}`, authHeader(token))
159      .then(() => {
160        fetchCart();
161      })
...
165  };
```

- **стр. 144–149** — гость: оставляем всё, кроме этой позиции, сохраняем.
- **стр. 151–155** — вошедший и это кастомная сумка: убираем из `customItems` и из списка на экране. На сервер не ходим — там её нет.
- **стр. 157–161** — вошедший и товар каталога: `DELETE /api/cart/:id` → на сервере `removeFromCart`, потом перезагрузка.

Функция `addCustomItem(item)` (стр. 169–178) — добавить сумку из конструктора

- **Кто вызывает:** `CustomStudioPage` (`handleAddToCart`, стр. 186).

```
169  const addCustomItem = (item) => {
170    const updated = [...items, item];
171    setItems(updated);
172
173    if (!token) {
174      saveGuestCart(updated);
175      return;
176    }
177    saveCustomItems([...loadCustomItems(), item]);
178  };
```

- **стр. 170–171** — добавляем на экран.
- **стр. 173–176** — гость — сохраняем всю корзину в `guestCart`.
- **стр. 177** — вошедший — добавляем сумку в `customItems`.

Функция `updateQuantity(id, newQuantity)` (стр. 180–204) — изменить количество

- **Кто вызывает:** `CartPage` — кнопки − и +.

```
180  const updateQuantity = (id, newQuantity) => {
181    if (newQuantity < 1) return;
182    const updated = items.map((item) =>
183      item.id === id ? { ...item, quantity: newQuantity } : item
184    );
185    setItems(updated);
186
187    if (!token) {
188      saveGuestCart(updated);
189      return;
190    }
191
192    if (isCustom(id)) {
193      saveCustomItems(updated.filter((item) => isCustom(item.id)));
194      return;
195    }
196
197    // The new number is already on the screen; the server keeps it for next time.
198    api
199      .patch(`/api/cart/${id}`, { quantity: newQuantity }, authHeader(token))
200      .catch((error) => {
201        console.log(error);
202        fetchCart();
203      });
204  };
```

- **стр. 181** — меньше 1 нельзя (для удаления — мусорка).
- **стр. 182–184** — новый массив: у нужной позиции копия с новым количеством (`{ ...item, quantity }`), остальные без изменений.
- **стр. 185** — сразу показываем на экране.
- **стр. 187–190** — гость — сохраняем в браузере.
- **стр. 192–195** — кастомная сумка вошедшего — сохраняем все кастомные позиции в `customItems`.
- **стр. 198–199** — товар каталога вошедшего — `PATCH /api/cart/:id` → на сервере `updateQuantity`.
- **стр. 200–203** — не получилось — перечитываем корзину с сервера, чтобы на экране было правильное число.

Функция `clearCart()` (стр. 206–223) — очистить корзину

- **Кто вызывает:** `CartPage` (кнопка Clear all items), `OrderSuccessPage` (после оплаты).

```
206  const clearCart = () => {
207    setItems([]);
208
209    if (!token) {
210      localStorage.removeItem(GUEST_KEY);
211      return;
212    }
213
214    localStorage.removeItem(CUSTOM_KEY);
215    api
216      .delete('/api/cart', authHeader(token))
217      .then(() => {
218        fetchCart();
219      })
...
223  };
```

- **стр. 207** — экран — пустая корзина.
- **стр. 209–212** — гость — удалить ключ из браузера.
- **стр. 214** — вошедший — удалить кастомные сумки.
- **стр. 215–219** — `DELETE /api/cart` → на сервере `clearCart`, потом перечитать.

Подсчёты (стр. 225–231)

```
225  const getTotalPrice = () => {
226    return items.reduce((total, item) => total + item.price * item.quantity, 0);
227  };
228
229  const getTotalItems = () => {
230    return items.reduce((total, item) => total + item.quantity, 0);
231  };
```

- **`getTotalPrice()`** — сумма корзины: `reduce` начинает с 0 и прибавляет цену × количество каждой позиции. Вызывают `CartPage`, `CheckoutPage`.
- **`getTotalItems()`** — сколько штук всего. Вызывает `Header` (число на значке корзины).

Что раздаётся всем (стр. 233–248)

```
233  return (
234    <CartContext.Provider
235      value={{
236        items,
237        addToCart,
238        addCustomItem,
239        removeFromCart,
240        updateQuantity,
241        getTotalPrice,
242        clearCart,
243        getTotalItems
244      }}
245    >
246      {children}
247    </CartContext.Provider>
248  );
```

- **стр. 234–245** — `Provider` раздаёт в «канал» объект с корзиной и всеми функциями.
- **стр. 246** — внутри рисуется всё приложение (`children`).

```
251 // Short way for a page to read the cart: const { items } = useCart();
252 const useCart = () => useContext(CartContext);
253
254 export { CartProvider, useCart };
```

- **стр. 252** — **`useCart()`** — свой хук: `useContext(CartContext)` достаёт объект из «канала». Страница пишет `const { items, addToCart } = useCart();`.
- **стр. 254** — экспорт двух имён.

Часть 9. Сайт: шапка и подвал

9.1 front/src/shared/ui/Header.jsx — шапка

Кто подключает: `App.jsx` (стр. 6, 70). **Что подключает:**

- **стр. 1** — React, `useState`;
- **стр. 2** — `NavLink` (ссылка, которая знает, активна ли она) и `useNavigate` из `react-router-dom`;
- **стр. 3** — иконки `ShoppingBag`, `User`, `LogOut`, `Menu`, `X` из `lucide-react` (каждая — готовый компонент-картинка);
- **стр. 4** — `useCart` из `CartContext.jsx`.

Пропсы: `token`, `setToken`, `setUser`, `user` — все из `App.jsx`.

```
6  const Header = ({ token, setToken, setUser, user }) => {
7    const navigate = useNavigate();
8    const [menuOpen, setMenuOpen] = useState(false);
9
10   const { getTotalItems } = useCart();
11
12   const baseClass =
13     "text-[#f5efe8] no-underline text-[18px] tracking-[0.18em]";
14
15   const isManager = user?.role === "manager" || user?.role === "admin";
16
17   const links = [
18     { to: "/beaded-bags", label: "BEADED BAGS" },
19     { to: "/about", label: "ABOUT" },
20     { to: "/", label: "HOME", end: true },
21     { to: "/bracelets", label: "BRACELETS" },
22     { to: "/custom-studio", label: "CUSTOM STUDIO" },
23     ...(isManager ? [{ to: "/admin", label: "MANAGE" }] : []),
24   ];
```

- **стр. 7** — `useNavigate()` возвращает функцию `navigate('/адрес')` — перейти на страницу из кода.
- **стр. 8** — открыто ли мобильное меню.
- **стр. 10** — из корзины берём только функцию подсчёта штук.
- **стр. 12–13** — общие стили ссылок (светлый цвет, без подчёркивания, размер, разрядка букв).
- **стр. 15** — менеджер ли пользователь.
- **стр. 17–24** — список ссылок меню. **стр. 20** — `end: true` у главной: ссылка `/` активна только на точном адресе `/`, иначе она была бы активна на любой странице. **стр. 23** — ссылка MANAGE добавляется только менеджеру: если `isManager` — массив с одной ссылкой, иначе пустой; `...` высыпает его в список.

```
26   const handleLogout = () => {
27     localStorage.removeItem("token");
28     localStorage.removeItem("user");
29     // Custom bags of this account must not stay for the next person on this browser.
30     localStorage.removeItem("customItems");
31     setToken(null);
32     setUser?.(null);
33     setMenuOpen(false);
34     navigate("/");
35   };
```

Функция `handleLogout()` — выход. Вызывается кнопкой Logout (стр. 60).

- **стр. 27–28** — стираем токен и пользователя из браузера.
- **стр. 30** — стираем кастомные сумки этого аккаунта.
- **стр. 31** — `setToken(null)` — функция из `App.jsx`: `App` перерисуется без токена, `CartContext` переключится на гостевую корзину.
- **стр. 32** — `setUser?.(null)` — вызвать, только если функцию передали (`?.` перед скобками).
- **стр. 33** — закрыть мобильное меню.
- **стр. 34** — перейти на главную.

На сервер при выходе запрос не нужен: сервер токены не хранит, браузер просто перестаёт его отправлять.

Разметка (стр. 37–118):

- **стр. 38** — `<header>` на всю ширину, чёрный фон.
- **стр. 39** — внутренний контейнер высотой 90px: элементы в строку, по краям.
- **стр. 40–47** — кнопка мобильного меню (видна только на узких экранах, `md:hidden`). `onClick={() => setMenuOpen((v) => !v)}` — переключить: `!v` — противоположное значение. Внутри иконка: открыто — крестик `X`, закрыто — три полоски `Menu`.
- **стр. 49–55** — меню для широких экранов (`hidden md:flex` — скрыто на телефоне). `links.map(...)` рисует `NavLink` для каждой ссылки: `to` — адрес, `end` — точное совпадение, текст — `label`.
- **стр. 57–99** — правая часть: вход/выход и корзина.
  - **стр. 58** — `{token ? (...) : (...)}` — если вошли — кнопка выхода, иначе ссылка на вход.
  - **стр. 59–68** — кнопка Logout: `onClick={handleLogout}`, иконка `LogOut`, слово «Logout» (скрыто на самых узких экранах).
  - **стр. 70–79** — ссылка на `/login` с иконкой `User` и словом «Login».
  - **стр. 82–98** — ссылка на `/cart`: иконка сумки и кружок с числом `{getTotalItems ? getTotalItems() : 0}` — сколько штук в корзине (если функция есть — вызвать, иначе 0).
- **стр. 102–116** — мобильное меню: показывается, только если `menuOpen` (`{menuOpen && (...)}`). Те же ссылки столбиком; клик по ссылке закрывает меню (`onClick={() => setMenuOpen(false)}`).
- **стр. 121** — `export default Header`.

9.2 front/src/shared/ui/Footer.jsx — подвал

Кто подключает: `App.jsx` (стр. 7, 100). **Что подключает:** React, `NavLink`. Логики нет — только разметка.

- **стр. 5–6** — `sectionTitleClass` — общие стили заголовков колонок.
- **стр. 8–9** — `footerLinkClass` — общие стили ссылок.
- **стр. 12** — `<footer>`, чёрный фон, отступы.
- **стр. 14** — сетка: 1 колонка на телефоне, 2 на планшете, 4 на компьютере.
- **стр. 16–29** — колонка 1: название DARIN'S и одна фраза.
- **стр. 31–47** — колонка SHOP: ссылки `NavLink` на `/beaded-bags`, `/bracelets`, `/custom-studio`.
- **стр. 49–59** — колонка INFO: ссылка на `/about`.
- **стр. 61–80** — колонка SOCIALS: обычные ссылки `<a>` на Instagram, Etsy, Facebook, TikTok. `target="_blank"` — открыть в новой вкладке; `rel="noopener noreferrer"` — новая вкладка не получит доступ к нашей (защита).
- **стр. 83** — разделительная линия.
- **стр. 85–110** — нижняя строка: копирайт и ссылки на `/privacy-policy` и `/terms-of-service`.

Часть 10. Сайт: главная страница (pages/home/)

10.1 HomePage.jsx — главная, собирает четыре секции

Кто подключает: `App.jsx` (адрес `/`). **Что подключает:** `useRef` (стр. 1) и четыре секции из этой же папки (стр. 2–5).

```
7  const HomePage = () => {
8      const categoriesRef = useRef(null);
9
10     const handleScrollToCategories = () => {
11         categoriesRef.current?.scrollIntoView({
12             behavior: "smooth",
13             block: "start",
14         });
15     };
16
17     return (
18         <div className="home-page">
19             <HeroSection onShopClick={handleScrollToCategories} />
20             <AboutSection />
21             <CustomBagSection />
22             <CategoriesSection categoriesRef={categoriesRef} />
23         </div>
24     );
25 };
```

- **стр. 8** — `useRef(null)` — «ссылка» на HTML-элемент секции категорий. Сама секция привяжет её к себе (`CategoriesSection`, стр. 14).
- **стр. 10–15** — **функция** `handleScrollToCategories()` — плавно прокрутить страницу к категориям. `categoriesRef.current` — сам HTML-элемент; `scrollIntoView` — встроенный метод браузера; `behavior: "smooth"` — плавно; `block: "start"` — верх секции к верху экрана. Вызывается кнопкой SHOP в `HeroSection`.
- **стр. 19** — первая секция; ей передаём функцию прокрутки как проп `onShopClick`.
- **стр. 20–21** — вторая и третья секции, без пропсов.
- **стр. 22** — секция категорий; ей передаём `ref`, чтобы она к нему привязалась.

Что такое `motion` (во всех секциях ниже). Библиотека `framer-motion`. `motion.div`, `motion.h1`, `motion.img` и т.д. — обычные теги с анимацией. Атрибуты: `initial` — как выглядит в начале (`opacity: 0` — прозрачный, `y: 30` — сдвинут на 30px вниз); `animate` — к чему прийти; `whileInView` — анимировать, когда элемент появился на экране при прокрутке; `viewport={{ once: true }}` — только один раз; `transition` — длительность (`duration`, секунды), задержка (`delay`) и кривая скорости (`ease`); `whileHover` / `whileTap` — при наведении / нажатии.

10.2 HeroSection.jsx — первый экран

Что подключает: `useState` (стр. 1), `Link` (стр. 2), `motion` (стр. 3), картинку `pinkBag.png` (стр. 4 — Vite превратит импорт картинки в её адрес). **Пропсы:** `onShopClick` — функция прокрутки из `HomePage`.

- **стр. 7** — `shopHovered` — наведена ли мышь на кнопку SHOP (для анимации).

- **стр. 10–11** — секция на всю ширину, светлый фон; внутри две колонки (на телефоне — одна под другой).
- **стр. 12–93** — левая колонка (58% ширины):
  - **стр. 13–21** — огромный заголовок «EXCLUSIVE HANDMADE», выезжает снизу и проявляется за 0,8 с.
  - **стр. 23–28** — чёрная полоса под заголовком, «вырастает» слева направо (`scaleX` от 0 до 1, `originX: 0` — от левого края).
  - **стр. 30–40** — подпись «BY DARIN'S» и короткая полоса.
  - **стр. 42–92** — две кнопки:
  - **стр. 48–77** — кнопка SHOP (`motion.button`). **стр. 50** `onClick={onShopClick}` — вызывает прокрутку из `HomePage`. **стр. 51–52** — при наведении `setShopHovered(true)`, при уходе — `false`. **стр. 53** — при нажатии чуть уменьшается. **стр. 57–62** — светлый круг внутри, появляется при наведении. **стр. 63–76** — текст SHOP: при наведении буквы раздвигаются (`letterSpacing`), справа выезжает стрелка `→`.
  - **стр. 79–91** — ссылка CUSTOM на `/custom-studio` (`Link` — переход без перезагрузки страницы).
- **стр. 95–108** — правая колонка: фото розовой сумки, выезжает справа; при наведении чуть увеличивается и поворачивается.

10.3 AboutSection.jsx — «Уникальность в деталях»

Что подключает: `motion` (стр. 2), картинку `AboutSectionBag.png` (стр. 3). Логики нет.

- **стр. 7** — секция; `-mt-[...]` — отрицательный верхний отступ, чтобы секция чуть «наехала» на предыдущую.
- **стр. 9–23** — фото чёрной сумки слева; выезжает слева при появлении на экране (`whileInView`).
- **стр. 25–49** — справа заголовок «Uniqueness in the details» (стр. 26–35) и абзац текста (стр. 37–48), оба проявляются снизу при прокрутке.

10.4 CustomBagSection.jsx — «Создай свой дизайн»

Что подключает: `useState` (стр. 1), `Link` (стр. 2), `motion` и `AnimatePresence` (стр. 3), пять фото сумок разных цветов (стр. 5–9).

```
11 const bagOptions = [
12   { color: "#25cfff", image: skyBag },
13   { color: "#2d46ff", image: blueBag },
14   { color: "#39c24a", image: greenBag },
15   { color: "#ff00f5", image: pinkBag },
16   { color: "#8d47ff", image: purpleBag },
17 ];
```

- **стр. 11–17** — пять вариантов: цвет кружка и фото сумки этого цвета.

```
19 const CustomBagSection = () => {
20   const [selected, setSelected] = useState(0);
21   const [createHovered, setCreateHovered] = useState(false);
```

- **стр. 20** — номер выбранного цвета (сначала 0 — голубой).
- **стр. 21** — наведена ли мышь на кнопку CREATE.
- **стр. 26–82** — левая колонка: заголовок «CREATE YOUR OWN DESIGN» (стр. 33–38), текст (стр. 40–46), кнопка CREATE — ссылка на `/custom-studio` (стр. 49–80) с той же анимацией наведения, что у SHOP.
- **стр. 84–101** — средняя колонка: пять круглых кнопок-цветов. `bagOptions.map((item, i) => ...)` — по кнопке на вариант. **стр. 89** `onClick={() => setSelected(i)}` — выбрать цвет. **стр. 93–97** — `style`: фон — цвет варианта; у выбранного (`selected === i`) толстая белая рамка и свечение.
- **стр. 103–116** — правая колонка: фото выбранной сумки. **стр. 104** `AnimatePresence mode="wait"` — даёт старой картинке анимированно исчезнуть, прежде чем появится новая. **стр. 106** `key={selected}` — при смене цвета React считает картинку новой и проигрывает анимацию. **стр. 107** — `bagOptions[selected].image` — фото выбранного цвета. **стр. 110–112** — появление, состояние и исчезновение.

10.5 CategoriesSection.jsx — две плитки категорий

Что подключает: `Link`, `motion`, два фото (стр. 4–5). **Пропсы:** `categoriesRef` — из `HomePage`.

```
7  const CategoriesSection = ({ categoriesRef }) => {
8    const categories = [
9      { title: "Beaded Bags", image: rectangleWithBag, link: "/beaded-bags" },
10     { title: "Bracelets",   image: rectangleBracelets, link: "/bracelets" },
11   ];
12
13   return (
14     <section ref={categoriesRef} className="w-full bg-black">
```

- **стр. 8–11** — две категории: название, фото, адрес.
- **стр. 14** — `ref={categoriesRef}` — привязываем «ссылку» из `HomePage` к этой секции. Теперь `categoriesRef.current` — этот элемент, и кнопка SHOP может к нему прокрутить.
- **стр. 15** — сетка: одна колонка на телефоне, две на компьютере.
- **стр. 16–54** — `categories.map((item, i) => ...)` — по плитке на категорию:
  - **стр. 17–23** — появляется снизу при прокрутке; `delay: i * 0.12` — вторая чуть позже первой;
  - **стр. 24–27** — вся плитка — ссылка `Link` на страницу категории; `group` — класс Tailwind, чтобы дочерние элементы реагировали на наведение на плитку;
  - **стр. 28–32** — фото; при наведении на плитку увеличивается (`group-hover:scale-[1.06]`);
  - **стр. 34** — затемнение, при наведении сильнее;
  - **стр. 36–41** — название категории сверху по центру;
  - **стр. 43–51** — надпись «Explore →» снизу, появляется только при наведении.

Часть 11. Сайт: страница «Обо мне»

11.1 front/src/pages/about/AboutPage.jsx

Кто подключает: `App.jsx` (адрес `/about`). **Что подключает:** `useState` (стр. 1), `motion` (стр. 2), фон `background.png` (стр. 3).

```
5  // Bead colors that match the beaded bags.
6  const BEAD_COLORS = ["#2F6BFF", "#1B3FD1", "#36C5F0", "#7FE3FF", "#BFEAFF", "#E8F7FF", "#9B8CFF"];
7
8  const BEAD_COUNT = 44;
9  const BEAD_SIZE_PX = { min: 8, max: 22 };
10 const FALL_SECONDS = { min: 6, max: 12 };
11 const SWAY_SECONDS = { min: 3, max: 6 };
12 const MAX_START_DELAY_S = 8;
13
14 // Random value in [min, max).
15 const rand = (min, max) => min + Math.random() * (max - min);
```

- **стр. 6** — цвета падающих бусин (оттенки синего).
- **стр. 8** — сколько бусин: 44.
- **стр. 9** — размер бусины от 8 до 22 пикселей.
- **стр. 10** — падение длится от 6 до 12 секунд.
- **стр. 11** — покачивание влево-вправо — от 3 до 6 секунд.
- **стр. 12** — начальная задержка — до 8 секунд, чтобы бусины не падали все сразу.
- **стр. 15** — **функция** `rand(min, max)` — случайное число от `min` до `max`.

```
17 const AboutPage = () => {
18   // Generate the beads once, on first render (random values, so not derived from props/state).
19   const [beads] = useState(() =>
20     Array.from({ length: BEAD_COUNT }).map((_, i) => ({
21       id: i,
22       left: rand(0, 100), // % across the screen
23       duration: rand(FALL_SECONDS.min, FALL_SECONDS.max),
24       delay: rand(0, MAX_START_DELAY_S),
25       size: rand(BEAD_SIZE_PX.min, BEAD_SIZE_PX.max),
26       sway: rand(SWAY_SECONDS.min, SWAY_SECONDS.max),
27       color: BEAD_COLORS[Math.floor(rand(0, BEAD_COLORS.length))],
28     }))
29   );
```

- **стр. 19** — `useState(() => ...)` — функция вызывается **один раз** при первой отрисовке, и бусины не пересоздаются при каждой перерисовке. Функцию изменения не берём (`[beads]`) — бусины не меняются.
- **стр. 20** — массив из 44 элементов; `map((_, i) => ...)` — для каждого создаём объект бусины (`_` — значение не нужно, нужен только номер `i`).
- **стр. 21–27** — у каждой бусины: номер; позиция по горизонтали в процентах; время падения; задержка; размер; время покачивания; случайный цвет из списка (`Math.floor` — целый номер цвета).

Разметка (стр. 31–189):

- **стр. 32–35** — весь экран, фоновая картинка (`backgroundImage` со ссылкой на файл).
- **стр. 36–91** — тег `<style>` с CSS прямо внутри компонента:
  - **стр. 37–42** — анимация `fall`: бусина едет сверху вниз за экран и вращается, в начале и конце прозрачная;
  - **стр. 44–47** — анимация `sway`: покачивание влево-вправо через `margin-left`;
  - **стр. 48–79** — класс `.bead`: круг, абсолютное положение, «кристальный» вид из нескольких градиентов (блик, грани, радужный отлив, основной цвет `var(--bead)`), тени, две анимации одновременно;
  - **стр. 81–90** — `.bead::after` — маленький блик на каждой бусине.
- **стр. 93** — тёмная полупрозрачная пелена поверх фона, чтобы текст читался.
- **стр. 95–108** — `beads.map(...)` — рисуем 44 `div` с классом `bead`. **стр. 99–106** — свои значения для каждой: позиция `left`, размер, цвет через CSS-переменную `--bead`, длительность и задержка двух анимаций.
- **стр. 110–187** — текст поверх бусин (`z-10` — выше):
  - **стр. 112–119** — маленькая надпись сверху;
  - **стр. 121–128** — заголовок «ABOUT ME»;
  - **стр. 130–135** — короткая линия;
  - **стр. 137** — сетка из трёх колонок: текст | линия | текст;
  - **стр. 139–151** — «Hi, I'm Darina!» и подпись;
  - **стр. 153–159** — вертикальная линия-разделитель (только на компьютере);
  - **стр. 161–182** — три абзаца о мастере: массив строк `.map(...)`, каждый абзац появляется чуть позже предыдущего (`delay: 0.7 + i * 0.12`).

Часть 12. Сайт: каталоги и карточка товара

12.1 front/src/pages/beaded-bags/BeadedBagsPage.jsx — каталог сумок

Кто подключает: `App.jsx` (адрес `/beaded-bags`). **Что подключает:** `useEffect`, `useState` (стр. 1); `Link` (стр. 2); `useCart` (стр. 3); `api`, `imageUrl` (стр. 4). **Обращается к серверу:** `GET /api/products` → `getAllProducts`.

```
6  const BeadedBagsPage = () => {
7    const [bags, setBags] = useState([]);
8    const [loading, setLoading] = useState(true);
9
10   const { items, addToCart } = useCart();
11
12   // Runs once when the page opens: bring the catalog and keep only the bags.
13   useEffect(() => {
14     api
15       .get("/api/products")
16       .then((response) => {
17         const bagsOnly = response.data.data.filter((product) => product.category === "bags");
18         setBags(bagsOnly);
19         setLoading(false);
20       })
21       .catch((error) => {
22         console.log(error);
23         setLoading(false);
24       });
25   }, []);
```

- **стр. 7** — список сумок (сначала пустой).
- **стр. 8** — идёт ли загрузка (сначала да).
- **стр. 10** — из корзины: текущие позиции (чтобы знать, что уже в корзине) и функция добавления.
- **стр. 13, 25** — эффект один раз при открытии (`[]`).
- **стр. 14–15** — запрос всех товаров. Токен не нужен — каталог открыт.
- **стр. 17** — оставляем только товары с категорией `bags`. Фильтр в браузере.
- **стр. 18–19** — сохраняем и выключаем «загрузку» — страница перерисуется с карточками.
- **стр. 21–24** — ошибка — в консоль, загрузку всё равно выключаем.

Разметка:

- **стр. 29–38** — тёмный верхний блок: заголовок «BEADED BAGS» и описание.
- **стр. 40–46** — блок с цитатой.
- **стр. 48–57** — заголовок «Featured Collection».
- **стр. 59–62** — пока `loading` — надпись «Loading beautiful bags...».
- **стр. 64–135** — иначе сетка карточек: `bags.map((bag) => ...)`:
  - **стр. 66** — `isInCart` — есть ли уже эта сумка в корзине: `items.some(...)` — есть ли хоть одна позиция с таким id;

- **стр. 67** — `outOfStock` — остаток ровно 0 (`null` — не отслеживается, значит в наличии);
- **стр. 72–79** — фото: `src={imageUrl(bag.file_name)}` — полный адрес картинки на сервере; при ошибке загрузки — сообщение в консоль; при наведении увеличивается;
- **стр. 81–91** — метка остатка, только если остаток — число: 0 — красная «Sold out», 1–3 — оранжевая «Only N left», больше — белая «N in stock»;
- **стр. 94–101** — название и первый стиль из массива (`bag.style?.[0]` — первый элемент, если массив есть; иначе «Standard»);
- **стр. 103–105** — первая деталь или «Handcrafted with love.»;
- **стр. 107–112** — ссылка «View Details →» на `/products/<id>` — карточка товара;
- **стр. 115–117** — цена;
- **стр. 119–130** — кнопка. **стр. 120** `onClick={() => !isInCart && !outOfStock && addToCart(bag)}` — добавить, только если ещё не в корзине и есть в наличии (`&&` — следующее выполняется, только если предыдущее истинно). **стр. 121** `disabled={outOfStock}` — неактивна, если нет в наличии. **стр. 122–127** — стиль зависит от состояния. **стр. 129** — текст: «Out of Stock», «In Cart» или «Add to Cart».

- **стр. 140–172** — блок «Care Instructions» — три колонки текста об уходе.

12.2 front/src/pages/bracelets/BraceletsPage.jsx — каталог браслетов

Устроена точно так же, как `BeadedBagsPage`, отличия:

- **стр. 7** — состояние называется `bracelets`;
- **стр. 17–19** — фильтр `product.category === "bracelets"`;
- **стр. 33–58** — другие тексты: «BRACELETS», «Our Collection»;
- **стр. 74–78** — у картинки нет обработчика `onError`;
- **стр. 139–171** — свои советы по уходу.

Все строки логики (стр. 13–27 — загрузка, стр. 68–69 — `isInCart` / `outOfStock`, стр. 118–129 — кнопка) делают то же самое, что в 12.1.

12.3 front/src/pages/product/ProductDetailPage.jsx — карточка товара

Кто подключает: `App.jsx` (адрес `/products/:id`). Сюда ведут ссылки «View Details» из каталогов. **Что подключает:** `useEffect`, `useState` (стр. 1); `useParams`, `useNavigate`, `Link` (стр. 2); `useCart` (стр. 3); иконку `ArrowLeft` (стр. 4); `api`, `imageUrl` (стр. 5). **Обращается к серверу:** `GET /api/products/:id` → `getProductById`.

```
7  const ProductDetailPage = () => {
8    const { id } = useParams();
9    const navigate = useNavigate();
10   const { items, addToCart } = useCart();
11   const [product, setProduct] = useState(null);
12   const [loading, setLoading] = useState(true);
13   const [notFound, setNotFound] = useState(false);
```

- **стр. 8** — `useParams()` возвращает параметры адреса. Для `/products/65f1a...` → `{ id: '65f1a...' }`. Имя `id` совпадает с `:id` в `App.jsx` (стр. 86).

- **стр. 9** — функция перехода (для кнопки «Назад»).
- **стр. 10** — корзина.
- **стр. 11–13** — товар (пока `null`), загрузка, «не найден».

```
17   useEffect(() => {
18     api
19       .get(`/api/products/${id}`)
20       .then((response) => {
21         setProduct(response.data.data);
22         setLoading(false);
23       })
24       .catch((error) => {
25         console.log(error);
26         setNotFound(true);
27         setLoading(false);
28       });
29   }, [id]);
```

- **стр. 19** — запрос одного товара по id.
- **стр. 21–22** — пришёл — сохраняем, загрузка закончена.
- **стр. 24–27** — ошибка (404 или неверный id) — «не найден».
- **стр. 29** — `[id]` — если перейти на другой товар, запрос повторится.

```
31   if (loading) {
32     return ( ...надпись "Loading..." по центру... );
37   }
38
39   if (notFound || !product) {
40     return ( ...«Product Not Found» и ссылка на главную... );
51   }
52
53   const isInCart = items.some((item) => item.id === product._id);
54   const outOfStock = product.stock === 0;
55   const backPath = product.category === "bracelets" ? "/bracelets" : "/beaded-bags";
56   const backLabel = product.category === "bracelets" ? "Bracelets" : "Beaded Bags";
```

- **стр. 31–37** — ранний возврат: пока грузится — только надпись «Loading...».
- **стр. 39–51** — не найден — экран «Product Not Found» со ссылкой на главную.
- **стр. 53** — уже в корзине?
- **стр. 54** — нет в наличии?
- **стр. 55–56** — куда ведёт кнопка «Назад» и её текст: в каталог браслетов или сумок, в зависимости от категории.

Разметка (стр. 58–165):

- **стр. 61–67** — кнопка «← Back to ...»: `onClick={() => navigate(backPath)}`.
- **стр. 71** — две колонки: фото и описание.
- **стр. 72–79** — фото; если не загрузилось — скрыть (`e.target.style.display = "none"`).
- **стр. 82–84** — категория.
- **стр. 86–88** — название.

- **стр. 90–101** — все стили товара метками (`product.style.map`), только если массив не пустой (`?.length > 0`).
- **стр. 103–114** — все детали абзацами.
- **стр. 116–121** — цена.
- **стр. 123–142** — остаток (только если число): цветная точка (красная / оранжевая / зелёная) и текст «Currently out of stock» / «Hurry — only N left» / «N in stock».
- **стр. 144–156** — кнопка добавления: та же логика, что в каталоге (стр. 145 — добавить, только если не в корзине и в наличии; стр. 146 — неактивна без остатка; стр. 155 — текст по состоянию).
- **стр. 158–160** — подпись «Handmade with care · Ships within 7–14 days».

Часть 13. Сайт: конструктор сумки

13.1 front/src/pages/custom-studio/CustomStudioPage.jsx

Кто подключает: `App.jsx` (адрес `/custom-studio`). Сюда ведут кнопки CUSTOM и CREATE на главной.
Что подключает:

- **стр. 1** — React, `useState`, `useEffect`;
- **стр. 2** — `motion`, `AnimatePresence` из `framer-motion`;
- **стр. 3** — `useCart` (нужна функция `addCustomItem`);
- **стр. 4** — `useNavigate` (переход в корзину после добавления);
- **стр. 8–11** — четыре фото размеров (`10x8.png` и т.д.);
- **стр. 15–25** — десять фото сумки с разными ремешками для размеров Mini и Small;
- **стр. 67–81** — 15 фото бусин разных цветов.

Каждый `import картинка from "./файл.png"` — Vite при сборке кладёт файл в готовый сайт и подставляет в переменную его адрес.

С сервером не общается. Сумка попадает в корзину через `addCustomItem` (`CartContext.jsx`), а цену при оплате сервер считает сам (`OrderController.js`, стр. 15–16 и 68–85).

Данные конструктора (стр. 27–136)

```
27 const SIZES = [
28   { id: "mini", label: "Mini", cm: "10×8 cm", price: 120, image: sizeMini },
29   { id: "small", label: "Small", cm: "14×11 cm", price: 180, image: sizeSmall },
30   { id: "medium", label: "Medium", cm: "20×16 cm", price: 260, image: sizeMedium },
31   { id: "large", label: "Large", cm: "28×22 cm", price: 360, image: sizeLarge },
32 ];
```

- **стр. 27–32** — четыре размера: код, название, размер в см, цена, фото. Цены совпадают с `SIZE_PRICES` на сервере.

```
34 const STRAPS = [
35   { id: "golden", label: "Golden", price: 30, images: { mini: strapMiniGolden, small:
strapSmallGolden } },
36   { id: "silver", label: "Silver", price: 30, images: { mini: strapMiniSilver, small:
strapSmallSilver } },
37   {
38     id: "mixed",
39     label: "Mixed",
40     price: 40,
41     // Mixed comes with a gold or a silver chain, so it has two sets of photos.
42     chainImages: {
43       gold: { mini: strapMiniMixedGold, small: strapSmallMixedGold },
44       silver: { mini: strapMiniMixedSilver, small: strapSmallMixedSilver },
45     },
46   },
47   { id: "beaded", label: "Beaded", price: 45, images: { mini: strapMiniBeaded, small:
strapSmallBeaded } },
48 ];
```

- **стр. 34–48** — четыре ремешка: код, название, доплата, фото для размеров mini и small. У Mixed вместо `images` — `chainImages`: отдельные фото для золотой и серебряной цепочки. Цены совпадают с `STRAP_PRICES` на сервере.

```
50 // Strap photos only exist for mini/small; other sizes fall back to the mini photo.
51 // Mixed strap has a chain color (gold/silver); we pick its photo from chainImages.
52 const strapImageFor = (strap, sizeId, chainColor = "gold") => {
53   const set = strap.chainImages ? strap.chainImages[chainColor] || strap.chainImages.gold :
strap.images;
54   // No photo for this strap, so show the mini photo instead of breaking the page.
55   if (!set) return strap.images?.mini ?? "";
56   return set[sizeId] || set.mini;
57 };
```

Функция `strapImageFor(strap, sizeId, chainColor)` — какое фото показать для ремешка. Вызывается на стр. 167, 190, 292.

- **стр. 52** — `chainColor = "gold"` — значение по умолчанию, если не передали.
- **стр. 53** — набор фото: у Mixed — по цвету цепочки (если такого нет — золотой), у остальных — `images`.
- **стр. 55** — набора нет — фото mini или пустая строка (`??` — если слева `null` / `undefined`).
- **стр. 56** — фото для выбранного размера; для Medium и Large фото нет — берём mini.

```
59 // Display label for a strap; Mixed appends its chain color (e.g. "Mixed (Gold)").
60 const strapLabelFor = (strap, chainColor) => {
61   if (strap.id !== "mixed") return strap.label;
62   return `${strap.label} (${chainColor === "gold" ? "Gold" : "Silver"})`;
63 };
```

Функция `strapLabelFor(strap, chainColor)` — название ремешка для показа. Вызывается на стр. 163 и 293.

- **стр. 61** — не Mixed — просто название.
- **стр. 62** — Mixed — «Mixed (Gold)» или «Mixed (Silver)».

```
83 const COLORS = [
84   { id: "aquamarine", name: "Aquamarine", hex: "#68c4e8", beads: beadsAquamarine },
...
98   { id: "black", name: "Black", hex: "#494c4a", beads: beadsBlack },
99 ];
```

- **стр. 83–99** — 15 цветов бусин: код, название, цвет-запаска (если фото не загрузится) и фото бусин.

```
101 const BagPreview = ({ image, alt }) => {
102   return (
103     <div className="relative w-full h-full">
104       <img src={image} alt={alt} className="absolute inset-0 w-full h-full object-contain drop-
shadow-2xl" />
...
110   );
111 };
```

- **Компонент `BagPreview`** (стр. 101–111) — большое фото сумки с тенью. Используется на стр. 446.

```
113 const StepHeader = ({ step, total, title }) => {
114   return (
115     <div className="mb-6">
116       <p ...>Step {step} of {total}</p>
119       <h3 ...>{title}</h3>
122     </div>
123   );
124 };
```

- **Компонент `StepHeader`** (стр. 113–124) — заголовок шага: «Step 1 of 3» и «Choose Size». Используется на стр. 264, 288, 352.

```
126 const STEPS = ["Size", "Strap", "Color"];
127 // Names for the steps, so the code says STEP.SIZE instead of 0.
128 const STEP = { SIZE: 0, STRAP: 1, COLOR: 2 };
129
130 const CHAIN_FINISHES = [
131   { id: "gold", label: "Gold" },
132   { id: "silver", label: "Silver" },
133 ];
134
135 // How long the "Added ✓" state shows before redirecting to the cart.
136 const REDIRECT_DELAY_MS = 1200;
137
138 // Shared look for selectable option buttons (size / strap / chain finish).
139 const optionClass = (selected) => {
140   return selected
141     ? "border-black bg-black text-white"
142     : "border-black/20 bg-white text-black hover:border-black/50";
143 };
```

- **стр. 126** — названия трёх шагов (для вкладок).
- **стр. 128** — номера шагов по именам.
- **стр. 130–133** — два цвета цепочки для ремешка Mixed.
- **стр. 136** — через 1,2 с после добавления — переход в корзину.
- **стр. 139–143** — **функция `optionClass(selected)`** — стиль кнопки варианта: выбранная — чёрная, остальные — белые.

Компонент `CustomStudioPage` (стр. 145–524)

```
145 const CustomStudioPage = () => {
146   const [step, setStep] = useState(0);
147   const [config, setConfig] = useState({
148     size: "small", // Small is chosen first because it is the most popular size
149     strap: "golden",
150     chainColor: "gold",
151     color: COLORS[0],
152   });
153   const [addedToCart, setAddedToCart] = useState(false);
154   const [lightbox, setLightbox] = useState(null);
155   const { addCustomItem } = useCart();
156   const navigate = useNavigate();
```

- **стр. 146** — текущий шаг (0 — размер).
- **стр. 147–152** — выбор покупателя: размер Small, ремешок Golden, цепочка золотая, цвет — первый из списка (целый объект цвета).
- **стр. 153** — добавлена ли сумка в корзину (для надписи и перехода).
- **стр. 154** — что показать в увеличенном просмотре (`null` — ничего).
- **стр. 155** — функция добавления сумки из корзины.
- **стр. 156** — функция перехода.

```
158   const sizeInfo = SIZES.find((s) => s.id === config.size);
159   const strapInfo = STRAPS.find((s) => s.id === config.strap);
160   const totalPrice = sizeInfo.price + strapInfo.price;
161
162   // "Mixed" shows its chain color in the display label; other straps just use their label.
163   const strapLabel = strapLabelFor(strapInfo, config.chainColor);
164
165   // Preview follows the active step: size photo on Size, strap photo otherwise.
166   const onSizeStep = step === STEP.SIZE;
167   const previewImage = onSizeStep ? sizeInfo.image : strapImageFor(strapInfo, config.size,
config.chainColor);
168   const previewLabel = onSizeStep ? `${sizeInfo.label} · ${sizeInfo.cm}` : `${sizeInfo.label} ·
${strapLabel}`;
169   const openPreviewLightbox = () => setLightbox({ image: previewImage, label: previewLabel });
```

Эти строки пересчитываются при каждой перерисовке (после любого выбора):

- **стр. 158** — полные данные выбранного размера (`find` по коду).
- **стр. 159** — полные данные выбранного ремешка.
- **стр. 160** — **цена = размер + ремешок**. Цвет на цену не влияет.
- **стр. 163** — название ремешка для показа.
- **стр. 166** — на шаге размера ли мы.
- **стр. 167** — большое фото: на шаге размера — фото размера, иначе — фото с ремешком.
- **стр. 168** — подпись к фото.
- **стр. 169** — **функция `openPreviewLightbox()`** — открыть фото крупно.

```
171   // Any change makes a new bag, so the Add to Cart button works again.
172   function updateConfig(patch) {
173     setConfig((c) => ({ ...c, ...patch }));
174     setAddedToCart(false);
175   }
```

Функция `updateConfig(patch)` — изменить выбор. Вызывается кнопками размеров (стр. 269), ремешков (стр. 299), цепочки (стр. 337), цветов (стр. 360).

- **стр. 173** — `setConfig` с функцией: получает прошлый выбор `c` и возвращает новый: все старые поля (`...c`) плюс изменённые (`...patch`). Например, `updateConfig({ size: 'large' })` меняет только размер.
- **стр. 174** — после любого изменения кнопку «Add to Cart» снова можно нажать.

```
177   // Drive the post-add redirect from an effect so the timer is cleared if the user leaves first.
178   useEffect(() => {
179     if (!addedToCart) return;
180     const timer = setTimeout(() => navigate("/cart"), REDIRECT_DELAY_MS);
181     return () => clearTimeout(timer);
182   }, [addedToCart, navigate]);
```

Эффект перехода в корзину.

- **стр. 179** — ещё не добавили — ничего не делаем.
- **стр. 180** — `setTimeout(функция, мс)` — выполнить через 1,2 с: перейти на `/cart`.
- **стр. 181** — очистка: если пользователь ушёл со страницы раньше, `clearTimeout` отменит переход.
- **стр. 182** — срабатывает, когда меняется `addedToCart`.

```
184   function handleAddToCart() {
185     const name = `Custom Beaded Bag — ${config.color.name} / ${strapLabel}`;
186     addCustomItem({
187       id: `custom-${Date.now()}`,
188       name,
189       price: totalPrice,
190       image: strapImageFor(strapInfo, config.size, config.chainColor), // real thumbnail, not ""
191       category: "custom",
192       quantity: 1,
193       config: {
194         // What the customer chose. The server counts the price from it.
195         size: config.size,
196         strap: config.strap,
197         chainColor: strapInfo.id === "mixed" ? config.chainColor : null,
198         color: config.color.id,
199       },
200     });
201     setAddedToCart(true);
202   }
```

Функция `handleAddToCart()` — положить сумку в корзину. Вызывается кнопкой на последнем шаге (стр. 416).

- **стр. 185** — название, например «Custom Beaded Bag — Black / Golden».
- **стр. 186** — `addCustomItem` из `CartContext.jsx` (стр. 169).
- **стр. 187** — id: `custom-` + текущее время в мс. По приставке `custom-` корзина и сервер понимают, что это сумка из конструктора.
- **стр. 189** — цена для показа в корзине. При оплате сервер её **не берёт**, а считает сам.
- **стр. 190** — картинка для корзины.
- **стр. 193–199** — `config` — выбор покупателя кодами. Именно по `size` и `strap` сервер считает цену; цвет цепочки — только для Mixed.
- **стр. 201** — отмечаем «добавлено» → эффект выше через 1,2 с откроет корзину.

```
204   const canGoNext = step < STEPS.length - 1;
205   const canGoPrev = step > 0;
```

- **стр. 204** — есть ли следующий шаг (шагов 3, последний номер 2).

- **стр. 205** — есть ли предыдущий.

Разметка (стр. 207–523):

- **стр. 209–225** — чёрная шапка: заголовок «Custom Studio» и подпись, оба плавно появляются.
- **стр. 227–250** — вкладки шагов: `STEPS.map((s, i) => ...)`. **стр. 232** `onClick={() => setStep(i)}` — перейти на шаг кликом. **стр. 236** — цвет текста: текущий — чёрный, пройденный — серый, будущий — светлый. **стр. 241–243** — под текущей вкладкой линия; `layoutId="stepIndicator"` — framer-motion плавно «переносит» её при смене шага. **стр. 244–246** — под пройденными — светлая линия.
- **стр. 252** — две колонки: выбор слева, превью справа.
- **стр. 254–396** — левая колонка. `AnimatePresence mode="wait"` + `key={step}` (стр. 256) — при смене шага старое содержимое уезжает вправо, новое выезжает слева.
  - **стр. 262–284 — шаг 1, размер** (показывается, только если `step === STEP.SIZE`): заголовок шага; `SIZES.map(...)` — по кнопке на размер. **стр. 269** — клик: `updateConfig({ size: sz.id })`. **стр. 270** — `aria-pressed` — для экранных читалок: нажата ли кнопка. **стр. 271** — стиль через `optionClass`. **стр. 273–279** — название, размер в см, цена.
  - **стр. 286–348 — шаг 2, ремешок**: `STRAPS.map(...)`. **стр. 291–293** — для каждого: выбран ли, его фото для текущего размера, подпись. **стр. 298–314** — кнопка выбора: клик — `updateConfig({ strap: st.id })`; внутри фото, название, доплата «+$30». **стр. 315–322** — маленькая кнопка-лупа поверх фото (видна при наведении): клик открывает фото крупно (`setLightbox`). **стр. 328–346** — только для Mixed: выбор цвета цепочки, `CHAIN_FINISHES.map(...)`, клик — `updateConfig({ chainColor })`.
  - **стр. 350–394 — шаг 3, цвет**: `COLORS.map(...)` — сетка из 15 круглых кнопок. **стр. 360** — клик: `updateConfig({ color: c })` (весь объект цвета). **стр. 362–365** — фон — цвет-запаска; у выбранного толстая рамка. **стр. 367** — фото бусин; `loading="lazy"` — грузить, только когда видно. **стр. 368–370** — на выбранном — галочка. **стр. 374** — «Selected: название». **стр. 376–392** — итог заказа: размер, ремешок, цвет (массив пар `[название, значение]` → `.map` в строки) и итоговая цена.
  - **стр. 398–424 — кнопки навигации**: **стр. 399–406** — «← Back», если есть предыдущий шаг: `setStep((s) => s - 1)`. **стр. 407–413** — «Next →», если есть следующий: `setStep((s) => s + 1)`. **стр. 414–423** — иначе (последний шаг) — кнопка «Add to Cart — $цена»: `onClick={handleAddToCart}`, неактивна после добавления, текст меняется на «Added ✓ Redirecting…».
- **стр. 427–476** — правая колонка (на компьютере «прилипает» при прокрутке, `lg:sticky`):
  - **стр. 429–447** — большое превью. `key={previewImage}` — при смене фото анимация появления. **стр. 437** — клик открывает фото крупно. **стр. 438–443** — то же с клавиатуры: Enter или пробел (`e.preventDefault()` — чтобы пробел не прокручивал страницу). **стр. 446** — компонент `BagPreview`.
  - **стр. 450–463** — под превью: на шаге цвета — кружок с фото бусин и «цвет · ремешок»; на других шагах — название размера или ремешка.
  - **стр. 465–475** — «Estimated price» и цена крупно; справа — размер, ремешок и «Made to order · 2–3 weeks» (постоянный текст).
- **стр. 479–483** — блок с пояснением, что сумка делается на заказ.
- **стр. 485–521 — увеличенный просмотр (лайтбокс)**: показывается, только если `lightbox` не `null`. **стр. 487–492** — тёмный фон на весь экран; клик по нему — `setLightbox(null)` (закрыть).

стр. 494–501 — кнопка ✕ (тоже закрывает). **стр. 502–518** — фото и подпись; **стр. 507** `onClick={(e) => e.stopPropagation()}` — клик по самой картинке не «всплывает» до фона и не закрывает просмотр.

Часть 14. Сайт: корзина, оформление, «Спасибо»

14.1 front/src/pages/cart/CartPage.jsx — корзина

Кто подключает: `App.jsx` (адрес `/cart`). Сюда ведут значок корзины в шапке и конструктор после добавления. **Что подключает:** React (стр. 1); `Link`, `useNavigate` (стр. 2); `useCart` (стр. 3); иконки `Trash2`, `Plus`, `Minus`, `ShoppingBag` (стр. 4). **С сервером напрямую не общается** — всё через функции `CartContext`.

```
6  const CartPage = () => {
7    const { items, removeFromCart, updateQuantity, getTotalPrice, clearCart } = useCart();
8    const navigate = useNavigate();
```

- **стр. 7** — из корзины: позиции и четыре функции.
- **стр. 8** — переход на оформление.
- **стр. 10–30** — **если корзина пуста** (`items.length === 0`) — отдельный экран: иконка сумки, «Your Cart is Empty», ссылка «Continue Shopping» на главную. `return` — дальше не рисуем.
- **стр. 32–41** — заголовок «Shopping Cart» и «N items in your cart» (`item` или `items` в зависимости от количества).
- **стр. 42** — две колонки: позиции (2/3 ширины) и итог (1/3).
- **стр. 45–115** — `items.map((item) => ...)` — карточка на каждую позицию:
  - **стр. 51–60** — фото `item.image`; не загрузилось — скрыть;
  - **стр. 65–73** — категория, название, цена за штуку (`toFixed(2)` — два знака после точки);
  - **стр. 79–85** — кнопка «−»: `updateQuantity(item.id, item.quantity - 1)` (меньше 1 функция не даст);
  - **стр. 86–88** — текущее количество;
  - **стр. 89–95** — кнопка «+»: `updateQuantity(item.id, item.quantity + 1)`;
  - **стр. 98–104** — мусорка: `removeFromCart(item.id)`;
  - **стр. 109–113** — сумма по позиции: цена × количество.
- **стр. 118–123** — «Clear all items»: `onClick={clearCart}` — очистить всё.
- **стр. 127–184** — блок итога:
  - **стр. 133–148** — Subtotal и Total — `getTotalPrice()`, доставка «Free»;
  - **стр. 150–155** — кнопка Checkout: `navigate("/checkout")`;
  - **стр. 157–162** — ссылка «Continue Shopping»;
  - **стр. 164–182** — значки платёжных систем (просто текст в рамках, для вида).

14.2 front/src/pages/checkout/CheckoutPage.jsx — оформление заказа

Кто подключает: `App.jsx` (адрес `/checkout`). Сюда ведёт кнопка Checkout в корзине. **Что подключает:** `useState` (стр. 1); `Link` (стр. 2); `useCart` (стр. 3); `api`, `authHeader` (стр. 4). **Обращается к серверу:** `POST /api/orders/checkout-session` → `createCheckoutSession`.

```
6  const CheckoutPage = () => {
7    const { items, getTotalPrice } = useCart();
8    const [form, setForm] = useState({
9      name: "",
10     email: "",
11     phone: "",
12     address: "",
13     city: "",
14     country: "",
15     notes: "",
16   });
17   const [deliveryMethod, setDeliveryMethod] = useState("delivery");
18   const [errors, setErrors] = useState({});
19   const [loading, setLoading] = useState(false);
20   const [message, setMessage] = useState("");
```

- **стр. 7** — позиции корзины и сумма.
- **стр. 8–16** — все поля формы в одном объекте, сначала пустые.
- **стр. 17** — способ получения: доставка (по умолчанию) или самовывоз.
- **стр. 18** — ошибки по полям: `{ email: "Valid email required" }`.
- **стр. 19** — идёт ли отправка (чтобы заблокировать кнопку).
- **стр. 20** — общее сообщение об ошибке от сервера.

```
22   const validate = () => {
23     const e = {};
24     if (!form.name.trim()) e.name = "Required";
25     if (!form.email.trim() || !/\S+@\S+\.\S+/.test(form.email)) e.email = "Valid email required";
26     if (deliveryMethod === "delivery") {
27       if (!form.address.trim()) e.address = "Required";
28       if (!form.city.trim()) e.city = "Required";
29       if (!form.country.trim()) e.country = "Required";
30     }
31     return e;
32   };
```

Функция `validate()` — проверка формы в браузере (для удобства; сервер проверяет сам). Вызывается из `handleSubmit` (стр. 42).

- **стр. 23** — пустой объект ошибок.
- **стр. 24** — имя пустое — ошибка.
- **стр. 25** — почта пустая или не похожа на почту (`\S+@\S+\.\S+` — «не пробелы @ не пробелы . не пробелы»).
- **стр. 26–30** — адрес, город и страна обязательны только при доставке.
- **стр. 31** — возвращаем найденные ошибки.

```
34   const handleChange = (e) => {
35     setForm({ ...form, [e.target.name]: e.target.value });
36     if (errors[e.target.name]) setErrors({ ...errors, [e.target.name]: undefined });
37   };
```

Функция `handleChange(e)` — одна на все поля ввода. Вызывается при каждом нажатии клавиши в поле (`onChange`).

- **стр. 35** — `e.target` — поле, в котором печатают; `name` — его атрибут `name` (например `"city"`), `value` — текст. `[e.target.name]` — имя ключа из переменной. Получается: копия формы, в которой поле `city` заменено.
- **стр. 36** — если у этого поля была ошибка — убрать её (пользователь начал исправлять).

```
39  const handleSubmit = (e) => {
40    e.preventDefault();
41    setMessage("");
42    const errs = validate();
43    if (Object.keys(errs).length > 0) {
44      setErrors(errs);
45      return;
46    }
47
48    setLoading(true);
49
50    // A guest may also order, so the token is sent only when there is one.
51    const token = localStorage.getItem("token");
52    const config = token ? authHeader(token) : {};
53
54    const body = {
55      items,
56      customer: { name: form.name, email: form.email, phone: form.phone },
57      shipping: { address: form.address, city: form.city, country: form.country },
58      deliveryMethod,
59      notes: form.notes,
60    };
61
62    api
63      .post("/api/orders/checkout-session", body, config)
64      .then((response) => {
65        // The server opened a payment session and sent back its address.
66        window.location.href = response.data.url;
67      })
68      .catch((error) => {
69        console.log(error);
70        setMessage(error.response?.data?.message || "Could not start checkout.");
71        setLoading(false);
72      });
73  };
```

Функция `handleSubmit(e)` — кнопка Proceed to Payment. Вызывается формой (`onSubmit`, стр. 107).

- **стр. 40** — `e.preventDefault()` — не дать браузеру перезагрузить страницу (стандартное поведение формы).
- **стр. 41** — стереть прошлое сообщение.
- **стр. 42–46** — проверяем; `Object.keys(errs)` — список имён полей с ошибками; если не пусто — показываем ошибки и выходим.
- **стр. 48** — «идёт отправка» — кнопка станет неактивной.
- **стр. 51–52** — если вошли — добавляем заголовок с токеном (сервер запишет id в заказ), если нет — пустые настройки (гость).
- **стр. 54–60** — тело запроса: позиции корзины (как есть, с ценами — сервер их пересчитает), контакты, адрес, способ, комментарий.
- **стр. 62–63** — `POST /api/orders/checkout-session` → на сервере `createCheckoutSession`.

- **стр. 66** — сервер вернул адрес страницы оплаты Stripe. `window.location.href = ...` — **полный переход** браузера на другой сайт (Stripe). `navigate` здесь не подходит — он работает только внутри нашего сайта.
- **стр. 68–72** — ошибка (нет в наличии, оплата не настроена) — показываем сообщение сервера и разблокируем кнопку.
- **стр. 75–89** — **корзина пуста** — экран «Your cart is empty» со ссылкой на главную.
- **стр. 91–94** — **функция** `inputClass(field)` — стиль поля: с ошибкой — красная линия, без — серая.

Разметка (стр. 96–258):

- **стр. 99–104** — заголовок «Checkout».
- **стр. 107** — `<form onSubmit={handleSubmit} noValidate>` — `noValidate` отключает встроенные сообщения браузера, используем свои.
- **стр. 108–125** — «Contact»: поля `name`, `email`, `phone`. У каждого `name="..."` (по нему работает `handleChange`), `value={form....}`, `onChange={handleChange}`. Под полем — ошибка, если есть (`{errors.name && <p>...}`).
- **стр. 127–151** — «Delivery Method»: две кнопки из массива `[{ id: "delivery" }, { id: "pickup" }]`. **стр. 139** — клик `setDeliveryMethod(opt.id)`. Выбранная — чёрная.
- **стр. 153–173** — «Shipping Address» — только при доставке (`deliveryMethod === "delivery" &&`): поля `address`, `city`, `country` с ошибками.
- **стр. 175–187** — «Notes» — многострочное поле `textarea`.
- **стр. 189–191** — сообщение сервера красным.
- **стр. 193–199** — кнопка отправки: неактивна во время отправки; текст «Redirecting to payment...» или «Proceed to Payment».
- **стр. 202–254** — справа «Your Order»: `items.map(...)` — фото, название, количество, сумма по позиции; ниже Subtotal, Shipping «Free», Total — `getTotalPrice()`; ссылка «← Edit cart» на `/cart`.

14.3 front/src/pages/order/OrderSuccessPage.jsx — «Спасибо за заказ»

Кто подключает: `App.jsx` (адрес `/order-success`). Сюда **Stripe** возвращает покупателя после оплаты: `/order-success?session_id=cs_...` (адрес задан в `OrderController.js`, стр. 159). **Что подключает:** `useEffect`, `useState` (стр. 1); `Link`, `useSearchParams` (стр. 2); иконку `CheckCircle` (стр. 3); `useCart` (стр. 4); `api` (стр. 5). **Обращается к серверу:** `GET /api/orders/by-session/:sessionId` → `getOrderBySession`; через `clearCart` — `DELETE /api/cart`.

```
7  const OrderSuccessPage = () => {
8    const [params] = useSearchParams();
9    const sessionId = params.get("session_id");
10   const { clearCart } = useCart();
11   const [order, setOrder] = useState(null);
12
13   // The payment went through, so empty the cart and show the order number.
14   useEffect(() => {
15     clearCart();
16     if (!sessionId) return;
17     api
18       .get(`/api/orders/by-session/${sessionId}`)
19       .then((response) => {
20         setOrder(response.data.data);
21       })
22       .catch((error) => {
23         console.log(error);
24       });
25     // This runs once when the page opens, so clearCart is left out on purpose.
26     // eslint-disable-next-line react-hooks/exhaustive-deps
27   }, [sessionId]);
```

- **стр. 8** — `useSearchParams()` — параметры после `?` в адресе.
- **стр. 9** — `params.get("session_id")` — id сессии оплаты Stripe.
- **стр. 10** — функция очистки корзины.
- **стр. 11** — краткие данные заказа (пока `null`).
- **стр. 15** — оплата прошла — очищаем корзину (и на сервере, если вошли).
- **стр. 16** — нет id в адресе — дальше ничего не делаем.
- **стр. 17–21** — спрашиваем у сервера номер заказа по id сессии и сохраняем.
- **стр. 25–26** — комментарий и указание линтеру не ругаться, что `clearCart` не указан в зависимостях: эффект должен сработать один раз.
- **стр. 27** — зависимость — `sessionId`.

Разметка (стр. 29–55):

- **стр. 31** — зелёная иконка-галочка.
- **стр. 32–34** — «Thank You».
- **стр. 36–46** — если `order` пришёл — «order DH-... is confirmed, receipt sent to почта»; иначе — общий текст без номера.
- **стр. 48–53** — кнопка «Back to Home».

Часть 15. Сайт: вход, регистрация, пароль

15.1 front/src/pages/login/LoginPage.jsx — вход и регистрация

Кто подключает: `App.jsx` (адрес `/login`), с пропсами `setToken` и `setUser`. Сюда ведут ссылка Login в шапке и переход с закрытых страниц `/admin`. **Что подключает:** `useState` (стр. 1); `Link`, `useNavigate` (стр. 2); `api` (стр. 3). **Обращается к серверу:** `POST /api/auth/signin` → `validateSignin` → `signin`; `POST /api/auth/signup` → `validateSignup` → `signup`.

```
5  const LoginPage = ({ setToken, setUser }) => {
6
7    const [isLogin, setIsLogin] = useState(true);
8    const [email, setEmail] = useState("");
9    const [password, setPassword] = useState("");
10   const [name, setName] = useState("");
11   const [showPassword, setShowPassword] = useState(false);
12   const [message, setMessage] = useState("");
13   const navigate = useNavigate();
```

- **стр. 5** — пропсы из `App.jsx`: функции, которые обновят токен и пользователя во всём приложении.
- **стр. 7** — режим: `true` — вход, `false` — регистрация.
- **стр. 8–10** — значения полей почты, пароля и имени.
- **стр. 11** — показывать ли пароль открытым текстом.
- **стр. 12** — сообщение об ошибке.
- **стр. 13** — функция перехода.

```
15   const applyAuth = (data) => {
16     localStorage.setItem("token", data.token);
17     localStorage.setItem("user", JSON.stringify(data.user));
18     setToken(data.token);
19     setUser?.(data.user);
20     navigate(data.user?.role === "manager" || data.user?.role === "admin" ? "/admin" : "/");
21   };
```

Функция `applyAuth(data)` — сохранить вход. Вызывается из `handleSubmit` (стр. 35), когда сервер ответил успешно. `data` — ответ сервера `{ token, user }`.

- **стр. 16** — токен в `localStorage` (переживёт перезагрузку страницы).
- **стр. 17** — пользователь строкой.
- **стр. 18** — обновить токен в `App.jsx` → `App` спросит `/api/auth/me`, `CartContext` перенесёт гостевую корзину на сервер.
- **стр. 19** — обновить пользователя (шапка сразу покажет MANAGE менеджеру).
- **стр. 20** — менеджера — на `/admin`, остальных — на главную.

```
23  const handleSubmit = (e) => {
24    // Without preventDefault the browser would reload the page on submit.
25    e.preventDefault();
26    setMessage("");
27
28    // The same form is used for signing in and for registering.
29    const url = isLogin ? "/api/auth/signin" : "/api/auth/signup";
30    const body = isLogin ? { email, password } : { name, email, password };
31
32    api
33      .post(url, body)
34      .then((response) => {
35        applyAuth(response.data);
36      })
37      .catch((error) => {
38        console.log(error);
39        // The server explains what went wrong, for example "Invalid credentials".
40        setMessage(error.response?.data?.message || "Server error");
41      });
42  };
```

Функция `handleSubmit(e)` — отправка формы (`onSubmit`, стр. 137).

- **стр. 25** — не перезагружать страницу.
- **стр. 26** — стереть старую ошибку.
- **стр. 29** — адрес: вход или регистрация.
- **стр. 30** — тело: для входа — почта и пароль; для регистрации ещё имя.
- **стр. 32–33** — запрос на сервер. Токен не нужен.
- **стр. 35** — успех — `applyAuth`.
- **стр. 40** — ошибка — показываем текст сервера («Invalid credentials», «Email already exists», «Password must be between 8 and 128 characters»).

```
44  const toggleLoginMode = (mode) => {
45    setIsLogin(mode);
46    setMessage("");
47  };
```

Функция `toggleLoginMode(mode)` — переключить вход/регистрацию и стереть ошибку. Вызывается вкладками (стр. 99, 109) и ссылкой внизу (стр. 245).

Разметка (стр. 49–254):

- **стр. 50** — экран в две половины.
- **стр. 51–76** — левая тёмная половина (только на больших экранах, `hidden lg:flex`): рамки, «welcome to», крупное «DARIN'S», фраза.
- **стр. 78–252** — правая половина с формой:
  - **стр. 80–94** — на телефоне вместо левой половины — логотип-ссылка на главную;
  - **стр. 96–117** — две вкладки «Sign In» и «Create Account»: клик — `toggleLoginMode(true/false)`; активная подчёркнута;
  - **стр. 119–135** — заголовок «SIGN IN» или «REGISTER» и подпись — по `isLogin`;
  - **стр. 137** — `<form onSubmit={handleSubmit}>`;

- **стр. 138–152** — поле имени — **только при регистрации** (`{!isLogin && (...)}`); `onChange={(e) => setName(e.target.value)}` — при каждом вводе сохраняем текст;
- **стр. 154–166** — поле почты, `type="email"`, `required` — браузер не даст отправить пустым;
- **стр. 168–211** — поле пароля: **стр. 174** `type={showPassword ? "text" : "password"}` — показывать точки или текст; **стр. 181–209** — кнопка-глаз: `onClick={() => setShowPassword(!showPassword)}` — переключить; внутри SVG-иконка глаза (открытый или зачёркнутый);
- **стр. 213–222** — ссылка «Forgot password?» на `/forgot-password` — только в режиме входа;
- **стр. 224–228** — сообщение об ошибке, если есть;
- **стр. 230–237** — кнопка отправки: «Sign In» или «Create Account»;
- **стр. 241–250** — «Don't have an account? Register» / «Already have an account? Sign In» — переключает режим.

15.2 front/src/pages/login/ForgotPasswordPage.jsx — «Забыли пароль?»

Кто подключает: `App.jsx` (адрес `/forgot-password`). Сюда ведёт ссылка на странице входа. **Что подключает:** `useState` (стр. 1); `Link` (стр. 2); `api` (стр. 3). **Обращается к серверу:** `POST /api/auth/forgot-password` → `forgotPassword`.

```
5  const ForgotPasswordPage = () => {
6    const [email, setEmail] = useState("");
7    const [sent, setSent] = useState(false);
8    const [message, setMessage] = useState("");
9    const [loading, setLoading] = useState(false);
10
11   const handleSubmit = (e) => {
12     e.preventDefault();
13     setMessage("");
14     setLoading(true);
15
16     api
17       .post("/api/auth/forgot-password", { email })
18       .then((response) => {
19         setSent(true);
20         setMessage(response.data.message || "If an account exists, a reset link has been sent.");
21         setLoading(false);
22       })
23       .catch((error) => {
24         console.log(error);
25         setMessage(error.response?.data?.message || "Server error");
26         setLoading(false);
27       });
28   };
```

- **стр. 6** — введённая почта.
- **стр. 7** — отправлен ли запрос (тогда вместо формы — сообщение).
- **стр. 8** — текст сообщения.
- **стр. 9** — идёт ли отправка.
- **стр. 11–28** — **функция** `handleSubmit(e)`: не перезагружать страницу (стр. 12), очистить сообщение, включить «загрузку», отправить почту на сервер (стр. 16–17). Успех — `sent = true` и

сообщение сервера (оно всегда одинаковое — «если аккаунт существует, ссылка отправлена»). Ошибка — текст ошибки (например, лимит 5 запросов в час).

Разметка (стр. 30–94):

- **стр. 33–41** — заголовок «RESET» и подсказка.
- **стр. 43–54** — если `sent` — сообщение и кнопка «Back to Sign In».
- **стр. 55–90** — иначе форма: поле почты (стр. 61–68), ошибка (стр. 71–75), кнопка «Send Reset Link» / «Sending...» (стр. 77–83), ссылка назад (стр. 85–89).

15.3 front/src/pages/login/ResetPasswordPage.jsx — новый пароль по ссылке из письма

Кто подключает: `App.jsx` (адрес `/reset-password`). Сюда ведёт ссылка из письма: `/reset-password?token=...` (её собирает `forgotPassword` на сервере, стр. 111). **Что подключает:** `useState` (стр. 1); `Link`, `useSearchParams`, `useNavigate` (стр. 2); `api` (стр. 3). **Обращается к серверу:** `POST /api/auth/reset-password` → `resetPassword`.

```
5  const ResetPasswordPage = () => {
6    const [params] = useSearchParams();
7    const token = params.get("token");
8    const navigate = useNavigate();
9
10   const [password, setPassword] = useState("");
11   const [confirm, setConfirm] = useState("");
12   const [message, setMessage] = useState("");
13   const [done, setDone] = useState(false);
14   const [loading, setLoading] = useState(false);
```

- **стр. 6–7** — токен из адреса после `?token=`.
- **стр. 10–11** — новый пароль и повтор.
- **стр. 12–14** — сообщение, готово ли, идёт ли отправка.

```
16   const handleSubmit = async (e) => {
17     e.preventDefault();
18     setMessage("");
19
20     if (password.length < 8) {
21       setMessage("Password must be at least 8 characters.");
22       return;
23     }
24     if (password !== confirm) {
25       setMessage("Passwords do not match.");
26       return;
27     }
28
29     setLoading(true);
30
31     api
32       .post("/api/auth/reset-password", { token, password })
33       .then((response) => {
34         setDone(true);
35         setMessage(response.data.message || "Password updated.");
36         setLoading(false);
37         // Give the visitor a moment to read the message, then go to the login page.
38         setTimeout(() => navigate("/login"), 1800);
39       })
40       .catch((error) => {
41         console.log(error);
42         setMessage(error.response?.data?.message || "Reset failed.");
43         setLoading(false);
44       });
45   };
```

Функция `handleSubmit(e)` — отправка (стр. 82).

- **стр. 20–23** — пароль короче 8 — сообщение, на сервер не отправляем.
- **стр. 24–27** — пароли не совпадают — сообщение.
- **стр. 31–32** — отправляем токен из ссылки и новый пароль.
- **стр. 34–38** — успех: «готово», сообщение сервера, через 1,8 с — переход на вход.
- **стр. 40–44** — ошибка (ссылка просрочена или уже использована) — сообщение сервера.

```
47   if (!token) {
48     return ( ...«Invalid link», пояснение, ссылка «Request a new link» на /forgot-password... );
62   }
```

- **стр. 47–62** — в адресе нет токена — экран «Invalid link».

Разметка (стр. 64–128): заголовок «NEW PASSWORD» (стр. 67–75); если `done` — зелёное сообщение «… Redirecting to sign in…» (стр. 77–80); иначе форма (стр. 82–124): поле нового пароля (стр. 83–95), поле повтора (стр. 97–109), ошибка (стр. 111–115), кнопка «Update Password» / «Updating…» (стр. 117–123).

Часть 16. Сайт: страницы менеджера

Обе страницы открываются, только если `isManager` в `App.jsx` (стр. 89–96). Обе получают `token` пропсом и отправляют его в каждом запросе через `authHeader(token)`. На сервере каждый их запрос проходит `protect` + `restrictTo('manager', 'admin')`.

16.1 front/src/pages/admin/ManagerProductsPage.jsx — товары (`/admin`)

Кто подключает: `App.jsx` (стр. 20, 91). Сюда ведут ссылка MANAGE в шапке, переход после входа менеджера и ссылка «← Products» со страницы заказов. **Что подключает:** `useEffect`, `useState` (стр. 1); `Link` (стр. 2); `api`, `imageUrl`, `authHeader` (стр. 3). **Обращается к серверу:** `GET /api/products` (`getAllProducts`), `POST /api/products` (`createProduct`), `PUT /api/products/:id` (`updateProduct`), `DELETE /api/products/:id` (`deleteProduct`).

```
5  const EMPTY = {
6    model_name: "",
7    category: "bags",
8    price: "",
9    color_palette: "",
10   details: "",
11   style: "",
12   stock: "",
13 };
```

- **стр. 5–13** — пустая форма. Списки (цвета, детали, стиль) хранятся одной строкой через запятую — сервер сам разрежет их (`toArray`).

```
15 const ManagerProductsPage = ({ token }) => {
16   const [products, setProducts] = useState([]);
17   const [loading, setLoading] = useState(true);
18   const [form, setForm] = useState(EMPTY);
19   const [imageFile, setImageFile] = useState(null);
20   const [editingId, setEditingId] = useState(null);
21   const [filter, setFilter] = useState("all");
22   const [saving, setSaving] = useState(false);
23   const [message, setMessage] = useState(null);
```

- **стр. 16** — все товары.
- **стр. 17** — загрузка.
- **стр. 18** — значения формы.
- **стр. 19** — выбранный файл картинки.
- **стр. 20** — id товара, который редактируем (`null` — создаём новый).
- **стр. 21** — фильтр списка: all / bags / bracelets.
- **стр. 22** — идёт ли сохранение.
- **стр. 23** — сообщение `{ type: "success" | "error", text }`.

```
25  // Brings the whole catalog into the table.
26  const loadProducts = () => {
27    api
28      .get("/api/products")
29      .then((response) => {
30        setProducts(response.data.data);
31        setLoading(false);
32      })
33      .catch((error) => {
34        console.log(error);
35        setMessage({ type: "error", text: "Failed to load products" });
36        setLoading(false);
37      });
38  };
39
40  useEffect(() => {
41    loadProducts();
42  }, []);
```

- **стр. 26–38** — **функция** `loadProducts()` — загрузить все товары. Вызывается при открытии (стр. 41), после сохранения (стр. 96) и после удаления (стр. 115).
- **стр. 40–42** — один раз при открытии страницы.

```
44  const resetForm = () => {
45    setForm(EMPTY);
46    setImageFile(null);
47    setEditingId(null);
48  };
```

- **Функция** `resetForm()` — очистить форму и выйти из режима редактирования. Вызывается кнопкой Cancel (стр. 156), после сохранения (стр. 95), после удаления редактируемого товара (стр. 113).

```
50  const startEdit = (p) => {
51    setEditingId(p._id);
52    setForm({
53      model_name: p.model_name || "",
54      category: p.category || "bags",
55      price: p.price ?? "",
56      color_palette: (p.color_palette || []).join(", "),
57      details: (p.details || []).join(", "),
58      style: (p.style || []).join(", "),
59      stock: p.stock ?? "",
60    });
61    setImageFile(null);
62    window.scrollTo({ top: 0, behavior: "smooth" });
63  };
```

Функция `startEdit(p)` — заполнить форму данными товара. Вызывается кнопкой Edit у товара (стр. 319).

- **стр. 51** — запоминаем, какой товар редактируем.
- **стр. 53–59** — переносим поля товара в форму. Массивы склеиваем в строку через запятую (`join(", ")`). `??` у цены и остатка — заменяет только `null` / `undefined`, а `0` оставляет (с `||` ноль превратился бы в пустоту).

- **стр. 61** — файл не выбран (старая картинка останется, если не выбрать новую).
- **стр. 62** — плавно прокрутить наверх, к форме.

```
65   const handleSubmit = async (e) => {
66     e.preventDefault();
67     setMessage(null);
68
69     if (!editingId && !imageFile) {
70       setMessage({ type: "error", text: "Upload a product image" });
71       return;
72     }
73
74     const fd = new FormData();
75     fd.append("model_name", form.model_name);
76     fd.append("category", form.category);
77     fd.append("price", form.price);
78     fd.append("color_palette", form.color_palette);
79     fd.append("details", form.details);
80     fd.append("style", form.style);
81     fd.append("stock", form.stock);
82     if (imageFile) fd.append("image", imageFile);
83
84     setSaving(true);
85
86     // editingId is set when the manager pressed Edit on a row, so the same
87     // form both creates a new product and updates an existing one.
88     const request = editingId
89       ? api.put(`/api/products/${editingId}`, fd, authHeader(token))
90       : api.post("/api/products", fd, authHeader(token));
91
92     request
93       .then(() => {
94         setMessage({ type: "success", text: editingId ? "Product updated" : "Product added" });
95         resetForm();
96         loadProducts();
97         setSaving(false);
98       })
99       .catch((error) => {
100        console.log(error);
101        setMessage({ type: "error", text: error.response?.data?.message || "Save failed" });
102        setSaving(false);
103      });
104  };
```

Функция `handleSubmit(e)` — сохранить товар (`onSubmit` формы, стр. 148).

- **стр. 66** — не перезагружать страницу.
- **стр. 69–72** — новый товар без картинки нельзя (при редактировании картинку можно не менять).
- **стр. 74** — `new FormData()` — встроенный объект браузера для отправки формы **с файлом**. Обычный JSON файлы передавать не умеет.
- **стр. 75–81** — добавляем текстовые поля. Имена совпадают с тем, что читает сервер в `req.body`.
- **стр. 82** — файл картинки под именем `image` — именно его ждёт `upload.single('image')` на сервере.
- **стр. 84** — «сохраняется» — кнопка неактивна.

- **стр. 88–90** — редактируем — `PUT /api/products/:id` (`updateProduct`); создаём — `POST /api/products` (`createProduct`). Получив `FormData`, axios сам ставит тип `multipart/form-data`.
- **стр. 93–98** — успех: сообщение, очистить форму, перезагрузить список.
- **стр. 99–103** — ошибка: сообщение сервера (например, «Image is too large (max 8MB)» из обработчика ошибок в `index.js`).

```
106  const handleDelete = (product) => {
107    if (!window.confirm(`Delete "${product.model_name}"?`)) return;
108    setMessage(null);
109
110    api
111      .delete(`/api/products/${product._id}`, authHeader(token))
112      .then(() => {
113        if (editingId === product._id) resetForm();
114        setMessage({ type: "success", text: "Product deleted" });
115        loadProducts();
116      })
117      .catch((error) => {
118        console.log(error);
119        setMessage({ type: "error", text: error.response?.data?.message || "Delete failed" });
120      });
121  };
```

Функция `handleDelete(product)` — удалить товар. Вызывается кнопкой Delete (стр. 325).

- **стр. 107** — `window.confirm` — стандартное окно браузера «OK / Отмена». Отмена — выходим.
- **стр. 111** — `DELETE /api/products/:id` → `deleteProduct`.
- **стр. 113** — если удалили тот, что редактировали, — очистить форму.
- **стр. 115** — перезагрузить список.

```
123  const shown = products.filter((p) => filter === "all" || p.category === filter);
124
125  const inputClass = "...";
127  const labelClass = "...";
```

- **стр. 123** — какие товары показать: все или одной категории.
- **стр. 125–128** — общие стили полей и подписей.

Разметка (стр. 130–339):

- **стр. 132–145** — чёрная шапка «Manager Studio» и ссылка «View Orders →» на `/admin/orders`.
- **стр. 147** — две колонки: форма и список.
- **стр. 148–272 — форма**:
  - **стр. 149–162** — заголовок «Edit product» или «New product»; при редактировании — кнопка Cancel (`resetForm`);
  - **стр. 164–173** — название. Шаблон всех полей: `value={form.поле}` и `onChange={(e) => setForm((f) => ({ ...f, поле: e.target.value }))}` — при вводе копируем форму и меняем одно поле;
  - **стр. 175–199** — категория (выпадающий список `select` с двумя вариантами) и цена (`type="number"`);

    - **стр. 201–211** — остаток; пусто — без ограничения;
    - **стр. 213–241** — цвета, детали, стиль — через запятую;
    - **стр. 243–253** — выбор файла: `type="file"`, `accept="image/*"` — только картинки; `onChange={(e) => setImageFile(e.target.files?.[0] || null)}` — первый выбранный файл;
    - **стр. 255–263** — сообщение зелёным или красным по `message.type`;
    - **стр. 265–271** — кнопка: «Saving…» / «Save changes» / «Add product».
- **стр. 274–336 — список товаров**:
    - **стр. 276–286** — кнопки фильтра all / bags / bracelets → `setFilter(c)`;
    - **стр. 287–289** — «N items»;
    - **стр. 292–295** — загрузка или «No products yet.»;
    - **стр. 298–333** — `shown.map((p) => ...)` — карточка: фото (`imageUrl`), название, категория и цена, остаток («Unlimited stock» если `null`, красное «Out of stock» если 0, иначе «N in stock»), кнопки Edit (`startEdit(p)`) и Delete (`handleDelete(p)`).

16.2 front/src/pages/admin/ManagerOrdersPage.jsx — заказы (`/admin/orders`)

Кто подключает: `App.jsx` (стр. 21, 95). Сюда ведёт ссылка «View Orders →». **Что подключает:** `useEffect`, `useState` (стр. 1); `Link` (стр. 2); `api`, `authHeader` (стр. 3). **Обращается к серверу:** `GET /api/orders` (`getAllOrders`), `GET /api/orders/stats` (`getStats`), `PATCH /api/orders/:id/status` (`updateOrderStatus`).

```
5  const STATUSES = ["all", "pending", "paid", "in_production", "shipped", "completed", "cancelled"];
6  const FLOW = ["pending", "paid", "in_production", "shipped", "completed", "cancelled"];
7
8  const STATUS_STYLE = {
9    pending: "bg-yellow-100 text-yellow-800",
10   paid: "bg-blue-100 text-blue-800",
11   in_production: "bg-purple-100 text-purple-800",
12   shipped: "bg-indigo-100 text-indigo-800",
13   completed: "bg-green-100 text-green-800",
14   cancelled: "bg-red-100 text-red-700",
15 };
16
17 const label = (s) => s.replace("_", " ");
18 const money = (n) => `$${Number(n || 0).toFixed(2)}`;
```

- **стр. 5** — кнопки фильтра (с «all»).
- **стр. 6** — варианты в выпадающем списке статуса (без «all»). Совпадают с `enum` в `orderModel.js`.
- **стр. 8–15** — цвет метки для каждого статуса.
- **стр. 17** — **функция `label(s)`** — `in_production` → `in production` (подчёркивание на пробел).
- **стр. 18** — **функция `money(n)`** — `$50.00`; `n || 0` — если пусто, 0.

```
20 const StatCard = ({ label, value }) => {
21   return (
22     <div className="border border-black/10 p-6">
23       <p ...>{label}</p>
24       <p ...>{value}</p>
25     </div>
26   );
27 };
```

- **Компонент** `StatCard` (стр. 20–27) — карточка статистики: подпись и число. Используется на стр. 134–136.

```
29 const ManagerOrdersPage = ({ token }) => {
30   const [orders, setOrders] = useState([]);
31   const [loading, setLoading] = useState(true);
32   const [filter, setFilter] = useState("all");
33   const [search, setSearch] = useState("");
34   const [openId, setOpenId] = useState(null);
35   const [message, setMessage] = useState(null);
36   const [stats, setStats] = useState(null);
37   const [period, setPeriod] = useState("30");
```

- **стр. 30–37** — заказы; загрузка; выбранный статус; текст поиска; id раскрытого заказа; сообщение; статистика; период статистики (7 / 30 / all, по умолчанию 30 дней).

```
39   // Reads the orders, filtered by status and by the search box.
40   const load = () => {
41     setLoading(true);
42
43     const params = new URLSearchParams();
44     if (filter !== "all") params.set("status", filter);
45     if (search.trim()) params.set("search", search.trim());
46
47     api
48       .get(`/api/orders?${params}`, authHeader(token))
49       .then((response) => {
50         setOrders(response.data.data);
51         setLoading(false);
52       })
...
58   };
59
60   useEffect(() => {
61     load();
62     // eslint-disable-next-line react-hooks/exhaustive-deps
63   }, [filter]);
```

- **стр. 40–58** — **функция** `load()` — загрузить заказы. Вызывается эффектом (стр. 61) и формой поиска (стр. 161).
- **стр. 43** — `URLSearchParams` — встроенный объект для сборки строки параметров.
- **стр. 44–45** — добавляем статус и поиск, если заданы. Получается `status=paid&search=anna`.
- **стр. 48** — `GET /api/orders?status=paid&search=anna` с токеном → `getAllOrders`.
- **стр. 50** — сохраняем список.

- **стр. 60–63** — перезагружать при каждой смене фильтра. Комментарий на стр. 62 — указание линтеру не требовать `load` в зависимостях.

```
65   // The small summary boxes at the top. If this call fails the page still
66   // works, so the error is only written to the console.
67   const loadStats = () => {
68     const query = period === "all" ? "" : `?days=${period}`;
69
70     api
71       .get(`/api/orders/stats${query}`, authHeader(token))
72       .then((response) => {
73         setStats(response.data.data);
74       })
...
78   };
79
80   useEffect(() => {
81     loadStats();
82     // eslint-disable-next-line react-hooks/exhaustive-deps
83   }, [period]);
```

- **стр. 67–78** — **функция** `loadStats()` — статистика. **стр. 68** — «all» — без параметра, иначе `?days=7` или `?days=30`. **стр. 71** — `GET /api/orders/stats` → `getStats`.
- **стр. 80–83** — перезагружать при смене периода.

```
85   // Moves one order along the flow: pending -> paid -> in production -> ...
86   const changeStatus = (id, status) => {
87     setMessage(null);
88
89     api
90       .patch(`/api/orders/${id}/status`, { status }, authHeader(token))
91       .then((response) => {
92         // Put the updated order back in the list without reloading everything.
93         setOrders((prev) => prev.map((order) => (order._id === id ? response.data.data : order)));
94         setMessage({ type: "success", text: "Status updated" });
95       })
...
100  };
```

Функция `changeStatus(id, status)` — сменить статус. Вызывается выпадающим списком (стр. 262).

- **стр. 90** — `PATCH /api/orders/:id/status` с телом `{ status }` → `updateOrderStatus`.
- **стр. 93** — в списке заменяем только этот заказ ответом сервера, остальные оставляем — без повторной загрузки всего списка.

Разметка (стр. 102–278):

- **стр. 104–111** — чёрная шапка «Orders».
- **стр. 114–151** — статистика (только если `stats` пришла): кнопки периода 7 / 30 / All (`setPeriod(v)`); три карточки `StatCard` — Total Sales, Orders, Avg Order; список Top products (до 5 строк: название, сколько продано, на какую сумму).
- **стр. 153–174** — ссылка «← Products» на `/admin` и форма поиска: **стр. 161** `onSubmit={(e) => { e.preventDefault(); load(); }}` — по Enter или кнопке Search загрузить заново; поле поиска меняет `search`.

- **стр. 176–188** — кнопки фильтра по статусу: `STATUSES.map` → `setFilter(s)` (эффект перезагрузит список).
- **стр. 190–194** — сообщение.
- **стр. 196–199** — загрузка или «No orders found.».
- **стр. 201–274** — `orders.map((o) => ...)` — заказ:
  - **стр. 204–217** — строка-кнопка: клик раскрывает или сворачивает (`setOpenId(openId === o._id ? null : o._id)`). В строке номер, имя, сумма, цветная метка статуса (`STATUS_STYLE[o.status]`), дата (`new Date(o.createdAt).toLocaleDateString()` — дата в формате браузера);
  - **стр. 219–270** — подробности, только у раскрытого: покупатель (имя, почта, телефон); получение (самовывоз или адрес, `filter(Boolean).join(", ")` — без пустых частей); комментарий; позиции (название, для кастомной сумки — параметры, ×количество, сумма); итог;
  - **стр. 258–269** — выпадающий список статуса: `value={o.status}`, `onChange={(e) => changeStatus(o._id, e.target.value)}`; варианты из `FLOW`.

Часть 17. Сайт: юридические страницы

17.1 PrivacyPolicyPage.jsx и TermsOfServicePage.jsx

Кто подключает: `App.jsx` (адреса `/privacy-policy` и `/terms-of-service`). Сюда ведут ссылки в подвале. **Что подключает:** только React. Логики нет, с сервером не общаются.

Обе устроены одинаково:

- **стр. 2–28** (в Privacy) — массив `sections`: объекты `{ title, body }` — заголовок раздела и его текст.
- **стр. 30** — компонент.
- **стр. 34–39** — заголовок страницы и дата обновления.
- **стр. 41–52** — `sections.map((s) => ...)` — для каждого раздела `<section>` с заголовком `h2` и абзацем. `key={s.title}` — заголовки уникальны.

Тексты: Privacy — какие данные собираются, как используются, как хранятся (пароли — только хешем), права пользователя, контакты. Terms — согласие с условиями, товары ручной работы и заказы из конструктора, оплата, доставка и самовывоз, аккаунты.

Часть 18. Что в проекте не используется

Это стоит знать, если спросят «а это зачем?»:

Что	Где	Состояние
`Header.css`	`front/src/shared/ui/`	нигде не подключается, шапка оформлена Tailwind
`react-image-magnifier-lens`	`front/package.json`	установлена, в коде не используется
`express-mongo-sanitize`, `xss-clean`	`back/package.json`	установлены, в `index.js` не подключены
`API`	`front/src/api.jsx`, стр. 14	экспортируется, нигде не импортируется
`GET /api/orders/:id`	`ordersRoute.js`, стр. 24	адрес работает, но ни одна страница его не вызывает
поле `avatar` у пользователя	`userModel.js`, стр. 21	сохраняется и отдаётся, но на сайте не показывается